Journal of Computational Science 82 (2024) 102421

Contents lists available at ScienceDirect

Journal of Computational Science

journal homepage: www.elsevier.com/locate/jocs

Reinforcement Twinning: From digital twins to model-based reinforcement learning

Lorenzo Schena [a,b], Pedro A. Marques [a,c], Romain Poletti [a,d], Samuel Ahizi [a], Jan Van den Berghe [a,e], Miguel A. Mendez [a,*]

[a] von Karman Institute, Rhode-St-Genése, 1640, Belgium
[b] Vrije Universiteit Brussel (VUB), Department of Mechanical Engineering, Elsene, Brussels, 1050, Belgium
[c] Université Libre de Bruxelles, Av. Franklin Roosevelt 50, Brussels, 1050, Belgium
[d] University of Ghent, Sint-Pietersnieuwstraat 41, Ghent, 9000, Belgium
[e] Institute of Mechanics, Materials, and Civil Engineering (iMMC), Université Catholique de Louvain, Louvain-la-Neuve, 1348, Belgium

ARTICLE INFO

Keywords:
Digital twins
System identification
Reinforcement learning
Adjoint-based assimilation

ABSTRACT

The concept of digital twins promises to revolutionize engineering by offering new avenues for optimization, control, and predictive maintenance. We propose a novel framework for simultaneously training the digital twin of an engineering system and an associated control agent. The training of the twin combines methods from adjoint-based data assimilation and system identification, while the training of the control agent combines model-based optimal control and model-free reinforcement learning. The training of the control agent is achieved by letting it evolve independently along two paths: one driven by a model-based optimal control and another driven by reinforcement learning. The virtual environment offered by the digital twin is used as a playground for confrontation and indirect interaction. This interaction occurs as an "expert demonstrator", where the best policy is selected for the interaction with the real environment and "cloned" to the other if the independent training stagnates. We refer to this framework as Reinforcement Twinning (RT). The framework is tested on three vastly different engineering systems and control tasks, namely (1) the control of a wind turbine subject to time-varying wind speed, (2) the trajectory control of flapping-wing micro air vehicles (FWMAVs) subject to wind gusts, and (3) the mitigation of thermal loads in the management of cryogenic storage tanks. The test cases are implemented using simplified models for which the ground truth on the closure law is available. The results show that the adjoint-based training of the digital twin is remarkably sample-efficient and completed within a few iterations. Concerning the control agent training, the results show that the model-based and the model-free control training benefit from the learning experience and the complementary learning approach of each other. The encouraging results open the path towards implementing the RT framework on real systems.

1. Introduction

Mathematical models of engineering systems have always been fundamental for their design, simulation and control. Recent advances in the Internet of Things (IoT), data analytics and machine learning are promoting the notion of 'dynamic' and 'interactive' models, i.e. architectures that integrate and enrich the mathematical representation of a system with real-time data, evolve through the system's life-cycle, and allow for real-time interaction. Such kinds of models are currently referred to as *digital twins*.

Although the specific definition can vary considerably in different fields [1–9], a digital twin is generally seen as a virtual replica of a physical object (or system) capable of simulating its characteristic, functionalities and behaviour *in real-time*.

Besides the obvious need for domain-specific knowledge and engineering, constructing such a replica involves various disciplines, from sensor technology and instrumentation for real-time monitoring to virtual/augmented reality and computer graphics for interfacing and visualization, and data-driven modelling, assimilation and machine learning for model updating and control. This work focuses on the last aspects and the required intersection of disciplines, with a particular emphasis on the need for real-time interaction. Real-time predictions are essential to deploying digital twins for control or monitoring purposes and require fast models. Fast models can be built from (1)

* Corresponding author.
E-mail address: mendez@vki.ac.be (M.A. Mendez).

https://doi.org/10.1016/j.jocs.2024.102421
Received 9 February 2024; Received in revised form 19 June 2024; Accepted 11 August 2024
Available online 14 August 2024
1877-7503/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

macroscopic/lumped formulations, eventually enriched by data-driven 'closure' laws, (2) surrogate/template models, or (3) reduced-order models derived from high-fidelity simulations. In the context of system identifications for control purposes, the first would be referred to as 'white' models; the second would be referred to as 'black box models' while the third falls somewhere in between (see [10] for the palette of grey shades in data-driven models). Regardless of the approach, the main conceptual difference between digital twinning and traditional engineering modelling is the continuous model update to synchronize the twin with the physical system and to provide a *live* representation of its state from noisy and partial observations.

It is thus evident that the notion of digital twin lies at the intersection between established disciplines such as system identification (see [11–13]) and data assimilation (see [14,15]). Both are significantly enhanced (and perhaps more intertwined) by the rapid growth and popularization of machine learning, as reviewed in the following section of this article. Both seek to combine observational data with a numerical model, and both involve an observation phase (known as training or calibration, depending on the field) in which the model is confronted with online data and a 'prediction' phase in which the model is interrogated. However, these disciplines are built around different problems and use different methods.

Data assimilation is mostly concerned with the *state estimation* problem. It is assumed that the model is known (at least within quantifiable uncertainties), but observations are limited and uncertain. These uncertainties are particularly relevant because the system is chaotic and extremely high dimensional (e.g. atmospheric/oceanic models) and thus highly sensitive to the initial conditions from which predictions are requested. The traditional context is weather prediction and climate modelling [16–19], where the goal is to forecast the future evolution of a system from limited observations. The most common techniques are 4D variational assimilation [20–22] and, more recently, Ensemble Kalman methods [23–25]. Much effort is currently ongoing towards methods that combine these techniques (e.g. [26,27].

Nonlinear system identification is mostly concerned with the *model identification* problem. The system is subject to external inputs and observed through noisy measurements but is deterministic. The model can be fit into structures that are derived either from first principles or from general templates such as Nonlinear AutoRegressive models with exogenous inputs (NARX, [28]) or Volterra models [10], and the main goal is to predict how the system responds to actuation. The traditional context is control engineering, i.e. driving the system towards a desired trajectory. Besides the stronger focus on input–output relation (and thus the focus on actuated systems), the main difference with respect to traditional data assimilation is in the deterministic nature of the system to be inferred: errors in the initial conditions are not amplified as dramatically as, for example, in weather forecasting. Therefore, the resulting uncertainties remain within the bounds of model uncertainties and the state estimation is less critical than in data assimilation. Moreover, in many actuated systems, the forced response dominates over the natural one and the initial conditions are quickly forgotten. Tools for system identification are reviewed by [11,29].

Machine learning is progressively entering both communities with a wealth of general-purpose function approximations such as Artificial Neural Networks (ANNs, see Goodfellow et al. [30]) or Gaussian Processes (GPr, see Rasmussen and Williams [31]). More specifically, the fusion between these disciplines materializes through a combination of variational or ensemble tools from data assimilation to train dynamic models with model structures from machine learning to solve identification, forecasting or control problems. In this regard, the literature in control engineering has significantly anticipated the current developments (see neural controllers in Suykens et al. [29], Norgaard et al. [32]), although their extension to broader literature has been limited by the mathematical challenges of nonlinear identification and the success of much simpler strategies based on piece-wise linearization or adaptive controllers [33,34].

While machine learning enters the literature of data assimilation and system identification, cross-fertilization proceeds bilaterally. The need for identifying dynamic models from data is also growing in the literature on reinforcement learning, which is significantly drawing ideas from system identification and optimal control theory. Reinforcement learning is a subset of machine learning concerned with the training of an agent via trial and error to achieve a goal while acting on an environment [35,36]. The framework differs from classic control theory because of its roots in sequential decision making: the environment to be controlled and the agent constitute a Markov Decision Process (MDP, [37]) rather than a dynamical system with the usual state-space representation. Model-free approaches solely rely on input–output information and function approximations of the agent's performance measures, while model-based approaches combine these with a model of the environment [38,39]. The model can be used as a forecasting tool, thus playing the same role as in model predictive control, or as a playground to accelerate the learning [40]. Model-free approaches have gained burgeoning popularity thanks to their success in video games [41,42] or natural language processing [43], for which the definition of a system model is cumbersome. However, these algorithms have shown significant and somewhat surprising limitations for engineering problems governed by simple PDEs [44,45]. This article proposes a framework combining ideas from data assimilation, system identification and reinforcement learning with the goal of: (1) training a digital twin on real-time data and (2) solving a control problem. We refer to this framework as Reinforcement Twinning (RT). The rest of the article is structured as follows. Section 2 provides a concise literature review of related works across the intersected disciplines and identifies the main novelties of the proposed framework. Section 3 presents the general framework and the key definitions, while Section 4 goes into the mathematical details of its implementation. Section 5 introduces three test cases selected for their relevance in the engineering literature and stemming from largely different fields. These are (1) the control of a wind turbine subject to time-varying wind speed, (2) the trajectory control of flapping-wing micro air vehicles (FWMAVs) subject to wind gusts, and (3) the mitigation of thermal loads in the management of cryogenic storage tanks. Although all test cases are analysed using synthetic data that might look somewhat contrived, the main focus is here to show that the model-based and the model-free training can benefit from each other. Section 6 presents the main results; Section 7 closes with conclusions and perspectives.

2. Related work

This section locates the contribution of this work in the broad literature of the intersected fields. We briefly review recent developments in methods combining machine learning with data assimilation (Section 2.1) and system identification (Section 2.2). We then move towards the cross-fertilization in the context of model-based reinforcement learning (Section 2.3) and focus on recent developments on the combination between model-based and model-free reinforcement learning (Section 2.4). Finally, Section 2.5 briefly reports on the main novelties of the proposed approach.

2.1. Machine learning for data assimilation

An extensive overview of the state of the art of data assimilation (DA) is provided by [46] while [47] give an overview of how machine learning (ML) is entering DA. [48] discuss the formal links between DA and ML in a Bayesian framework while [49] analyses the link by building parallelism between time steps in a DA problem and layer labels in a deep ANN. As explored in [24], the formulation of hybrid DA-ML algorithms consists in introducing ML architectures either (1) to complement/correct the forecasting or the observation model or (2) to replace at least one of them. An early attempt at DA-ML hybridization within the first class was proposed by [50], who

used a variational approach to train an ANN that replaces missing dynamical equations. More recent approaches within the first category are proposed by [51] while [52] presents several approaches in both categories. In particular, [51] implements a Recurrent Neural Network (RNN, Madhavan [53]) to correct the forecasting model, while [52] use convolutional neural networks (CNNs) to improve the observations passed to a Kalman filter. Within the second class of hybrid DA-ML methods, [52,54] use a Kalman filter to train the parameters of an ANN that acts as a forecasting model. Outside geophysical sciences, the use of ANNs as a forecasting tool in assimilation frameworks is spreading in economics [55] and epidemiology [56]. Finally, within the literature on data assimilation, several works have combined ML and DA strategies for model identification [57,58].

2.2. Machine learning for system identification and control

Recent reviews on deep learning for system identifications are proposed by [59,60] while earlier reviews are provided by [12,13]. System identification has been historically influenced by developments in machine learning and adopted the use of ANNs for system modelling already in the 90's [61–63]. Early works (see also Suykens et al. [29] and Norgaard et al. [32]) focused on the use of feedforward neural networks (FNNs) or recurrent neural networks (RNNs); these can be seen as special variants of nonlinear autoregressive and nonlinear state-space models. The popularization of ML has significantly enlarged the zoology of ANNs architectures used as model structures for state-spacem odeling. The most popular examples include variants of the RNN, such as long-short-term memory (LSTM) networks [64] and Echo State Networks (ESN, [65]), as well as convolutional neural networks (CNNs, [66]) and the latest developments in neural ODEs (also known as ODE-net, [67]).

Important contributions within the first class of approaches are the works by [59,68,69] who use LSTM networks [64]. These are variants of RNNs, which are very popular in natural language processing and use three gates (forget/input/output) to preserve information over longer time steps, thus allowing for better handling of long-term dependencies. [70] used ESNs, which gained popularity for forecasting in stochastic systems. These are variants of RNNs implementing the reservoir computing formalism, i.e. ANNs in which weights and biases are selected randomly. [71] used CNNs, which gained popularity in image recognition and allow for identifying patterns at different scales, while [58,72] use ODE-nets, recently proposed as a new paradigm for neural state-space modelling. ODE-nets treat the layers of an ANN as intermediate time steps of a multistep ODE solver and are commonly trained using the adjoint method, a ubiquitous tool in data assimilation. An overview of the challenges of learning models of dynamical systems from data has been recently presented by [73], who also propose a "gradual" approach which seeks to build models of gradually increasing complexity.

2.3. Model based reinforcement learning and optimal control

Recent reviews of model-based reinforcement learning (MBRL) are proposed by [74–76]. MBRL uses a model of the environment to guide the agent, with the goal of increasing its sample efficiency (i.e. reducing the number of trials required to learn a task). A model allows the agent to sample the environment with arbitrary state–action values. In contrast, a model-free approach can only sample visited states, eventually stored in a replay buffer to allow multiple re-uses [77,78]. In the reinforcement learning terminology, a model allows the agent to *plan*, giving the possibility to *imagine* the consequences of actions and, more generally, update policy and/or state–action value predictors without interacting with the environment.

MBRL methods can be classified based on how the model is built and how it is used. Concerning the approaches for model constructions, a large arsenal of function approximations has been deployed in the literature. ANNs [79–81], Gaussian Processes (GPr, Deisenroth and Rasmussen [82], Boedecker et al. [83]), time-varying linear models [84] are some examples. Concerning model usage, the simplest approach consists of using it solely to sample the state–action space, while more advanced approaches use the model in the training steps [85,86]. One of the seminal MBRL approaches is the Dyna algorithm proposed by [87]. This uses experience (i.e. trajectories in the state–action space and associated rewards) collected from both environment and model. More sophisticated approaches use the model to "look ahead", i.e. predicting the future evolution of the system under a given policy. A notable example is the Monte Carlo Tree Search in the AlphaGO [88,89] algorithm that reached superhuman performances in the game of Go. Moving from Markov Decision Processes to dynamical systems, the boundaries between MBRL and Model Predictive Control (MPC) become blurred, particularly when the latter uses black-box models (see Hedengren et al. [90], Raw [91] and Nagabandi et al. [81] or Weber et al. [92]).

Several authors have proposed methods to leverage physical insights in the models for MBRL, using physics-informed neural networks [93] as [94] or Deep Lagrangian Networks (DeLaN Lutter et al. [95]) as [96]. A more extreme approach is proposed by [97], who uses 'white-box' models (eventually enhanced as proposed by Lutter et al. [97]) in MBRL. This work shows a traditional physics-based model trained via machine learning methods. The opposite case is provided by [98], who uses an ANN agent trained with adjoint-based optimization. An overview of methods to integrate physics-based models and machine learning is provided by [99,100]. Regardless of its structure, a differentiable model of the environment can be trained using optimal control theory [101].

2.4. The hybridization of model-based and model-free control

The idea of combining model-based and model-free control methods is gaining momentum in the literature of robotics and control and is generally referred to as hybrid learning [102]. Hybrid learning seeks to combine the model-based methods' sample efficiency with the model-free methods' robustness (see Dulac-Arnold et al. [103]). Hybrid approaches differ in how the model-based and model-free policies are chosen or merged. An early idea proposed by [104] consisted of keeping these separate and independent, with procedures known as Hierarchical Mixtures of Experts (HME) [105,106] used to identify the best of the two at a given state. More recently, a stronger interaction was proposed by [107], who used a model-based policy to bound a model-free policy, with the goal of guaranteeing the stability of the controller even during the exploration. A similar approach is developed in the context of continuous reinforcement learning [108–110] by [102]. Alternatively, if models are cast in a locally linear or linear-quadratic form, iterative optimal control methods [111,112] have been used to generate a pool of "expert" local policies from which a model-free learner must generalize and improve upon [113–116], or to warm-start a model-free policy search step [117]. [118] proposed an approach decoupling the policy evaluation and policy improvement steps, thereby developing a hybrid method in which one step is carried out in a model-free approach and the other proceeds in a model-based approach.

An alternative path is to obtain a hybridization in the value function space. Value functions aim to predict each state's value and quantify how well a controller can perform from a given state of the system; value function approximators can thus be used to guide optimal controllers as proposed by [119]. Models can also be used to generate roll-outs to enhance model-free value estimation methods [120], potentially weighting the collected data according to the trust in the model at hand [121]. Other combinations propose the use of control-theoretic tools to provide guarantees on model-free methods. An example can be found in [122], in which results on Lyapunov stability theory are used to identify safe regions in the state-space [123] in which the controller is constrained to operate. In addition, if considerations on robustness are also made, this effectively leads to an H_∞ model-free RL approach [124].

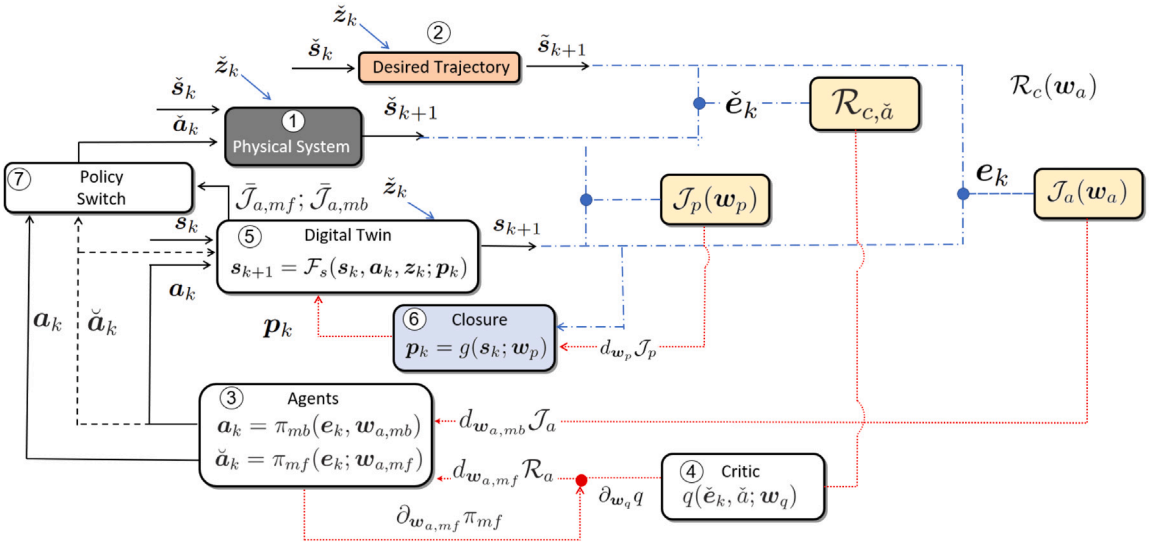


Fig. 1. Structure of the proposed approach to blend assimilation, model-based and model-free control. We use one control agent, acting on the physical system and its digital twin, and we combine the learning from a model-free and a model-based approach. The dashed blue lines are used to track the distance measures, while the red dotted lines track the updates of the parameters involved in the training. The full algorithmic implementation is provided in Algorithm 1.1.

2.5. Novelties of the proposed approach

The RT framework proposed in this work is a hybrid learning approach in which the model-based part is based on optimal control and adaptive modelling using methods from data assimilation. The first novelty within the landscape of model-based methods is the use of physics-based models with closure laws adapted in real-time: the derived model allows for real-time interaction and adapts to the system evolution, becoming a digital twin of the real system. Although the model is here trained using an ensemble adjoint-based approach to compute the gradient of the cost function driving the model derivation, variants of the algorithm using Ensemble Kalman filtering are possible and currently being explored. Departing from traditional data assimilation, we assume that all states are observable and the underlying dynamic is deterministic. Moreover, since the main role of the model is to provide input–output relations in an actuated system, the approach is closer to system identification than traditional data assimilation.

The second novelty within the land-scale of hybrid learning is in the strong interaction between the model-based and the model-free control agents, which alternate the role of disciple and master. A system similar to HME decides which policy is promoted to master and used on the real system. However, both are continuously trained in parallel, using the digital twin as a playground and sharing the collected expertise. We show that these agents can learn from each other via different mechanisms.

3. Definitions and general formulation

The main components of the RT approach implemented in this work are illustrated with the aid of the schematic in Fig. 1. All components are numbered following the presentation below.

We consider a dynamical system (component 1) with states $\check{\boldsymbol{s}} \in \mathbb{R}^{n_s}$ evolving according to *unknown* dynamics. We treat this system as a black box and consider it the *real* environment. We assume that these states can be sampled (measured) at uniform time intervals $t_k = k\Delta t$ over an observation time T_o to collect the sequence $\check{S} = [\check{\boldsymbol{s}}_1, \dots \check{\boldsymbol{s}}_{n_t}]$ with $n_t = T_o/\Delta t + 1$. All variables are sampled in the same way, and we use subscripts to identify the sample at specific time steps.

The real system evolves under the influence of control actions $\check{\boldsymbol{a}}_k$, provided by an agent, and exogenous (uncontrollable) inputs $\check{\boldsymbol{z}}_k = \check{\boldsymbol{d}}_k + \check{\boldsymbol{\epsilon}}_k \in \mathbb{R}^{n_z}$. We assume that these consist of a 'large' scale ($\check{\boldsymbol{d}}_k$) and a 'small scale' ($\check{\boldsymbol{\epsilon}}_k$) contribution, but neither of the two can be forecasted. These are treated as random processes with different integral time scales. With no loss of generality, we assume that observations and actions occur simultaneously. Therefore, in the reinforcement learning terminology, the collection of n_t observations/action pairs within the observation time T_o defines an *episode*.

The actions seek to keep the system along a desired trajectory $\tilde{S} = [\tilde{\boldsymbol{s}}_1, \dots, \tilde{\boldsymbol{s}}_{n_t}]$ (component 2) and we introduce the *real error* $\check{\boldsymbol{e}}_k = \check{e}(\check{\boldsymbol{s}}_k, \tilde{\boldsymbol{s}}_k) \in \mathbb{R}^{n_s}$ to measure the discrepancy between the current and the target states. The control agent (component 3) acts according to a *policy* which takes the form of a parametric function $\check{\boldsymbol{a}}_k = \pi(\check{\boldsymbol{e}}_k; \boldsymbol{w}_a)$ in the weights $\boldsymbol{w}_a \in \mathbb{R}^{n_\pi}$. The control problem involves identifying the weights $\boldsymbol{w}_a$ through an iterative "learning process" while the agent interacts with the system. This process is carried out by evolving two policies in parallel: the best one is promoted to "live" policy and interacts with the real system while the other becomes "idle" and continues to be trained in the background.

One of these policies evolves according to model-free strategy. It is denoted as π_{mf} and is defined by the set of parameters $\boldsymbol{w}_{a,mf}$. The other evolves according to a model-based strategy. It is denoted as π_{mb} and is defined by the set of parameters $\boldsymbol{w}_{a,mb}$. The decision of which of these becomes "live" or "idle" is taken by policy switch (element 7 in Fig. 1) that selects either $\boldsymbol{w}_a = \boldsymbol{w}_{a,mf}$ or $\boldsymbol{w}_a = \boldsymbol{w}_{a,mb}$ depending on which of the two is producing the best results on the virtual environment and the digital twin (component 5).

The working principle of this policy switch is described in Section 4.3. What follows is a description of the model-free and the model-based training, treated as independent strategies.

- The model-free policy (π_{mf}) training relies on classic model-free reinforcement learning. The optimal weights $\boldsymbol{w}_{a,mf}$ are defined according to a reward function, which measures how well the system is kept close to the desired trajectory. This evaluation is made *online*, updating the policy after each episode. The optimal weights maximize this function, which is written as the summation of instantaneous rewards $r_k = r(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k) \in \mathbb{R}$, possibly discounted by a factor γ:

$$\mathcal{R}_c(\boldsymbol{a}, \boldsymbol{e}) = \sum_{k=0}^{n_t-1} \gamma^k r_k(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k)\,. \tag{1}$$

The definition of the instantaneous reward function is problem-dependent and discussed in the following sections. As described in Section 4.1, the model-free loop implemented in this work

is an actor-critic algorithm, which relies on a parametric approximation of the state–action value function $Q(\check{\boldsymbol{e}}, \check{\boldsymbol{a}}; \boldsymbol{w}_q)$ in the parameters $\boldsymbol{w}_q$. This estimates the expected reward for an action-state pair and is essential in computing the update of the policy parameters $\boldsymbol{w}_a$. This parametric function is referred to as 'critic' (component 4) and is essential in computing the gradient $d_{\boldsymbol{w}_{a,mf}} \mathcal{R}_c$ driving the training of the agent from the model-free side (see connections in dashed red in Fig. 1). We highlight that the use of the error state $\boldsymbol{e}$ rather than the state $\boldsymbol{s}$ in the reward and Q functions is here justified by our focus on tracking problems.

- The model-based policy (π_{mb}) optimization loop combines system identification and optimal control. System identification seeks to continuously adapt and improve the digital twin (component 5). This is a model of the system which evolves the 'virtual states' $\boldsymbol{s}_k$ to $\boldsymbol{s}_{k+1}$, subject to the same exogenous inputs $\check{\boldsymbol{z}}_k$ as the physical system and under the 'virtual actions' $\boldsymbol{a}_k$. This solver advances the digital twin in real time and restarts at the end of each episode.
The model relies on unknown parameters $\boldsymbol{p}_k \in \mathbb{R}^{n_p}$, provided by a closure parametric function $\boldsymbol{p}_k = g(\boldsymbol{s}_k; \boldsymbol{w}_p)$ (component 6) in the weights $\boldsymbol{w}_p \in \mathbb{R}^{n_g}$. This closure function embeds unknown physics or terms that would render the prediction overly expensive. As we shall see in Section 5, these parameters can be classic empirical coefficients (e.g. aerodynamic coefficients or heat/mass transfer coefficients) of a carefully crafted physics-based model of the system. The forward step of the system is driven by an ODE solver, which defines the function $\boldsymbol{s}_{k+1} = \mathcal{F}_s(\boldsymbol{s}_k, \boldsymbol{a}_k, \boldsymbol{z}_k; \boldsymbol{p}_k)$.
The system identification consists in finding the optimal weights $\boldsymbol{w}_p$ according to a cost function $\mathcal{J}_p(\boldsymbol{w}_p)$ which measures how close the virtual trajectory $S = [\boldsymbol{s}_1, \dots \boldsymbol{s}_{n_t}]$ resembles the real one. The optimal weights $\boldsymbol{w}_p$ minimize this function, defined as

$$\mathcal{J}_p(\boldsymbol{w}_p) = \mathbb{E}_{\sim S(\boldsymbol{z}, \boldsymbol{w}_p)} \left\{ \int_0^{T_o} \mathcal{L}_p\left(\check{\mathbf{s}}(t_k), \boldsymbol{s}(t; \boldsymbol{w}_p)\right) \, \mathrm{dt} \right\}, \tag{2}$$

with $\mathcal{L}_p : \mathbb{R}^{n_s \times n_s} \to \mathbb{R}$ the Lagrangian function for the identification problem and the expectation $\mathbb{E}_{\sim S(\boldsymbol{z}, \boldsymbol{w}_p)}\{\}$ over possible virtual trajectories accounting for the stochastic exogenous inputs. More specifically, starting from one sample signal $\boldsymbol{z}_k$, with $k = 0, n_t - 1$, we build a set of N_z realizations of exogenous inputs as described in Section 4.2 and we denote as $S(\boldsymbol{z}, \boldsymbol{w}_p)$ the set of trajectories generated for a given sample of the exogenous input $\boldsymbol{z}$ and a given set of weights $\boldsymbol{w}_p$. Two reasons motivate the choice for the integral formulation in (2) even if only a discrete set of samples $\hat{\mathbf{s}}(t_k)$ is available. First, the integral is particularly handy in deriving the adjoint problem driving the computation of the gradient $d\mathcal{J}/d\boldsymbol{p}$, as presented in Section 4.2. Second, the choice keeps the approach independent from the specific numerical solver to advance the virtual states $\boldsymbol{s}(t)$. We return to both points in Section 4.3. We here stress that the closure model $g(\boldsymbol{s}; \boldsymbol{w}_g)$ makes the digital twin predictive only for a given sample of the exogenous disturbance signal and within $t \in [0, T_o]$, i.e. no forecasting is considered.
The model-based control seeks to find the optimal policy to keep the digital twin along the desired trajectory. We introduce the *virtual error* $\boldsymbol{e}(t_k) = e(\boldsymbol{s}_k, \check{\mathbf{s}}_k)$ to measure the distance between the current virtual state and the target state and $\boldsymbol{a}_{mb} = \pi_{mb}(\boldsymbol{e}; \boldsymbol{w}_{a,mb})$ the *virtual actions* taken on the digital twin. The performances of the controller are measured by the cost function $\mathcal{J}_a(\boldsymbol{w}_{a,mb})$. For convenience, we define this cost function similarly to the one in (2), i.e.:

$$\mathcal{J}_a(\boldsymbol{w}_{a,mb}) = \mathbb{E}_{\sim S(\boldsymbol{z}, \boldsymbol{w}_{a,mb})} \left\{ \int_0^{T_o} \mathcal{L}_a\left(\tilde{\mathbf{s}}(t), \boldsymbol{s}(t; \boldsymbol{w}_{a,mb})\right) \, \mathrm{dt} \right\}, \tag{3}$$

with $\mathcal{L}_a : \mathbb{R}^{n_s} \to \mathbb{R}$ the Lagrangian function for the model-based control problem. The same notation as in (2) applies. The optimal control weights for the model-based approach are those that minimize this function. The process of computing a policy update using the virtual environment (model-based loop) without interacting with the real system is here referred to as *offline (policy) planning*.
It is worth stressing that the weights $\boldsymbol{w}_{a,mb}$ minimizing $\mathcal{J}_a$ in (3) do not necessarily maximize $\mathcal{R}_c$ in (1). These problems will not have the same solution unless the virtual environment closely follows the real one. The way these loops interact in the agent's training is described in the following section, along with more details on the computation of the gradient $d\mathcal{J}_p/d\boldsymbol{w}_p$ driving the identification and the gradient $d\mathcal{J}_a/d\boldsymbol{w}_{a,mb}$ driving the agent's training from the model-based side (see connections in dashed red in Fig. 1).

4. Mathematical tools and algorithms

We here detail the building blocks of the proposed approach. Only the general architecture is discussed, leaving customizations to specific environments to Section 5. The model-free paradigm (connection of components 1-2-3-4) is illustrated in Section 4.1. This leverages standard Deep Reinforcement Learning (DRL) methodologies. The model-based paradigm (connections of components 1-2-3-5-6) is illustrated in Section 4.2. This combines adjoint-based nonlinear system identification and adjoint-based optimal control. The model-free and model-based loops could be used independently, and we first introduce their working principle independently. Then, Section 4.3 details how these are connected and presents the proposed algorithm.

4.1. The model-free loop: (1)-(2)-(3)-(4)

We consider a classic actor-critic reinforcement learning formalism (Sutton and Barto [35], Bhatnagar et al. [125]). Although more recent variants exist, we here focus on the classic Deep Deterministic Policy Gradient (DDPG) algorithm proposed by [78] with the prioritized experience replay strategy by [126]. This algorithm offers a good compromise between performance and simplicity, and it has been proven successful in several flow control tasks [45,127].

The DDPG trains two functions while interacting with the system. The first, referred to as *actor*, aims to approximate the policy function $\pi_{mf} : \check{\boldsymbol{e}} \to \check{\boldsymbol{a}}$, which provides the action to be taken at a certain state. This policy is deterministic, but we add noise generated by a random process to explore the action space.

The second, referred to as *critic*, aims to approximate the state action value function $Q : (\check{\boldsymbol{e}}, \check{\boldsymbol{a}}) \to \mathbb{R}$; this provides the expected reward for taking action $\check{\boldsymbol{a}}$ while the system is in the error state $\check{\boldsymbol{e}}$ and then following a certain policy.

We use Q to denote the (unknown) true Q value and $q(\check{\boldsymbol{e}}, \check{\boldsymbol{a}}; \boldsymbol{w}_q)$ to denote the approximation from the $Q-$network. In line with the previous section and Fig. 1, the parameters $\boldsymbol{w}_{a,mf}$ denote the weights and biases of the actor-network while $\boldsymbol{w}_q$ denotes the weights and biases for the critic network. These are identified through an online optimization process, which is based on stochastic, momentum-accelerated gradient-based optimization as in classic deep learning (see [30]). The gradient driving the optimization of the policy reads (see [128]):

$$\frac{d\mathcal{R}_c}{d\boldsymbol{w}_a} = \mathbb{E}_{\sim \check{S}} \left\{ \frac{dq(\check{\boldsymbol{e}}, \check{\boldsymbol{a}}; \boldsymbol{w}_q)}{d\boldsymbol{a}} \frac{d\pi(\check{\boldsymbol{e}}; \boldsymbol{w}_a)}{d\boldsymbol{w}_a} \right\}, \tag{4}$$

where $\mathcal{R}_c$ is the reward function in (1).

The expectation is evaluated on an ensemble of trajectories or transitions and is readily available using back-propagation on the actor and the critic networks. Its accuracy strongly relies on the accuracy of the critic network. The recursive nature of the state value action function allows for measuring the performances of the critic from the Bellman equation [35]. For the true Q function, this reads:

$$Q(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k) = \mathbb{E}_{\sim s_{k+1}} \left\{ r_k + \gamma Q(\check{\boldsymbol{e}}_{k+1}, \boldsymbol{a}^{\pi}_{k+1}) \right\}. \tag{5}$$

where $r_k = r(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k)$. The expectation over possible future states accounts for the stochastic nature of the exogenous inputs, and $\boldsymbol{a}^{\pi}$ denote

actions according to the current policy π (i.e. $\check{\boldsymbol{a}}_k^{\pi} = \pi(\check{\boldsymbol{e}}_k; \boldsymbol{w}_a)$). The same recursive form should then exist for the q network. Therefore, defining

$$y_k = r_k + \gamma\ q(\check{\boldsymbol{e}}_{k+1}, \pi(\check{\boldsymbol{e}}_{k+1}); \boldsymbol{w}_q) \tag{6}$$

the expected state–action value at step k, and as $\delta_k = y_k - q(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k)$ the Temporal Difference (TD) error, a cost function for training the critic network can be written as

$$\mathcal{J}_q(\boldsymbol{w}_q) = \mathbb{E}_{\sim s_k, s_{k+1}}\left\{\delta_k^2\right\}, \tag{7}$$

where the expectation $\mathbb{E}_{\sim s_k, s_{k+1}}$ is computed over a set of transitions from one state to the following (regardless of their position in the system trajectory).

The gradient of (7) with respect to $\boldsymbol{w}_q$ can be easily computed using back-propagation on the critic network. However, the training of the critic network is notoriously unstable [129,130]. The two classic approaches to stabilize the learning consist in (1) using an under-relaxation in the update of the weights and (2) keeping track of a large number of transitions by using a replay buffer. The implementation of both methods is discussed in Section 4.3.

Finally, the sampling from the buffer is carried out in the form of batches to evaluate the gradient $d\mathcal{J}_q/d\boldsymbol{w}_q$. The sampling prioritizes transitions with the largest TD-error, δ_k, [131] building a non-zero probability distribution for each sample proportional to it, i.e. $p_i = |\delta_i| + \mathrm{tol}$, with tol a small offset. This yields to the sampling probability of the $i-$th transition, $P(i)$ to be expressed as:

$$P(i) = \frac{p_i^{\alpha^*}}{\sum_k p_k^{\alpha^*}}, \tag{8}$$

in which α^* sets the prioritization influence, with $\alpha^* = 0$ implying uniform sampling.

4.2. The model-based loop (1)- (2)- (3)-(5)-(6)

The assimilation and model-based control problem in the loop (1)-(2)-(3)-(5)-(6) share the same mathematical framework. The dynamical system describing the evolution of the digital twin is written as

$$\begin{cases} \dot{\boldsymbol{s}} & = f(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{z}, \boldsymbol{p}) \\ \boldsymbol{s}(0) & = \boldsymbol{s}_0 \\ \boldsymbol{a} & = \pi_{mb}(\boldsymbol{e}, \boldsymbol{w}_{a,mb})\,;\ \boldsymbol{p} = g(\boldsymbol{s}, \boldsymbol{w}_p) \end{cases}, \tag{9}$$

where $f : \mathbb{R}^{n_s \times n_a \times n_p \times n_z} \to \mathbb{R}^{n_s}$ is the flow map of the dynamical system and is here assumed to be known.

This function could be derived from first principles, as described in Section 5 for the selected test cases, or using a general-purpose function approximator such as ANNs. The distinction is irrelevant to the method presented in this section, and we note in passing that using an ANN to approximate f in combination with the training techniques described in this section leads to neural ODEs [67].

The problem of *training* the agent and the closure law consists in finding the optimal parameters $\boldsymbol{w}_a$ and $\boldsymbol{w}_p$, respectively, using continuously provided data. The training data consists of a target trajectory $\tilde{\boldsymbol{s}}(t)$ for the control problem and measurements of the states $\check{\boldsymbol{s}}(t)$ for the identification problem. The performances of the assimilating agent are measured according to (2) and (3), respectively (see Fig. 1).

These functions are interconnected because they depend on the states, which depend on both the closure variables and the actions (thus both on $\boldsymbol{w}_a$ and $\boldsymbol{w}_p$). However, we here consider these independently: $\boldsymbol{w}_p$ should be identified to minimize $\mathcal{J}_p$ regardless of the performance of the controlling agent, and $\boldsymbol{w}_a$ should be identified to minimize $\mathcal{J}_a$ regardless of the performances of the assimilating agent. This is why only the relevant functional dependency is considered in Eqs. (2) and (3).

To maximize the sample-efficiency training strategy, the optimization of these functions is carried out using gradient-based optimization along with efficient computation of the gradients $d\mathcal{J}_p/d\boldsymbol{w}_p$ and $d\mathcal{J}_a/d\boldsymbol{w}_a$ using the adjoint method [132–134]. This method bypasses the need for computing the sensitivities $d\boldsymbol{s}/d\boldsymbol{p}$ and $d\boldsymbol{s}/d\boldsymbol{a}$. Moreover, because of the similarity in the cost functions (2) and (3), the adjoint problems share key terms. Concerning the gradient driving the system identification, the adjoint-based evaluation gives

$$\frac{d\mathcal{J}_p}{d\boldsymbol{p}} = \mathbb{E}_{\sim S(\boldsymbol{z}, \boldsymbol{w}_p)}\left\{\int_0^{T_o} \lambda_p^T(t) \frac{df}{d\boldsymbol{p}} \frac{dg}{d\boldsymbol{w}_p}\ \mathrm{dt}\right\}, \tag{10}$$

where the expectation operator is the same used for the cost function definition in (3), the gradient $dg/d\boldsymbol{w}_p$ is readily available from the closure definition and $\lambda_p \in \mathbb{R}^{n_s}$ is the vector of Lagrange multipliers (co-states) computed from the terminal value problem:

$$\begin{cases} \dot{\lambda}_p & = -\left(\frac{df}{ds}\right)^T \lambda_p - \left(\frac{d\mathcal{L}_p}{ds}\right)^T \\ \lambda_p(T_o) & = \boldsymbol{0}\,. \end{cases} \tag{11}$$

Similarly, the computation of the gradient $d\mathcal{J}_a/d\boldsymbol{p}$ driving the training of the controlling agent is:

$$\frac{d\mathcal{J}_a}{d\boldsymbol{a}} = \mathbb{E}_{\sim S(\boldsymbol{z}, \boldsymbol{w}_a)}\left\{\int_0^{T_o} \lambda_a^T(t) \frac{df}{d\boldsymbol{a}} \frac{d\pi}{d\boldsymbol{w}_a}\ \mathrm{dt}\right\}, \tag{12}$$

where the gradient $d\pi/d\boldsymbol{w}_a$ is available from the policy definition (e.g. via backpropagation if this is an ANN) and the evolution of the Lagrange multiplier $\lambda_a(t) \in \mathbb{R}^{n_s}$ is determined by the terminal problem:

$$\begin{cases} \dot{\lambda}_a & = -\left(\frac{df}{ds}\right)^T \lambda_a - \left(\frac{d\mathcal{L}_a}{ds}\right)^T \\ \lambda_a(T_o) & = \boldsymbol{0}\,. \end{cases} \tag{13}$$

In both problems, one has one adjoint state evolution $\lambda_p(t)$ and $\lambda_a(t)$ for each of the N_z trajectories used to evaluate the expectation in (10). The methodology to compute the population of trajectories is described in the remainder of this section.

Given a sample of the temporal evolution of the exogenous inputs $\boldsymbol{z}_k = \boldsymbol{z}(t_k)$, with $k = [0, n_t - 1]$, we build an analytic approximation of its large-scale component $\boldsymbol{d}_k = \boldsymbol{d}(t_k)$ using Support Vector Regression (SVR, [135]). Let $\boldsymbol{d}_k[j] \in \mathbb{R}$ denote the jth component of the exogenous input sampled at time step k, with $j = [0, n_z - 1]$. We seek to derive a continuous function $\tilde{\boldsymbol{d}}[j](t)$ such that $\tilde{\boldsymbol{d}}[j](t_k) \approx \boldsymbol{d}_k[j]$. This is a regression problem that we tackle with support vector regression. Hence, the continuous function is written as

$$\tilde{\boldsymbol{d}}[j](t; \boldsymbol{\alpha}[j]) = \sum_{k=0}^{n_t - 1} \kappa(t_k, t) \alpha[j]_k \tag{14}$$

with $t \in \mathbb{R}$, $\kappa(t_1, t_2) = \exp(-\gamma \|t_1 - t_2\|_2^2)$ a Gaussian kernel and γ a user defined scale parameter. We stress that $t \in [0, T_o]$ since no forecasting attempt is made. The vector of coefficients $\boldsymbol{\alpha}[j] = [\alpha[j]_1, \ldots \alpha[j]_{n_t}]$ is computed by minimizing the cost function (see Chang and Lin [136])

$$J(\boldsymbol{\alpha}[j]) = \frac{1}{2}\|\boldsymbol{\alpha}[j]\|_2^2 + C\left(\sum_{k=0}^{n_t - 1} \varepsilon_j + \varepsilon_j^*\right), \tag{15}$$

subject to $\boldsymbol{z}[j](t_k) - \tilde{\boldsymbol{d}}[j](t_k, \boldsymbol{\alpha}) < \epsilon + \varepsilon_j$, $\boldsymbol{z}[j](t_k) - \tilde{\boldsymbol{d}}[j](t_k, \boldsymbol{\alpha}) > \epsilon + \varepsilon_j$ and $\varepsilon_j \varepsilon_j^* > 0$ for all j in $[1, n_t]$. The coefficients C and ϵ are user-defined parameters controlling the regularization of the regression.

The regression model generates N_z samples of possible exogenous disturbances. These are modelled as a set of n_z Gaussian processes with mean $\tilde{\boldsymbol{d}}[j](t; \boldsymbol{\alpha}[j])$ and covariance matrices $\boldsymbol{\Sigma}_\epsilon[j]$. We sample these processes N_z times to obtain the exogenous disturbances in the virtual trajectories. An example of SVR regression is shown in Fig. 2 together with $N_z = 10$ random samples of the modelled exogenous disturbance.

4.3. The proposed RT algorithm

The proposed algorithm is illustrated in 1.1. It consists of six steps, repeated over n_e episodes. These are described in the following.

Algorithm 1.1: Reinforcement Twinning (RT) Algorithm (Part 1)

1 Initialize model-free $\pi_{mf}(\boldsymbol{w}_{a,mf}^{(0)})$ and model-based policies $\pi_{mb}(\boldsymbol{w}_{a,mb}^{(0)})$ and set live policy $\pi = \pi_{mf}$
2 Initialize critic network with weights $\boldsymbol{w}_q^{(0)}$ and $\boldsymbol{w}_q^{(0)}$ and targets $\boldsymbol{w}'_q$
3 Initialize closure function's weights $\boldsymbol{w}_p^{(0)}$
4 Initialize long term memory buffer $\mathcal{D}_\mathrm{L}$ and short term memory $\mathcal{D}_\mathrm{S}$
5 Initialize Step 6 counters and thresholds C_π, P_π, C_w, P_w, T_J and T_w
6 **for** $i = 1, \dots, n_e$ **do**
7 ▷ Step 1: Run a real episode
for $k = 1, ..., n_t$ **do**
8 Sample $\xi(i)$; if $\xi(i) \neq 0$, sample a random action $\boldsymbol{n}_k$ from $\mathcal{N}_n(\mu_a, \Sigma_a)$
9 Select action according to policy and exploration noise, $\check{\boldsymbol{a}}_k = \pi(\check{\boldsymbol{e}}_k; \boldsymbol{w}_a^{(i)}) + \xi(i)\boldsymbol{n}_k$
10 Act on real system, then collect reward r_k, states $\check{\boldsymbol{s}}_{k+1}$ and tracking error $\check{\boldsymbol{e}}_{k+1}$
11 Store error transitions $(\check{\boldsymbol{e}}_k, \check{\boldsymbol{a}}_k, r_k, \check{\boldsymbol{e}}_{k+1})$ and associated TD errors δ_k in $\mathcal{D}_\mathrm{L}$
12 Store state transitions $(\check{\boldsymbol{s}}_k, \check{\boldsymbol{z}}_k, \check{\boldsymbol{a}}_k, \check{\boldsymbol{s}}_{k+1})$ in $\mathcal{D}_\mathrm{S}$
13 **end**
14 ▷ Step 2: Update Critic and compute the model-free policy update
for $n = 1, \dots, n_Q$ **do**
15 Sample $\mathrm{MIN}(n_b, n_t)$ transitions from $\mathcal{D}_\mathrm{L}$ and compute $\mathcal{J}_q$ and $d\mathcal{J}_q/dq$ from (7)
16 Compute $q(\check{\boldsymbol{e}}, \check{\boldsymbol{a}}_k)$ and the update $\Delta\boldsymbol{w}_q^{(i)}$, then update $\boldsymbol{w}_q^{(i)} \leftarrow \boldsymbol{w}_q^{(i)} + \Delta\boldsymbol{w}_q^{(i)}$
17 Use target for τ under-relaxation: $\boldsymbol{w}_q^{(i)} \leftarrow \tau\boldsymbol{w}_q^{(i)} + (1-\tau)\boldsymbol{w}'_q$ and update target $\boldsymbol{w}'_q = \boldsymbol{w}_q^{(i)}$
18 Update the TD errors in the buffer $\mathcal{D}_\mathrm{L}$ for sampled transitions
19 **end**
20 **for** $n = 1, \dots, n_A$ **do**
21 Sample $\mathrm{MIN}(n_b, n_t)$ transitions from $\mathcal{D}_\mathrm{L}$
22 Compute $\mathcal{R}_c(\boldsymbol{w}_a)$ and new set of weights $\boldsymbol{w}_{a,mf}^{(n)} \leftarrow \boldsymbol{w}_{a,mf}^{(n-1)} + \Delta_f \boldsymbol{w}_{a,mf}$ from $d\mathcal{R}_c/d\boldsymbol{w}_{a,mf}$ in (4)
23 Update model-free actor policy: $\pi_{mf}^{(n)} = \pi_{mf}(\boldsymbol{w}_{a,mf}^{(n)})$
24 **end**
25 ▷ Step 3: Prepare inner-episode regressors
26 Use n_S series of $\boldsymbol{z_k}$ from $\mathcal{D}_\mathrm{S}$ use (14) to build $\mathcal{N}_z^{(l)}(\tilde{\boldsymbol{d}}[j](t; \boldsymbol{\alpha}[j]), \Sigma[j])$ with $j \in [1, n_z]$, $l \in [1, n_S]$
27 ▷ Step 4: Optimize digital twin on the $n_s \times N_Z$ virtual episodes with real actions
28 **for** $n = 1, \dots, n_G$ **do**
29 **for** $l = 1, \dots n_S$, *with* $g(\boldsymbol{s}, \boldsymbol{w}_p^{(i)})$ **do**
30 Forward N_z virtual trajectories with $\boldsymbol{s}_0 = \check{\boldsymbol{s}}_0^{(l)}$, $\boldsymbol{a}_k$ from $\check{\boldsymbol{a}}_k^{(l)}$, $\boldsymbol{z} \sim \mathcal{N}_z^{(l)}()$
31 Compute $\mathcal{J}_p(\boldsymbol{w}_p^{(i)})$ in (2) solve N_z adjoint problems for $\lambda_p(t_k)$ for each member
32 Compute $d\mathcal{J}_p/d\boldsymbol{p}$ from the ensemble (see eq. (10)) and the associated updates $\Delta\boldsymbol{w}_p^{(l)}$
33 **end**
34 Compute the update $\boldsymbol{w}_p^{(i)} \leftarrow \boldsymbol{w}_p^{(i)} + \Delta\boldsymbol{w}_p^{(i)}$
35 **end**
36 Save current optimal set of $\boldsymbol{w}_p = \boldsymbol{w}_p^*$
37 …
38 **end**

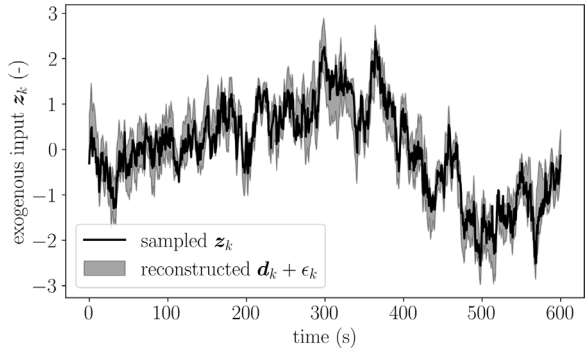


Fig. 2. Example of SVR regression of sampled exogenous input. A filtered exogenous input is passed as an input of the SVR (continuous line) which outputs $N_z = 10$ possible realizations.

- **Step 1: Interaction with the real environment (lines 7–13)**. A sequence of n_t interactions with the real system takes place within an observation time T_o. These episodes are denoted as *real* as opposed to the *virtual* ones carried out on the digital twin. These interactions may have an exploratory phase, carried out by adding a random component ($\boldsymbol{n}_k$) sampled from a random process. We here use a Gaussian process with mean $\boldsymbol{\mu}_a$ and covariance matrix $\boldsymbol{\Sigma}_a$ (line 6). The balance between exploration and exploitation is controlled by the sequence $\xi(i)$, which tends to zero as exploration ends. This sequence could be linked to the performance of the digital twin assimilation, but we leave it as a user-defined sequence for the purposes of this work. During this phase, transitions are stored in the long-term memory buffer $\mathcal{D}_\mathrm{L}$ and the short-term memory buffer $\mathcal{D}_\mathrm{S}$. The first is used to update the critic network

Algorithm 1.2: Reinforcement Twinning (RT) Algorithm (Part 2)

37 ▷ Step 5: Run N_z virtual episodes with current policy and compute model-based policy update ...
38 **for** $n = 1, \dots n_{mb}$ *with* $g(\boldsymbol{s}, \boldsymbol{w}_p^*)$ **do**
39 Forward N_z virtual trajectories with $\boldsymbol{s}_0 = \breve{\boldsymbol{s}}_0^{(l)}$, $\boldsymbol{a}_k = \pi_{mb}(\boldsymbol{s}_k; \boldsymbol{w}_a^{(i)})$, $\boldsymbol{z} \sim \mathcal{N}_z^{(l)}()$
40 Compute $\mathcal{J}_a(\boldsymbol{w}_a^{(i)})$ in (3) and solve N_z adjoint problems to get $\lambda_a(t_k)$ for each member
41 Compute $d\mathcal{J}_a/d\boldsymbol{a}$ from the ensemble (see eq. (12)
42 Compute the new set of weights $\boldsymbol{w}_{a,mb}^{(n)} \leftarrow \boldsymbol{w}_{a,mb}^{(n-1)} + \Delta\boldsymbol{w}_a^{(l)}$
43 Update model-based actor policy $\pi_{mb}^{(n)} = \pi_{mb}(\boldsymbol{w}_{a,mb}^{(n)})$
44 **end**
45 Save current optimal set of $\boldsymbol{w}_{a,mb} = \boldsymbol{w}_{a,mb}^*$
46 ▷ Step 6: Policy switching and parametrization cloning
47 Forward N_z virtual trajectories with $\boldsymbol{s}_0 = \breve{\boldsymbol{s}}_0^{(l)}$, $\boldsymbol{a}_k = \pi_{\text{live}}(\boldsymbol{s}_k, \boldsymbol{w}_{a,\text{live}})$, $\boldsymbol{z} \sim \mathcal{N}_z^{(l)}()$. Get $\bar{\mathcal{J}}_{a,\text{live}}$
48 Forward N_z virtual trajectories with $\boldsymbol{s}_0 = \breve{\boldsymbol{s}}_0^{(l)}$, $\boldsymbol{a}_k = \pi_{\text{idle}}(\boldsymbol{s}_k, \boldsymbol{w}_{a,\text{idle}})$, $\boldsymbol{z} \sim \mathcal{N}_z^{(l)}()$. Get $\bar{\mathcal{J}}_{a,\text{idle}}$
49 Policy switching and weight cloning based on the decision tree (Fig. 3)
50 **end**

and the model-free policy and contains transitions ranked by their TD error δ_k (see Eq. (7)), which is updated as the training of the critic progresses. The second is used for training the digital twin and contains transitions ordered in time and arranged in trajectories. Therefore, $\mathcal{D}_\text{L}$ contains n_L (random) transitions while $\mathcal{D}_\text{S}$ collects n_S trajectories. The TD error defines both the sampling and the cleaning of the $\mathcal{D}_\text{L}$ buffer. The triangular distribution in (8) is used both for sampling and cleaning: the transitions with the lowest probability of being sampled are also the ones continuously replaced by the new ones. On the other hand, no specific criteria are considered for the sampling/cleaning of the buffer $\mathcal{D}_\text{S}$, which are simply sampled and stored in chronological order.

- **Step 2: Critic Update and model-free policy update (lines 14–24)**. An optimization with n_Q iterations is carried out on the critic network, followed by an optimization with n_A iterations for the policy. Both use batches of n_b transitions sampled from $\mathcal{D}_\text{L}$. Concerning the training of the critic network (lines 14–19), this loss function and its gradient are computed from (7). This step is essentially a training of the critic network in a supervised learning formalism. Similarly to what was proposed by [78] for the original DDPG, the update of the critic is modulated by an under-relaxation τ using a target network (line 17). At every n_Q iteration, the optimized critic re-evaluates the TD errors of the sampled transitions in $\mathcal{D}_\text{L}$ (line 18). Limiting the TD updates for the current batch of trajectories allows for a pseudo-random update of the most critical transitions, according to the critic. In addition, this approach is also computationally cheaper than re-evaluating all the stored transitions in the buffer, usually in the order $\mathcal{O}(10^6)$. Successively, the critic is then used to train the policy in lines 20–24. In this second stage, the gradient with respect to the cost function is computed using Eq. (4). Note that we use the notation $\Delta\boldsymbol{w}$ to denote the optimization update for the weights $\boldsymbol{w}$, such that the updating reads $\boldsymbol{w} \leftarrow \boldsymbol{w} + \Delta\boldsymbol{w}$. Therefore, the update in a simple gradient descent over a cost function $J(\boldsymbol{w})$, for example, reads $\Delta\boldsymbol{w} = -\eta\, dJ(\boldsymbol{w})/d\boldsymbol{w}$. This makes the notation independent of the choice of the optimizer. Differently from the original DDPG algorithm in [78], the updates of the critic and the policy networks are carried out via two separate loops rather than in a single one. In our numerical experiments, we found this approach to lead to more stable model-free policy updates.
- **Step 3: Fit random processes for exogenous inputs (line 26)**. In principle, both the forward and backward integration of the ODEs driving the digital twin could be carried out using time steps that differ from the ones controlling the interactions with the real system. This would require some interpolation on the exogenous disturbances as well as on the states and the actions. The data storage between forward/backward evaluations could leverage modern checkpointing techniques (see Zhang and Constantinescu [137]). However, keeping the focus on the first proof of concept of the reinforcement twinning idea, we here assume that the numerical integration (both forward and backward) is carried out using the same time stepping from the interaction with the environment (step 1).
  Nevertheless, we use the SVR regression on the exogenous input (see (14)) to construct the set of random processes (here treated as Gaussian processes) from which the ensemble of possible realizations of the (stochastic) exogenous input will be sampled. These are used for ensemble averaging of the gradient over possible trajectories. For each of the trajectory $l \in [1, n_S]$ available in the short-term memory buffer $\mathcal{D}_\text{S}$, we denote as $\boldsymbol{z}^{(l)} \sim \mathcal{N}_z^{(l)}()$ a sample of the Gaussian processes generating the evolution of possible exogenous inputs.
- **Step 4: Optimize digital twin on the real actions (lines 27–36).** The goal of this step is to find the closure parameters $\boldsymbol{w}_p$ such that the digital twin's prediction matches with the n_S trajectories of the real system, stored in $\mathcal{D}_\text{S}$. Consequently, the loss function $\mathcal{J}_p$ in (2) is minimized using a gradient-based optimization loop. This consists of n_G iterations and computes the updates $\Delta\boldsymbol{w}_p$ from the gradient evaluated along the n_S trajectories available in $\mathcal{D}_\text{S}$. Again, fixing the numerical time stepping to the one of the real interaction avoids the need for action interpolation but might become problematic for stiff problems. For each trajectory (loop in lines 27–33), a total of N_z possible realizations of the exogenous input are considered (sampled from the $\mathcal{N}_z^{(l)}$ processes built in the previous step), and the gradient is computed using the adjoint method (see Eq. (10)). The associated updates are then averaged (line 32), and the optimization step is carried out (line 34). Each optimization step requires a total of $n_S \times N_Z$ forward and backward virtual episodes. It is worth highlighting that the ensemble averaging at this step would be essential in cases for which (1) the system is chaotic and highly sensitive to random disturbances (hence producing "largely different" trajectories for "similar" inputs) or (2) the policy becomes an exploratory/identification policy rather than a control one (hence the goal is not to keep a trajectory but explore as much as possible the state–action space). On the other hand, if the action is solely driven by the goal of trajectory tracking control and the system is inherently a noise-rejection system, the interest in computing many trajectories and then averaging the associated gradients is lost. The presented test cases fall within this second category; future work will consider cases under the first category.

- **Step 5: Model-based policy update. (lines 37–43)**. This step is structurally similar to the previous: it computes the ensemble-averaged adjoint-based gradient (line 41) of a cost function (J_a) and performs a number (n_{mb}) of gradient-based optimization steps (line 43). In this case, however, the interactions with the digital twin follow the model-based policy π_{mb} instead of the actions stored in D_S. This allows to evaluate the current set of model-based policy weights $\boldsymbol{w}_{a,mb}$. The response of the digital twin is governed by the closure law defined by the current best set of weights, i.e. $\boldsymbol{p} = g(\boldsymbol{s}, \boldsymbol{w}_p^*)$.
- **Step 6: Policy switching and parametrization cloning. (lines 44–47)**. This step decides which of the policies π_{mb} or π_{mf} becomes "live" and which one becomes "idle". The first is deployed in the next interaction with the real system, while the second continues to be trained with different mechanisms. The decision in this step is based on the policy performances in the virtual environment, evaluated and averaged over N episodes with randomized initial conditions and disturbances. The flow chart for this step is illustrated in Fig. 3. Defining as $\bar{J}_{a,\text{live}}$ and $\bar{J}_{a,\text{idle}}$ the averaged cost functions for the "live" and "idle" policies, we introduce a tolerance factor $T_J \in \mathbb{R}$, a threshold $P_\pi \in \mathbb{N}$, and a counter $C_\pi \in \mathbb{N}$ to avoid overly impulsive decisions. If the live policy performs worse than the idle one more than the tolerance factor ($\bar{J}_{a,\text{live}}/\bar{J}_{a,\text{idle}} < T_J$), the counter C_π gets incremented. If C_π exceeds the threshold P_π, the "idle" policy takes over and becomes "live". On the other hand, if the opposite is true, we check that the idle policy is not stuck in a local minima, by analysing its weight variation:

$$d\boldsymbol{w}_a = \frac{\|\boldsymbol{w}_a^i - \boldsymbol{w}_a^{i-1}\|}{\|\boldsymbol{w}_a^i\|}, \tag{16}$$

where i indicates the current number of the episode ($i \in [1, n_e]$). To this end, we introduce a tolerance T_w, a counter C_w and a threshold P_w and repeat the previous decision process: if $d\boldsymbol{w}_a$ falls below T_w, C_w increases. If C_w goes beyond P_w, it is reasonable to assume that the policy optimization step is trapped in a local minimum, and we replace the "idle" parametrization by cloning the "live" one.

We close with a few general remarks on the proposed algorithm.

Remark 1. The n_S hyperparameter in Steps 4 and 5 defines how much the digital twin should consider past experiences. In this work, we set n_S=1, thus focusing on the last trajectory solely, while having a variable N_z to promote robustness against randomness on the exogenous inputs and to take into account possible sampling errors which may occur in real-world scenarios.

Remark 2. The idle policy continues its training through different mechanisms. No special considerations are needed if the idle policy is the model-based policy: this continues to be trained on the digital twin regardless of whether it is active on the real system or not. On the other hand, if the idle policy is model-free one, the learning occurs indirectly through two mechanisms that establish a "demonstration" from the model-based to the model-free: (1) the storage of "expert" transitions in the memory buffer D_L and (2) the continuous training of the value function Q. The off-policy nature of the reinforcement learning algorithm here implemented allows the agent to learn from policies that are exploratory or simply suboptimal according to the current Q function. In this case, when the model-free policy is idle, and the Q function continues to be updated by observing the model-based policy in action on the real system– a sort of "demonstration" mechanism is established. The fact that the model-free could learn by the demonstration and eventually overtake the lead as a live policy is arguably the most significant contribution of this work.

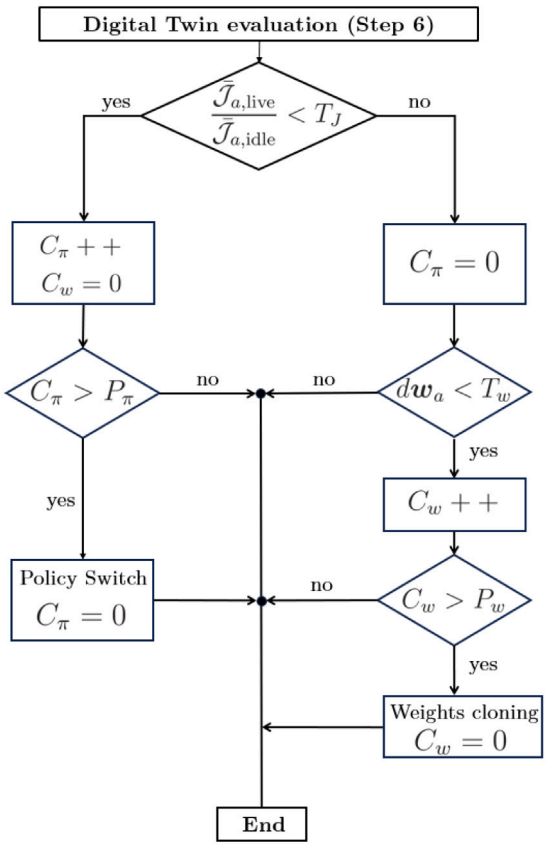


Fig. 3. Decision tree in step 6 of the Reinforcement Twinning algorithm (see 1.1). This step defines (1) which of the model-free or model-based policy becomes live and idle and (2) whether one of the two is consistently under-performing and is thus replaced by a clone of the other.

Remark 3. The model-free update (Step 2), the model-based update (Steps 3,4,5), and the individual evaluations occurring in (Step 6) are independent processes and can be parallelized, greatly reducing computational time.

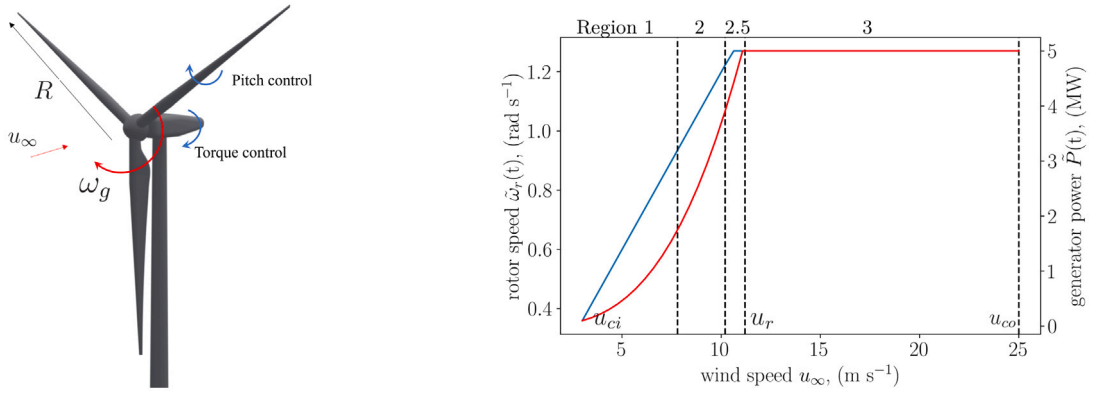
5. Selected test cases

We present the three selected test cases: (1) the control of a wind turbine subject to time-varying wind speed (Section 5.1), (2) the trajectory control of flapping-wing micro air vehicles (FWMAVs) subject to wind gusts (Section 5.2) and (3) the mitigation of thermal loads in the management of cryogenic storage tanks (Section 5.3). These are very simplified versions of the real problems, and the reader is referred to the specialized literature for more details.

5.1. Wind turbine control

Test case description and control problem. We consider the problem of regulating the rotor's angular speed ω_r and the extracted power P_e of a variable speed variable pitch horizontal turbine of radius R, subject to time-varying wind velocity u_∞ (see Laks et al. [138], Pao and Johnson [139]).

The operation (and thus the control objectives) of a wind turbine depends on the wind speed in relation to three regions, identified by three velocities: cut-in (u_{ci}), rated (u_r) and cut-out (u_{co}) wind speeds. These are shown in Fig. 4 for the NREL 5-MW turbine considered in this test case (details in the following subsection). In region 1 ($u_\infty < u_{ci}$), the wind speed is too low to justify the turbine operation and no power



(a) (b)

Fig. 4. Sketch of the main parameters involved in wind turbine control 4(a) and reference power curve 4(b), $\tilde{P}$ (in red), and rotor speed $\tilde{\omega}_r$ (in blue).

is produced. In region 2 ($u_{ci} < u_\infty < u_r$), the goal is to maximize the energy production. This is usually achieved by pitching the blades at the optimal value β^* and acting on the generator torque to keep the optimal rotational speed. In region 3 ($u_\infty > u_r$), the turbine is in above-rated conditions, and the goal is to keep the power extraction at its nominal value P_N, thus reducing the aerodynamic load acting on the rotor. This is usually achieved by keeping the nominal generator torque and acting on β. Finally, the wind turbine is shut down if $u_\infty > u_{co}$, preventing mechanical and electrical overloading.

Particularly critical is the transition between regions 2 and 3 (region 2.5 in Fig. 4,) where gusts or lulls could result in conflicting requirements between torque and pitch controllers. This region is typically handled via set point smoothing, mitigating transients when switching from one control logic to another [140,141].

In this test case, we combine a Maximum Power Point Tracking (MPPT) controller in below-rated conditions [142,143] with a regulation control in above-rated conditions. The controller can act on both the generator torque and the pitch angle, receiving as sole input the generator speed ω_g, as typically done in multi-megawatt wind turbines [144]. The current wind speed u_∞ acts as exogenous input, which we assume can be provided by real-time measurements. We thus collect a sequence $u_\infty(t_k)$ with $t_k = k\Delta t$. To avoid high-frequency excitation of the system, we filter the generator speed signal using an exponential smoothing [145] with the recursive relations:

$$\hat{y}(t_k) = (1-\hat{\alpha})y(t_k) + \hat{\alpha}\cdot y(t_{k-1})\,, \tag{17}$$

with α being the low-pass filter coefficient computed as $\hat{\alpha} = \exp(-2\pi f_c \Delta t)$, with f_c the minimum cut-off frequency, y the generic raw signal and $\hat{y}$ its low-pass filtered version.

We employ this filter for the generator rotational speed, ω_g and the measured wind speed, u_∞. From the latter, it is possible to define the target rotational speed $\tilde{\omega}_g$ and extracted power $\tilde{P}$ from the reference curves in Fig. 4. In Region 2, these two variables are linked by the operating conditions, while in Region 3, these are defined by mechanical constraints. In the below-rated conditions, the power is maximized if the rotor operates at its optimal tip-speed-ratio $\lambda_{r,*} = \omega_r R/u_\infty$. Thus, one can define the reference generator speed relying on the measured wind speed as

$$\tilde{\omega}_g = N_g \frac{\lambda_{r,*}\hat{u}_\infty}{R}\,, \tag{18}$$

where N_g is the gearbox ratio. This variable is constrained in the range $\omega_{g,min} \le \tilde{\omega}_g \le \omega_{g,\text{rated}}$, taking into account the machine specifications. Therefore, given $\tilde{\omega}_g$ the target rotational velocity, we define the tracking errors as $e_1(t) = (\check{\omega}_g - \tilde{\omega}_g)/\gamma_1$, and its integral over the past rotor revolution, $e_2(t) = (\int_{t_i-T_r}^{t_i} e_1(t')\,\mathrm{d}t')/\gamma_2$, where $\boldsymbol{\gamma} = [\gamma_1, \gamma_2]$ ensure that both inputs lie in $[-1, 1]$, and consider these as the inputs of the control logic. We then define the actuation vector as

$$\boldsymbol{a}(\boldsymbol{e}(t), \boldsymbol{w}_a) = \begin{bmatrix}\tau_g \\ \beta\end{bmatrix} = \begin{cases}\text{clip}\,(\,w_{a1}e_1(t) + w_{a2}e_2(t), \tau_{g,min}, \tau_{g,max}\,) \\ \text{clip}\,(\,w_{a3}e_1(t) + w_{a4}e_2(t), \beta_{min}, \beta_{max}\,)\end{cases} \tag{19}$$

where $\text{clip}(x, x_1, x_2)$ is a smooth step returning x_1 if $x < x_1$, x if $x_1 < x < x_2$ and x_2 if $x > x_2$. In this work, we define it as

$$\text{clip}(x, x_1, x_2) = \frac{1}{2}(\tanh(x)+1)(x_2 - x_1) + x_1\,, \tag{20}$$

though any differentiable smooth step fits the purposes of bounding the action space while ensuring differentiability. This allows to limit the actuation rate of change according to the actuators' specifications, $\dot{\tau}_{g,max}$ and $\dot{\beta}_{max}$. We further assume that the controller can be informed by measurements of the turbine's rotational velocity $\check{\omega}$ and the generator torque $\check{\tau}_g$. It is thus possible to define a tracking error on the power as $e_P = \check{\tau}_g \cdot \omega(t; \boldsymbol{w}_a) - \tilde{P}$ and use this signal to measure the controller's performance. The reference power $\tilde{P}$ follows the $K\omega^2$-law [146] in under-rated conditions, setting

$$\tau_g(t) = K\omega_g(t)^2 \qquad K = \frac{\pi\rho R^5 C_{p,max}}{2\lambda_{r,*}^3 N_g^3}\,, \tag{21}$$

with

$$\tilde{P} = K\omega_g(t)^3\,. \tag{22}$$

The target power production, also accounting for Region 3, is this $\tilde{P} = \text{MIN}(P_N, K\omega_g(t)^3)$. Combining these two references, the cost function driving the model-based optimal control law is

$$\mathcal{J}_a = \frac{\alpha_1}{2\,T_o}\int_0^{T_o} e_1(t)^2\,dt + \frac{\alpha_2}{2\,T_o}\int_0^{T_o} e_P(t)^2\,dt\,, \tag{23}$$

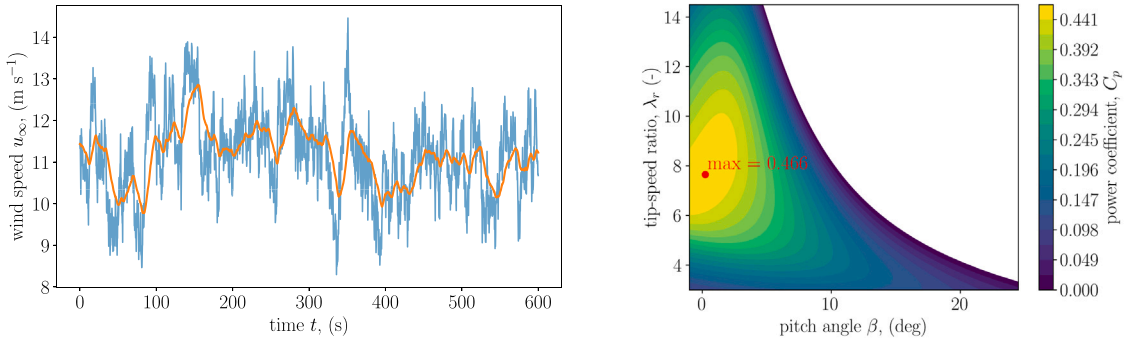
while for the model-free counterpart, we define the reward as

$$\mathcal{R}_a = -\frac{\alpha_1}{2\,n_t}\sum_{k=0}^{n_t-1} e_1^2(t_k) - \frac{\alpha_2}{2\,n_t}\sum_{k=0}^{n_t-1} e_P^2(t_k). \tag{24}$$

In both equations, α_1 and α_2 ensure that the two components of the cost function have comparable orders of magnitude. The choice of including both the velocity *and* the power extraction is justified by the fact that these parameters are independent, in the sense that one could extract the same amount of power with different generator velocities by acting on the torque, possibly violating mechanical constraints.

Selected conditions and environment simulator. Disregarding aeroelastic effects on both the blades and the tower, the simplest model of the rotor dynamics is provided by the angular momentum balance, which leads to a first-order system for the turbine's rotor's angular speed ω_r:

$$\dot{\omega}_r = \frac{1}{J}\left(\,\tau_a - \tau_g N_g\,\right), \tag{25}$$



(a) (b)

Fig. 5. Fig. 5(a): sample of the wind speed (exogenous input) considered for the test case 1, unfiltered (in blue) and filtered (orange). Fig. 5(b) shows the contour of the power coefficient $C_p(\lambda_r, \beta)$ for the NREL 5MW turbine considered for control purposes. The controller's objective is to keep the turbine near the maximum ($C_p = 0.46$) acting on the motor torque or the blade pitching.

where J is the rotor's moment of inertia, τ_a is the aerodynamic torque, τ_g is the opposing torque due to the generator and the gear train, and N_g the gearbox ratio. The gearbox efficiency is assumed to be unitary. The aerodynamic torque is linked to the wind velocity as

$$\tau_a = \frac{1}{2}\rho A_r \frac{C_p(\lambda_r, \beta)}{\omega_r} u_\infty^3 , \tag{26}$$

where ρ is the air density, $A_r = \pi R^2$ the turbine's swept area, and $C_p = P_e/(1/2\rho A u_\infty^3)$ is the power coefficient, that is the ratio between the extracted power P_e and the available one. For a given turbine design, this coefficient is a function of the tip-speed ratio λ_r and the blade pitching angle β. A predictive model can thus be obtained by combining (25) and (26) and a pre-computed $C_p(\lambda_r, \beta)$ map. This map is usually obtained via Blade Elementum Momentum Theory (BEMT, Moriarty and Hansen [147]), and we do the same for simulating the real environment in this test case. The resulting map for the selected turbine is shown in Fig. 5(b).

We consider the NREL 5-MW reference offshore wind turbine [144], commonly used as a reference in the literature [148–150]. This turbine has $R = 63$ m, $J = 115926$ kg m^2 and $\omega_{r,\text{rated}} = 1.27$ rad s^{-1} for $u_\infty \geq 11.4$ m s^{-1}. The gearbox ratio is $N_g = 97$. The actuation bounds of the pitch angle and torque actuators are respectively $(\beta_{\min}, \beta_{\max}) = (0, 90)$ deg and $(\tau_{\min}, \tau_{\max}) = (0, 43093)$ Nm. Their maximum rate of change are $\dot{\beta}_{max} = 8$ deg s^{-1} and $\dot{\tau}_{g,max} = 15000$ N m s^{-1}, respectively. From these, we define the cut-off frequency as the harmonic associated with the highest rate of change, obtaining $f_{c,\tau} = 0.05$ Hz and $f_{c,\beta} = 0.04$ Hz. We set the filter smoothing coefficient according to the slowest pitch actuator, finding $\hat{\alpha} = 0.98$.

We take $T_o = 600$ s for the observation time and assume a sampling of $f_s = 4$ Hz for all the simulated instrumentation. We simulate the incoming wind speed time series using the stochastic, turbulent wind simulator TurbSim [151] considering a wind velocity with average $u_\infty = 11$ m s^{-1} and turbulence intensity TI= 16%, thus oscillating between below and above rated. An example of velocity time series is shown in Fig. 5(a) together with the associated filtered signal.

Digital twin definition. The wind energy community has put significant effort into the development of digital twins of onshore and offshore wind turbines, with applications ranging from load estimations [152,153] to real-time condition monitoring [154,155]. In this work, we solely focus on the rotor's aerodynamic performance and the twinning process seeks to identify a closure law for the power coefficient from data, starting from (25) and (26). In fact, the BEMT, commonly used to obtain the C_p curve, assumes that the flow is steady, and this is questionable in the presence of large fluctuations in the wind speed [156]. Moreover, the coefficient can change over time as a result of turbine wear (e.g. due to blade erosion) or in the presence of extreme conditions (e.g. ice, dirt or bug buildup) [157]. This is why any strategy based on a pre-computed C_p curve is likely sub-optimal [158,159] and the idea of continuous updates of this closure law from operational data has merit. Although this exercise is purely illustrative, the approach can be readily extended to more complex and multi-disciplinary model formulations.

The digital twin employs the power coefficient parametrization formulated by [160]

$$\begin{cases} C_p(\lambda_r, \beta) = c_1\left(\dfrac{c_2}{\xi} - c_3\beta - c_4\xi\beta - c_5\beta^{c_{12}} - c_6\right)e^{-\frac{c_7}{\xi}} + c_8\xi \\ \xi^{-1} = (\lambda_r + c_9\beta)^{-1} - c_{10}(\beta^3 + 1)^{-1} \end{cases} \tag{27}$$

with the parameters $\boldsymbol{w}_p = [c_1, c_2, \dots c_{12}]$ to be identified. This model is known to be valid up to a maximum pitch excursion of $\beta = 30$ deg. Therefore, we limit the maximum allowable pitch command to this value in our numerical experiments.

The assimilation optimizes for $\boldsymbol{w}_p$, aiming to minimize the error between the predicted dynamics $\omega_g(t)$ and the real system behaviour $\breve{\omega}_g(t)$. This translates into minimizing the cost function

$$\mathcal{J}_p = \frac{1}{2}\int_0^T \left(\omega_g(t; \boldsymbol{w}_p) - \breve{\omega}_g(t)\right)^2 \mathrm{d}t . \tag{28}$$

5.2. Trajectory control of flapping wings micro air vehicles

Test case description and control problem. This test case considers the control of a Flapping-Wing Micro Air Vehicle (FWMAV) which aims to move from one position to another in the shortest possible time. FWMAVs are sometimes considered better alternatives to fixed-wing and rotary-wing micro aerial vehicles because of their higher agility and manoeuvrability [161]. The design and control of these drones seek to mimic the remarkable flight performances of hummingbirds, which can undertake complex manoeuvres with an impressive response time (see, for example, Cheng et al. [162] and Ortega-Jiménez and Dudley [163]).

Reproducing these performances requires advanced knowledge of the flapping wing's aerodynamics in the low Reynolds number regime, as well as the fluid–structure interaction resulting from the wing's flexibility. These remain significant challenges for advanced numerical simulations [164,165]. The proposed test case shows how simplified models of the wing's aerodynamics could be derived in real-time while an FWAVs seeks to achieve its control target. The considered problem is illustrated in Fig. 6(a). A drone initially at $\boldsymbol{x}_0 = [x_0, z_0]$ m and $\dot{\boldsymbol{x}}_0 = 0$ m/s is requested to move to $\tilde{\boldsymbol{x}_f} = [\tilde{x}_f, \tilde{z}_f]$ m as fast as possible, and then hover in that position (i.e. $\dot{\tilde{\boldsymbol{x}}}_f = [0, 0]$) by continuously adapting its flapping wing motion. We challenge the controller by introducing gusty wind disturbances $\boldsymbol{u}_\infty$,

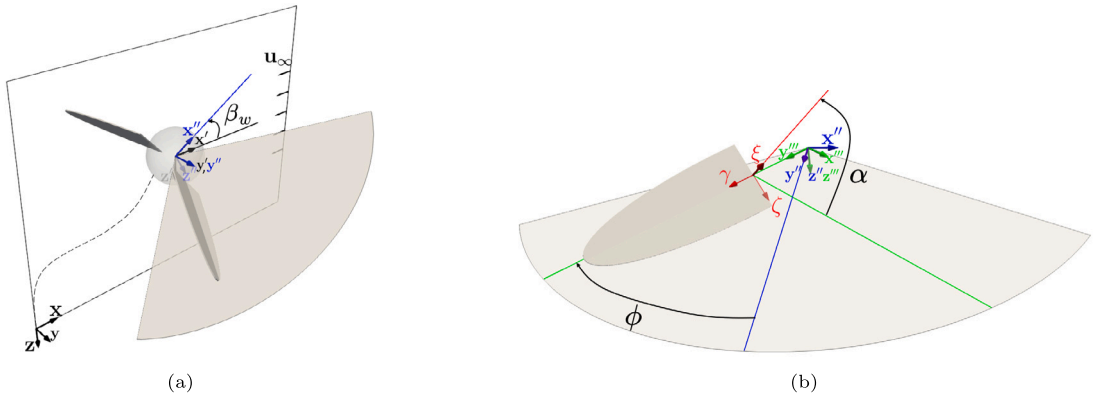


Fig. 6. (a) Schematic of the flapping drone going from the initial position $\vec{x}_0$ to the final position $\vec{x}_f$. $(x'y'z')$ is the stroke frame that is tilted by β_w from the inertial frame (xyz) (b) Zoom on the stroke plane that holds three frames (stroke frame $x'y'z'$, flapping plane $x''y''z''$ and the wing-fixed frame (ξ,γ,ζ)) which defines the flapping angle ϕ and the pitching angle α. The wing frame is offset on the wing root only for the clarity of the figure.

To simplify the body's kinematics, we consider only 2D trajectories characterized by the position vector $\boldsymbol{x}(t)$ defined in the ground frame (x, y, z). We also attach a moving frame (x', y', z') to the body with no rotational freedom. Starting from the body frame, three Euler angles (β_w, ϕ, α) and three reference frames characterize the wing kinematics (Fig. 6(b)). The wings always flap symmetrically so that the position of one wing defines the other one.

The body frame is first pitched by a stroke plane angle $\beta_w(t)$ along y' to define the stroke plane frame (x'', y'', z''). The wing tips are constrained to flap in the stroke plane (x'', y'') as it flaps along the normal z''. The flapping frame (x''', y''', z''') follows this rotational motion and defines the flapping angle $\phi(t)$ between the wing symmetry axis y''' and the lateral direction y'' (Fig. 6(b)). The last frame is the wing-fixed frame (ξ, γ, ζ) that results from the pitching rotation along the wing symmetry axis. The pitching angle $\alpha(t)$ is then defined between the chord-normal direction ξ and the stroke plane axis x'''. The reader is referred to [166,167] for a complete treatment of the flapping wing kinematics. We select an harmonic parametrization for ϕ and α:

$$\phi(t) = A_\phi \cos(2\pi f t) \quad \text{and} \quad \alpha(t) = A_\alpha \sin(2\pi f t), \tag{29}$$

where the beginning of the stroke ($t = 0$) is when the flapping amplitude is maximal ($\phi(t) = A_\phi$) and the chord plane is perpendicular to the stroke plane ($\alpha(t) = 0°$).

We assume that the flapping frequency f and the pitching amplitude A_α are fixed and set the flapping amplitude A_ϕ as a control parameter together with the stroke plane angle β_w, i.e. $\boldsymbol{a} = [A_\phi, \beta_w]^T$. We link the action vector to the state error using a Proportional–Derivative (PD) controller with an offset and capped outputs:

$$\boldsymbol{a}(\boldsymbol{e}; \boldsymbol{w}_a) = \begin{cases} \text{clip}\left(w_{a1} + w_{a2} e_x + w_{a3} e_z + w_{a4} \dot{e}_x + w_{a5} \dot{e}_z, A_{\phi,min}, A_{\phi,max} \right) \\ \text{clip}\left(w_{a6} + w_{a7} e_x + w_{a8} e_z + w_{a9} \dot{e}_x + w_{a10} \dot{e}_z, \beta_{w,min}, \beta_{w,max} \right) \end{cases} \tag{30}$$

where $\boldsymbol{e} = [e_x, e_z, \dot{e}_x, \dot{e}_z]$, with $e_x = x_f - x$, $e_z = z_f - z$ and dots denoting differentiation in time. The control action parameters are $\boldsymbol{w}_a \in \mathbb{R}^{10}$ and the function $\text{clip}(x, x_1, x_2)$ is the smoothing operation introduced in (20). The bias terms w_{a1}, w_{a6} helps stabilize the drone in hovering conditions once the target position is reached since this operation requires a continuous effort even when the state error is zero.

The reward function driving the model-free controller is written as

$$\mathcal{R}_c = -\frac{1}{2n_t} \sum_{k=1}^{n_t} \|\check{\boldsymbol{e}}_k\|_2^2 + \alpha_1 \|\check{\boldsymbol{e}}_k \circ \boldsymbol{h}(\boldsymbol{x}_k)\|_2^2 . \tag{31}$$

where $\circ$ denotes the Hadamard (entry by entry) product and $\boldsymbol{h}(\boldsymbol{x}_k)$ is a vector of two Gaussian functions with mean $\boldsymbol{x}_f$ and standard deviation $\sigma = 0.71$:

$$\boldsymbol{h}(\boldsymbol{x}_k) = \frac{\boldsymbol{x}_f^2}{2} \exp\left[-\frac{1}{2} \left(\frac{\boldsymbol{x}_k - \boldsymbol{x}_f}{\sigma} \right)^2 \right]. \tag{32}$$

This function is zero far from the target position and unitary once the drone reaches it, so the role of the second term in (31) is to penalize large velocities once the drone approaches the goal. The term $\alpha_1 \in \mathbb{R}$ in (31) weights the importance of the penalty. The cost function driving the model-based controller reproduces the same approach in a continuous domain and reads:

$$\mathcal{J}_a = \frac{1}{2T_0} \int_0^{T_0} \|\boldsymbol{e}(t)\|_2^2 + \alpha_1 \|\dot{\boldsymbol{e}}(t) \circ h(\boldsymbol{x}(t_k))\|_2^2 dt . \tag{33}$$

Selected conditions and environment simulator. We consider a drone with mass $m_b = 3$ g and a constant flapping frequency of $f = 20$ Hz. Given the initial position $\boldsymbol{x}_o = [0, 0]$ m, we set the target position at $\boldsymbol{x}_f = [5, 5]$ m. We assume that the manoeuvre should take less than $n_t = 100$ flapping cycles, hence we consider an observation time of $T_o = 5$ s. This defines the observation time for the twinning. We fix the pitching amplitude to $A_\alpha = 45$ deg and let the flapping amplitude vary in the range $A_\phi \in [50; 88]$ deg. These values are comparable to what is observed in hummingbird's flight, although the harmonic parametrization in (29) oversimplifies their actual wing dynamics [168]. The stroke plane angle is bounded in the range $\beta_w \in [-30, 30]$ deg.

We assume that the drone's wings are semi-elliptical and rigid with a span $R = 0.05$ m and a mean chord $\bar{c} = 0.01$ m. The wing roots are offset of $R_0 = 0.0225$ m from the body barycentre, which is also the centre of rotation of the wings. We decouple their motion from the body such that the wing inertia does not influence the body motion [169]. This hypothesis simplifies the body dynamics, which becomes only a function of the aerodynamic forces produced by the wings and the gravitational force. These are assumed to apply on the barycentre of the body, which is treated as a material point. Therefore, restricting the dynamics to the (x, z) plane, the force balance gives

$$m_b \ddot{x} = F_{x,w} - D_b \cos(\beta_b) \tag{34}$$

$$m_b \ddot{z} = F_{z,w} - m_b g - D_b \sin(\beta_b) \tag{35}$$

where the subscript w refers to forces produced by the wing and the subscript b identifies body quantities. Hence the aerodynamic forces produced by the flapping wings are $\boldsymbol{F}_w = [F_{x,w}, F_{z,w}]^T$, D_b is the magnitude of the drag force exerted on the body and β_b is the angle between the drag force and the x axis. Considering the velocity of the drone $\dot{\boldsymbol{x}} = [\dot{x}, \dot{z}]^T$ and the incoming horizontal wind disturbance (both in the ground frame), this angle is defined as $\beta_b = \tan^{-1}(\dot{z}/(\dot{x} + u_\infty))$. The magnitude of the drag force is computed as

$$D_b = \frac{1}{2} \rho S_b C_{D,b} U_b^2, \tag{36}$$

where ρ is the air density, $S_b = 0.0005$ m^2 is the body's cross-sectional area, $C_{D,b}$ is the body's drag coefficient, here taken as unitary, and U_b is the modulus of the relative velocity between the drone body and the wind, i.e. $U_b = \sqrt{(\dot{x}+u_\infty)^2+\dot{z}^2}$. The wind disturbance u_∞ results from a stationary Gaussian process with mean value $\bar{u}_\infty = 1$ m/s, in opposite direction to the x versor, and covariance matrix defined by a Gaussian kernel $\kappa(u_\infty(t), u_\infty(t+\tau)) = \exp(-\tau^2/(2\sigma_f^2))$ with $\sigma_f = 3$s.

To compute the wing forces in (34) and (35) we follow the semi-empirical quasi-steady approach of [170], based on the work by [171]. This is a common approach in control-oriented investigations (see e.g [164,167]) because it provides a reasonable compromise between accuracy and computational cost, at least for the case of smooth flapping kinematics and near hovering conditions (see [170]).

The approach is based on BEMT, which partitions the wing into infinitesimal elements (see Fig. 7(a)). Each element produces an infinitesimal aerodynamic force that must be integrated along the span to get the total lift and drag. For smooth flapping motion (29), the forces due to the flapping dominate all the other physical phenomena [170] so that the lift L_w and drag D_w expressions are

$$L_w(t) = \frac{1}{2}\rho C_{L,w}(\alpha)\int_{\Delta R}^{R} U_w^2(t,r)c(r)dr \tag{37}$$

$$D_w(t) = \frac{1}{2}\rho C_{D,w}(\alpha)\int_{\Delta R}^{R} U_w^2(t,r)c(r)dr, \tag{38}$$

where $C_{L,w}$ and $C_{D,w}$ are the lift and drag coefficient of the wing and U_w is the modulus of the relative velocity between the wing displacement, the body displacement and the wind. This velocity depends on the span-wise coordinate of the wing section r (see Fig. 7(b)) and is computed as

$$\begin{aligned}\boldsymbol{U}_w(r,t,\boldsymbol{u}) &= [\boldsymbol{U}_{w,x}, \boldsymbol{U}_{w,y}, \boldsymbol{U}_{w,z}] \\ &= \left(\boldsymbol{R}_2(\alpha)\begin{bmatrix}0\\0\\\dot{\phi}\end{bmatrix}\right)\times\begin{bmatrix}0\\r\\0\end{bmatrix} + \boldsymbol{R}_2(\alpha)\boldsymbol{R}_3(\phi)\boldsymbol{R}_2(\beta_w)\left(\begin{bmatrix}\dot{x}\\\dot{y}\\0\end{bmatrix}+\begin{bmatrix}U_{wi}\\0\\0\end{bmatrix}\right)\end{aligned} \tag{39}$$

The first term is the (linear) flapping velocity of the wing, while the second term gathers the velocity of the body and the wind. Both terms are projected in the wing frame thanks to three rotation matrices:

$$\boldsymbol{R}_2(\alpha) = \begin{bmatrix}\cos(\alpha) & 0 & \sin(\alpha)\\ 0 & 1 & 0\\ -\sin(\alpha) & 0 & \cos(\alpha)\end{bmatrix} \qquad \boldsymbol{R}_3(\phi) = \begin{bmatrix}\cos(\phi) & \sin(\phi) & 0\\ -\sin(\phi) & \cos(\phi) & 0\\ 0 & 0 & 1\end{bmatrix} \tag{40a}$$

$$\boldsymbol{R}_2(\beta_w) = \begin{bmatrix}\cos(\beta_w) & 0 & \sin(\beta_w)\\ 0 & 1 & 0\\ -\sin(\beta_w) & 0 & \cos(\beta_w)\end{bmatrix} \tag{40b}$$

where $\boldsymbol{R}_2(\beta_w)$ transform velocities from the (x', y', z') frame to the (x'', y'', z'') frame, $\boldsymbol{R}_3(\phi)$ from (x'', y'', z'') to (x''', y''', z''') and $\boldsymbol{R}_2(\alpha)$ from (x''', y''', z''') to (ξ, γ, ζ).

The force coefficients $C_{L,w}$ and $C_{D,w}$ in Eqs. (37) and (38) are modelled similarly to [170,172]:

$$C_{L,w} = a\sin(2\alpha_e) \tag{41}$$

$$C_{D,w} = b + c\,(1-\cos(2\alpha_e)), \tag{42}$$

where $\boldsymbol{w}_p = [a, b, c]$ are the closure parameters for this problem. This parametrization approximates the influence of the Leading Edge Vortex (LEV), generated at a high pitching angle due to flow separation and stably attached on the suction side (see [173]). The corresponding force coefficients depend on the effective angle of attack $\alpha_e = \cos^{-1}(U_{w,z}/\|\boldsymbol{U}_w\|_2)$, defined between the chord direction ζ and the relative velocity of the wing $\boldsymbol{U}_w$.

Finally, the forces in Eqs. (37) and (38) were computed in the reference wind frame (ξ', γ', ζ'), which rotates the wing frame by α_e along γ. The drag D_w is then aligned with ξ' while the lift is perpendicular to the drag (Fig. 7(b)). These forces are successively transformed into the wing frame and then into the body frame to retrieve $F_{x,w}$ and $F_{z,w}$ in the dynamic Eqs. (34) and (35).

In this test case, the real environment is simulated using $\boldsymbol{w}_p = [a, b, c] = [1.71, 0.043, 1.595]$ in (37) and (38), as approximated from [170] for semi-elliptical wings. For the digital twin, these are the parameters to be identified while interacting with the environment to minimize the cost function:

$$\mathcal{J}_p = \frac{1}{2}\int_0^T \left(\boldsymbol{s}(t;\boldsymbol{w}_p) - \check{\boldsymbol{s}}(t)\right)^2 \mathrm{d}t\,, \tag{43}$$

with $\boldsymbol{s}(t) = [x(t), z(t)]$ the drone position at time t.

5.3. Thermal management of cryogenic storage

Test case description and control problem. We consider the thermal management of a cryogenic tank. This is a fundamental challenge in long-duration space missions, which require storing large amounts of cryogenic propellant for long periods. These fluids must be stored in the liquid phase to maximize the volumetric energy density. However, their storage temperatures are extremely low (e.g., −250 °C for liquid hydrogen), and their latent heat of evaporation is much smaller than that of non-cryogenic propellants. These combined factors challenge the storage because evaporation due to thermal loads produces a continuous pressure rise over time [174–176].

The efficient management of this problem requires a combination of advanced insulation strategies (see Mer et al. [177], Jiang et al. [178]) and venting control strategies [179]. In this test case, we consider a classic active Thermodynamic Venting System (TVS), similar to the one presented in [179] and recently characterized in [180,181]. Fig. 8(a) shows the schematics of the TVS configuration.

We consider a tank of volume V_t filled with a volume of liquid $V_l(t)$, operating at a nominal pressure p_0 while subject to a time-varying heat load $Q_h(t)$. The TVS is composed of a vented branch and an injection loop. In the vented branch (blue line in Fig. 8(a)), a mass flow rate $\dot{m}_c$ is extracted from the tank, expanded through a Joule-Thomson valve (1) and used as a cold source in a heat exchanger (2) before being expelled. In the injection loop, a mass flow rate $\dot{m}_h$ is circulated by a pump (3) into the hot side of the heat exchanger and reinjected into the tank as a subcooled liquid. This injection can be carried out via a submerged jet or a spray bar [182–184], but this distinction is unnecessary for the illustrative purposes of this work. An extensive literature review of TVS approaches is presented by [185], who also introduced and tested two simplified approaches for the thermodynamic modelling of the problem (see Barsi and Kassemi [186] and Barsi and Kassemi [187]). Fig. 8(b) maps the (ideal) key state of the fluids at the inlet and outlet of each component in the $T-s$ (temperature vs. specific entropy) diagram. The state of the liquid and the vapour in the tank are denoted with subscripts (l) and (v), the states (1) and (2) are the inlet and outlet of the heat exchanger on the vented line, while states (1′) and (2′) are their counterparts on the injection side.

The control task is to keep the tank pressure below the maximum admissible one while venting the least amount of liquid. The simplest control problem, implemented in this work, consists of employing the pressure difference $\Delta p_V = p_0 - p_1$ as the sole control parameter. The Δp_V is linked to the mass flow rates in the J-T valve (1) and the pump (3) by limiting the operating range of the heat exchanger and by imposing that only latent heat is to be recovered from the venting line. Therefore, defining as $\Delta T_l(p_1) = T_l - T_1(p_1)$ the temperature drop produced by the J-T device and $\Delta h_c(p_1) = h_2 - h_1 = h_{v,\mathrm{sat}}(p_1) - h_{l,\mathrm{sat}}(p_0)$ the specific latent heat remaining on the vented line, one has

$$\dot{m}_h(p_1) = \frac{\Delta h_c(p_1)}{c_{p,l}\Delta T_l(p_1)}\,\dot{m}_c(p_1)\,. \tag{44}$$

Furthermore, to limit the control action exclusively to Δp_V, we assume that the pump (3) is equipped with flow regulation to follow

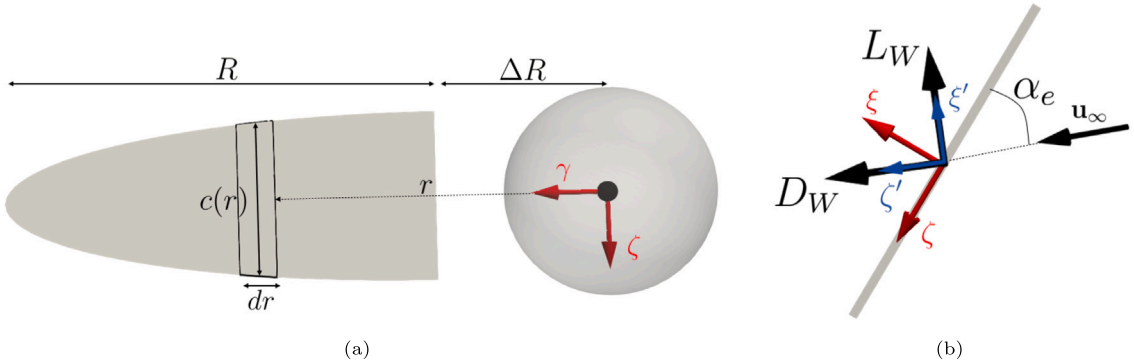


Fig. 7. (a) Main dimensions of the semi-elliptical wing. An element part of the BET is also highlighted (b) Effective angle of attack α_e, lift L_w and drag D_w shown with the wind frame (ξ', γ', ζ') and wing-fixed frame (ξ, γ, ζ) on a chord section of the wing.

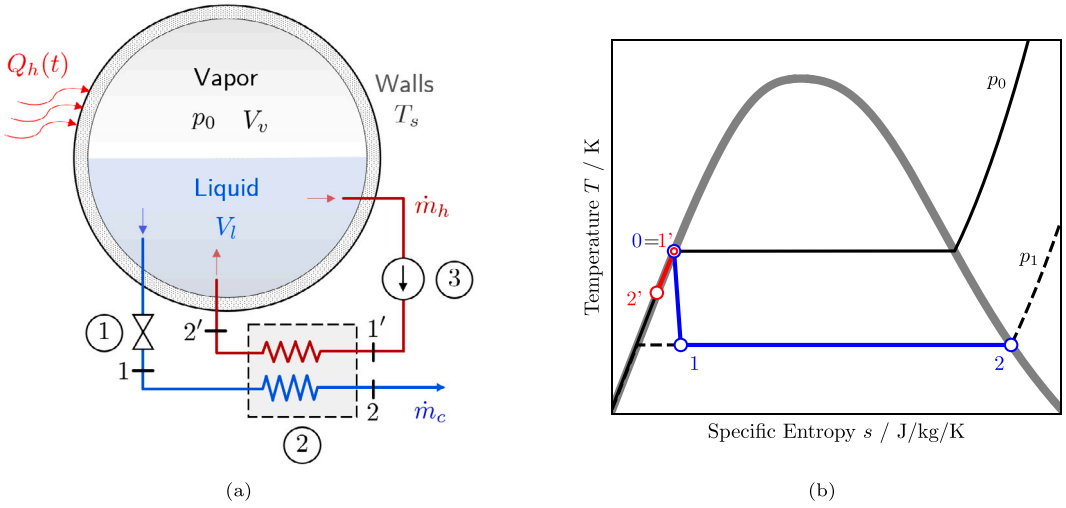


Fig. 8. Schematic of the thermal management problem considered in this work. A cryogenic tank of volume V_t, containing a volume V_l of saturated liquid and $V_t - V_l$ saturated vapour, is subject to a heat load $Q_h(t)$. The figure on the right traces the state of the system on the temperature-specific entropy diagram considering the simplified model used to simulate the environment.

(44) and we link the vented flow rate $\dot{m}_c$ to Δp_V and the J-T valve's characteristic through

$$\dot{m}_c(\Delta p_v) = \sqrt{\Delta p_V / K_{JT}}, \tag{45}$$

with K_{JT} the valve's pressure drop constant. Considering the maximum pressure drop along the JT-valve $\Delta p_{\max}$, the desired operational pressure $\tilde{p}$ and defining $e(t) = p(t) - \tilde{p}$ the tracking error, we consider a simple policy parametrization of the form

$$\Delta p_V = a(e, \boldsymbol{w}_a) = \frac{1}{2}\,\Delta p_{\max}\left(\tanh(w_{a1}e + w_{a2}e^3 + w_{a3}) + 1\right). \tag{46}$$

We treat Δp_{max} as a pre-defined constant and use $\boldsymbol{w}_a = [w_{a1}, w_{a2}, w_{a3}]$ as policy parameters. w_{a3} acts as an offset in the policy, controlling the threshold pressure above which the valve opens, while w_{a1} controls the slope of the hyperbolic tangent, i.e. the sensitivity of the policy to the tracking error. The cost function driving the model-based controller and the model-free agent are

$$\mathcal{J}_a = \frac{1}{T_o}\int_0^{T_o} \alpha_1 e^2(t) + \alpha_2\,\dot{m}_c \mathrm{d}t \quad \text{and} \quad \mathcal{R}_c = -\frac{1}{n_t}\sum_0^{n_t-1} \alpha_1 e(t_k)^2 + \alpha_2 \dot{m}_c(t_k), \tag{47}$$

with the $\alpha_1, \alpha_2 \in \mathbb{R}$ used to non-dimensionalize the inputs and weight the penalty on the mass consumption versus the primary objective of keeping the pressure constant.

To set up the data-driven identification to this control law, we assume that the tank is instrumented with a pressure sensor sampling $\check{p}(t)$ and a flow rate sensor on the vented line to measure $\check{m}_v(t)$.

Selected conditions and environment simulator. We consider a cryogenic tank with volume $V_t = 60$ m^3 with a desired operational pressure $\tilde{p} = 500$ kPa filled with liquid hydrogen. At the start of each episode, the tank is 70% full and pressurized to an initial value sampled from a normal distribution with mean $\bar{p}_0 = 500$ kPa and standard deviation $\sigma_{p_0} = 25$ kPa. The thermal load applied to the tank fluctuates from a minimum of roughly 50 W to a maximum of 300 W with a period of $T_h \approx 1.5$ h. This cyclic load mimics the large thermal fluctuations a tank could face in orbit as it is periodically exposed to direct sun radiation. The chosen period is approximately the time it takes the International Space Station (ISS) to complete an orbit around Earth. The load profile is shown in Fig. 9(a). This load is constructed as a pseudo-random signal made of Gaussian-like peaks randomly varying within $[0.9, 1.1]$ times the nominal peak, lifted by the minimal load and polluted by uniform noise with a standard deviation equal to 10% the nominal peak.

The simulation of the real environment and the digital twin are carried out using the same simplified model. In particular, we employ the homogeneous model presented in [185], adapted to account for the venting line similarly to [188]. This approach assumes that the liquid and the vapour are in saturation conditions. This is an oversimplification because the liquid is typically subcooled and the vapour is usually superheated. We refer to [187,189] for more advanced models and

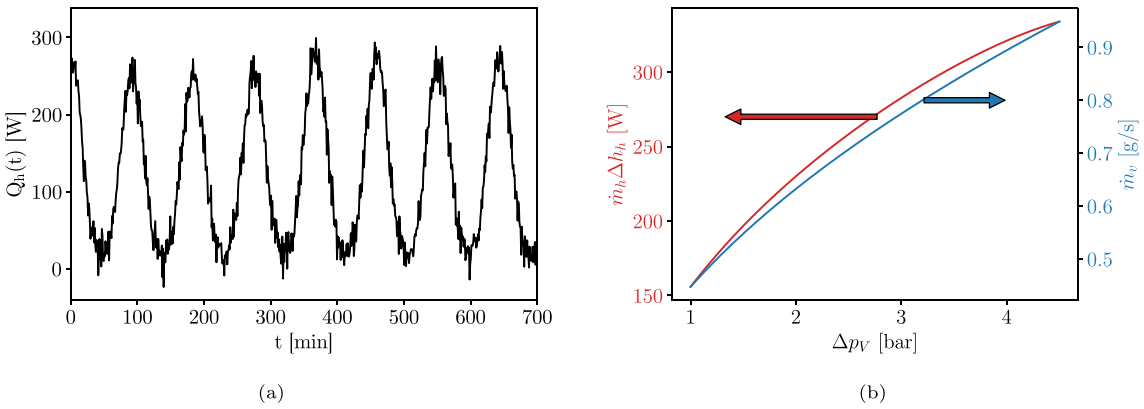


Fig. 9. Sample profile of the thermal load (exogenous input) considered in test case 2 (Fig a) and impact of the control parameter (p_1) on the vented flow rate and subcooling of the injection line.

to [190,191] for an in-depth discussion on the limits of thermodynamic models. Focusing only on the storage problem (no out-flow except venting), the conservation of internal energy of the solid, and the mass and enthalpy of the saturated mixture read

$$\frac{d}{dt}\left[m_s c_s T_s\right] = Q_h(t) - Q_{s\to f}\,, \tag{48a}$$

$$\frac{d}{dt}\left[\rho_v(V_t - V_l(t)) + \rho_l V_l(t)\right] = -\dot{m}_c\,, \tag{48b}$$

$$\frac{d}{dt}\left[\rho_v h_v(V_t - V_l(t)) + \rho_l h_l V_l(t)\right] = Q_{s\to f} + V_t\frac{dp}{dt} - \dot{m}_c h_l + \dot{m}_h \Delta h_h\,, \tag{48c}$$

where ρ is the density, h is the specific enthalpy, and $\Delta h_h = h_{2'} - h_{1'}$ is the variation of specific enthalpy on the injection loop provided by the heat exchanger. All properties are at saturation conditions, and the subscripts l and v refer to the liquid and the vapour phases, respectively. We consider the mass m_s and the wall's specific heat capacity c_s constant. Denoting as $H = \rho h$ the (volumetric) enthalpies and using the chain rule to have $d(\cdot)/dt = (d(\cdot)/dp)\,(dp/dt)$, these can be written as

$$m_s c_s \frac{dT_s}{dt} = Q_h(t) - Q_{s\to f}\,, \tag{49a}$$

$$\frac{dV_l}{dt} = \frac{1}{(\rho_l - \rho_v)}\left[\left(\frac{d\rho_v}{dp} - \frac{d\rho_l}{dp}\right)\frac{dp}{dt}V_l(t) - \dot{m}_c - \left(\frac{d\rho_v}{dp}\right)\frac{dp}{dt}V_t\right], \tag{49b}$$

$$\frac{dp}{dt}\left\{\left(\frac{dH_l}{dp} - \frac{dH_v}{dp}\right)V_l(t) + \left(\frac{dH_v}{dp} - 1\right)V_t\right\} = \left(H_g - H_l\right)\frac{dV_l}{dt} + Q_{s\to f} - \dot{m}_c h_l + \dot{m}_h \Delta h_h\,. \tag{49c}$$

The heat transferred from the wall to the fluid mixture is modelled using Newton's cooling law $Q_{s\to f} = U_s(T_s - T)$, where U_s is the overall heat transfer coefficient. Taking all liquid and vapour properties from thermophysical property libraries, this is a closed set of implicit differential equations for $\boldsymbol{s} = [T_s, V_l, p]$. The fluid properties are retrieved through the open-source library CoolProp [192], which is based on multiparameter Helmholtz-energy-explicit-type formulations and also provides accurate derivatives.

For a given design of the evaporator, one can link the subcooling in the injection line (term $\dot{m}_h\Delta h_h$ in (48c)) to p_1 using standard methods for heat exchanger design (see Cengel and Ghajar [193]). This can be written as $\dot{m}_h\Delta h_h = \varepsilon c_{p,l}\Delta T(p_1)$, with $\varepsilon = 1 - e^{-NTU}$ the efficiency of the heat exchanger, $NTU = U_{HX}A/(\dot{m}_h c_p)$ the number of transfer units and U_{HX} the overall heat transfer coefficient. In this test case we assume that the heat exchanger is a shell-and-tube design with 200 tubes of $D = 5$ mm diameter and $L = 2$ m length for a total exchange area of $A = 6.2$ m^2. Taking a wall/fluid heat transfer coefficient of $U_s = 50$ W/(m^{2}K), a global heat transfer coefficient of $U_{HX} = 75$ W/(m^{2}K) and a valve with $K_{JT} = 500$ GPa s^2/kg^2 gives the thermal power exchange and the vented flow rate in Fig. 9(b).

Model predictions are thus possible once these three parameters are provided. We consider these the model closure for the digital twin, hence $\boldsymbol{p} = \boldsymbol{w}_p = [U_s, U_{HX}, K_{JT}]$. These are to be inferred from real-time data by minimizing the following cost function.

$$\mathcal{J}_p = \frac{1}{T_o}\int_0^{T_o} \alpha_3(\check{T}_s(t) - T_s(t))^2 + \alpha_4(\check{V}_l(t) - V_l(t))^2 + \alpha_5(\check{p}_l(t) - p_l(t))^2 dt\,, \tag{50}$$

where the parameters $\alpha_3, \alpha_4, \alpha_5 \in \mathbb{R}$ are used to give comparable weight to these terms despite the largely different numerical values. To simplify the calibration of this digital twin model, in addition to the pressure sensor, we assume that a liquid level indicator is available, allowing for monitoring the liquid level $\check{V}_l(t)$, as well as temperature sensors to acquire $\check{T}_s(t)$.

Table 1
RMA Experiments: test cases configurations and hyper parameters settings.

	T_o (s)	dt (s)	n_e	N_z	n_Q	n_A	n_G	n_{mb}	P_π	P_w	T_J	T_w
T1 (Section 5.1)	600	$5{\cdot}10^{-1}$	20	5	500	1000	15	10	2	3	1	0.5
T2 (Section 5.2)	5	$5{\cdot}10^{-4}$	20	5	1000	1000	15	5	4	4	1	0.5
T3 (Section 5.3)	4.32 $\cdot 10^5$	30	20	1	1000	1000	30	5	4	20	1	0.5

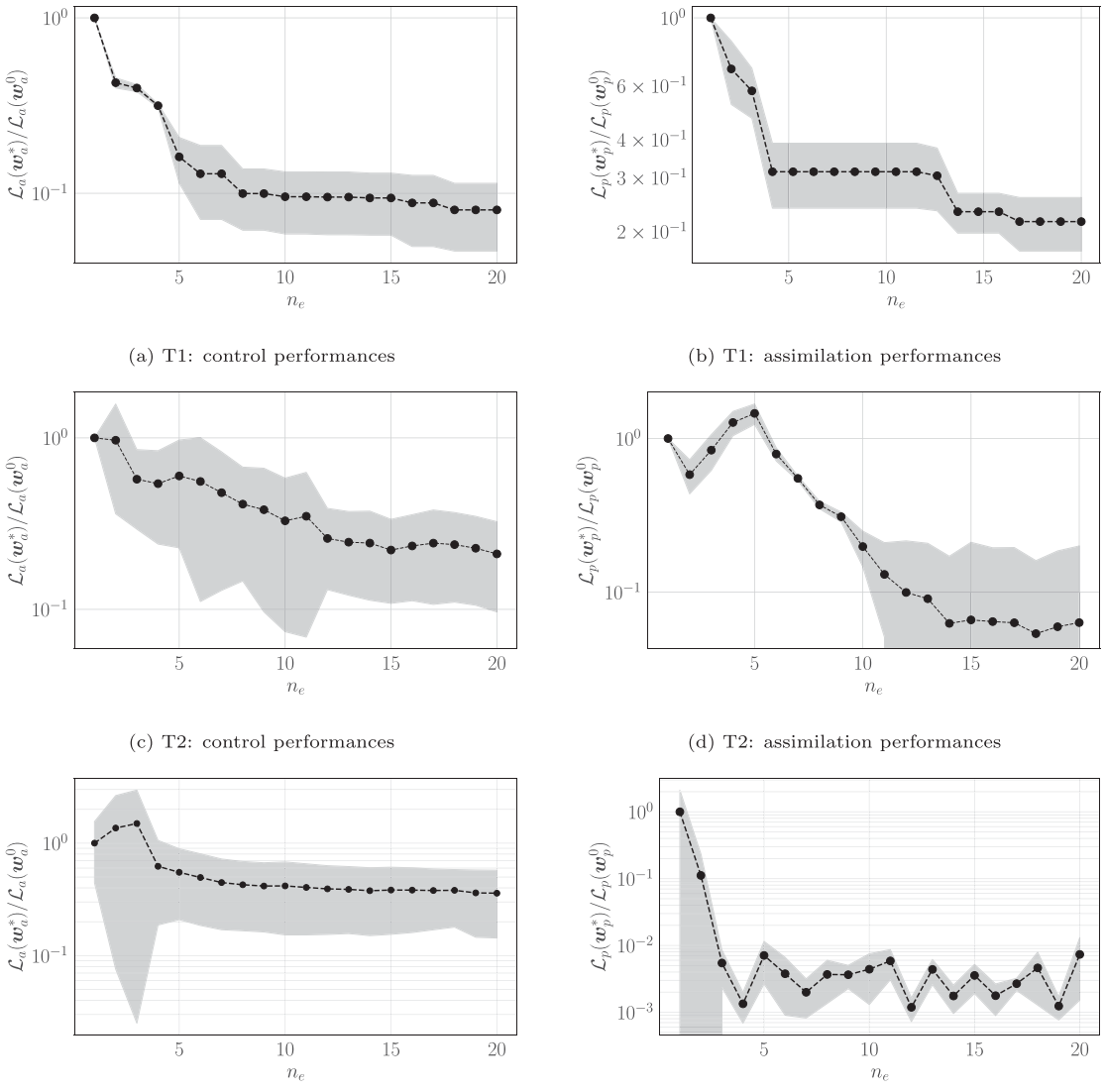
6. Results

This section presents the performances of the RT algorithm in controlling and assimilating a wind turbine subject to time-varying wind speed (T1, Section 5.1), a flapping wing micro air vehicle (FWMAV) against head wind (T2, Section 5.2) and a cryogenic tank exposed to fluctuating thermal load (T3, Section 5.3). The hyper-parameters selected for each test case are listed in Table 1. We use five different random seeds for our simulations to extract performance statistics across different initial states of the optimizers. For both T1 and T2 we take the gradients ensemble over $N_z = 5$ realizations, whereas in T3 we considered $N_z = 1$, given the lower level of noise in the exogenous disturbance. Finally, for all test cases, we set $n_S = 1$, as mentioned in Section 4.3.

Section 6.1 presents the overall learning performances for both the twinning and the controller for all test cases, while the following sections report on the digital twins predictions and the identified control laws for each test case.

6.1. Learning performances

The learning performances for all the three cases are reported in Fig. 10 and their numerical values tabulated in Table 2. The Figure shows the mean control or assimilation performances (first or second



(a) T1: control performances

(b) T1: assimilation performances

(c) T2: control performances

(d) T2: assimilation performances

(e) T3: control performances

(f) T3: assimilation performances

Fig. 10. Control $\mathcal{L}_a(\boldsymbol{w}_a)$, and assimilation performances, $\mathcal{L}_p(\boldsymbol{w}_p)$. Both are normalized by the initial cost, $\mathcal{L}_a(\boldsymbol{w}_a^0)$ and $\mathcal{L}_p(\boldsymbol{w}_p^0)$, respectively.

Table 2
RMA Learning performances, normalized by the mean initial cost.

	$\mathcal{L}(\boldsymbol{w}_p^*)/\mathcal{L}(\boldsymbol{w}_p^0)$	$\mathcal{L}(\boldsymbol{w}_{a,mb}^*)/\mathcal{L}(\boldsymbol{w}_{a,mb}^0)$	$\mathcal{L}(\boldsymbol{w}_{a,mf}^*)/\mathcal{L}(\boldsymbol{w}_{a,mf}^0)$
T1 (Section 5.1)	(2.1 ± 0.8) $\cdot 10^{-1}$	(**0.8 ± 0.5**) $\cdot 10^{-1}$	(1.3 ± 1.2) $\cdot 10^{-1}$
T2 (Section 5.2)	(3.2 ± 4.9) $\cdot 10^{-2}$	(4.18 ± 0.01) $\cdot 10^{-1}$	(**2.01 ± 0.45**) $\cdot 10^{-1}$
T3 (Section 5.3)	(3.15 ± 0.9) $\cdot 10^{-4}$	(**4.47 ± 2.68**) $\cdot 10^{-1}$	(4.80 ± 3.15) $\cdot 10^{-1}$

column, respectively) with a dashed line, and a confidence interval with a standard deviation above and below the mean performances. The proposed RT algorithm learns to control effectively the system in a limited number of iterations for all test cases, showing good sample efficiency. Indeed, the optimized policy performs up to ten times better than the initial one, as shown by the average control performance ($\mathcal{L}_a$). The large standard deviation at the beginning of the training of T2 and T3 can be explained by the initial exploration phase of the model-free loop. However, the standard deviation gets roughly constant as the policy optimization converges to a specific region of the search space, as shown in Figs. 10(c) and 10(d). Analysing the adjoint-based assimilation performances, Figs. 10(b), 10(d) and 10(f) show that the methodology is able to tailor the digital twin on the current system specifics within five to ten iterations. At the end of the training process, the assimilation reduces the initial cost $\mathcal{L}_p^{(0)}$ from the initial guess $\boldsymbol{w}_p$ by a factor 10, 100 and 10000 for T1, T2 and T3, respectively.

This, in turn, reflects on the model-based policy optimization (Step 5). In fact, as the digital twin predictions of the system response improve, the quality of the policy optimized "offline" increases. This also affects the policy switch (Step 6), since the performances on the twin become more representative of the real responses. The effect of Step 6 is illustrated in Fig. 11. All experiments begin with the model-free policy driving the interactions with the real environment, i.e. as "live policy", while the model-based is initially set as "idle". As shown in Fig. 11, the switch between model-free and model-based policies occurs at $n_e = 7$ in T1 and $n_e = 4$ for both T2 and T3, as $C_\pi > P_\pi$

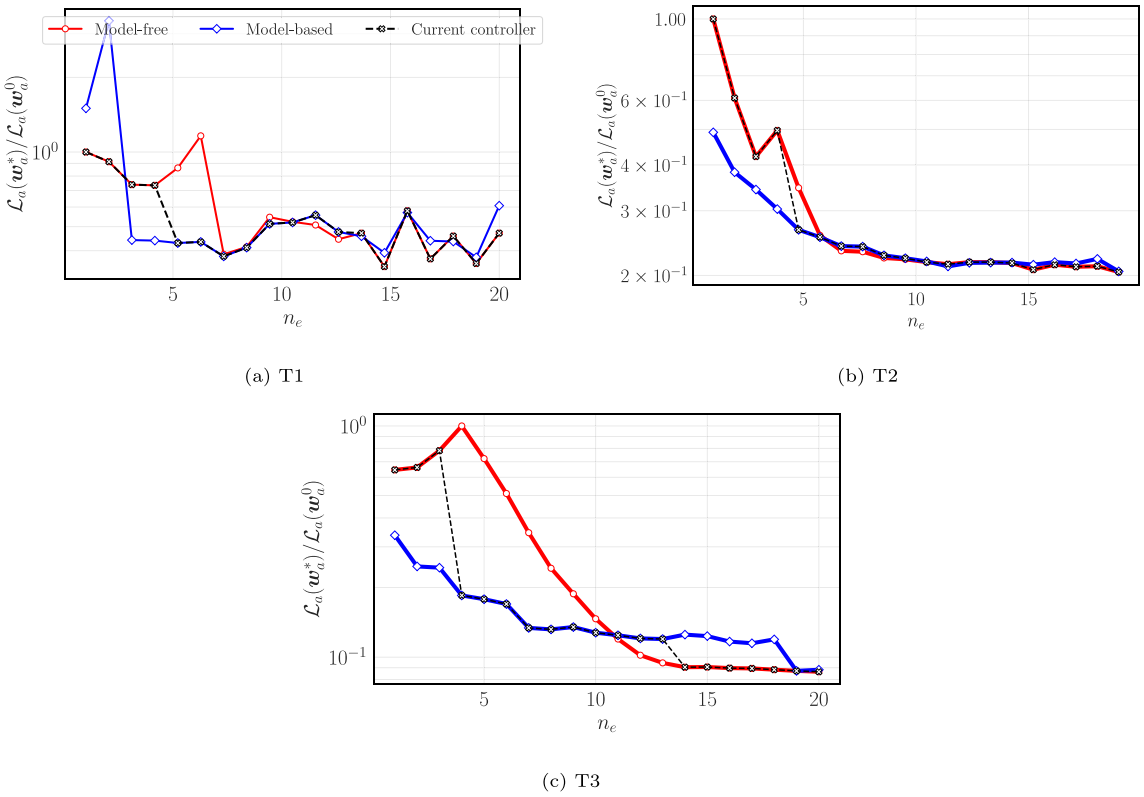


Fig. 11. Policy switch detail. Switch between "live" and "idle" policies, during interactions with the real system.

in the policy switch in Fig. 3. After the switch, the model-free control performance registers a steeper improvement despite remaining idle. This is due to a combination of factors: (1) the model-based acts as an "expert" for the model-free, providing high quality samples recorded in $\mathcal{D}_{\mathcal{L}}$ from which the model-free loop can learn; and (2) the critic function approximation gets better at predicting the value associated to the recorded transitions. Interestingly, in both T1 and T3 (Fig. 11(a), 11(c)), the model-free learns a better policy by imitating the model-based counterpart, eventually "surpassing the master" and earning again the status of a "live" policy again within a few episodes.

Comparing the performances of the model-based and model-free loops, the model based performs slightly better than the model based in T1, but worst in T2 and equally in T3, where they converge to the same policy. The different performances in the different cases highlights the benefits of having two approaches to tackle the same problem and the merit in their combination.

6.2. Performances on wind turbine control

The digital twin performances on T1 are shown in Fig. 12. Fig. 12(a) shows the average prediction versus the average system evolution as a function of time, while Fig. 12(b) plots one versus the other. We conclude that the digital twin successfully tracks the dynamics of the real system albeit with a systematic under-estimation of roughly 1 (rad /s) or 9.55 (RPM), especially at low rotational speeds. Whether this bias is inherently linked with the chosen parametrization or an insufficient number of samples requires further investigation. Differently from T2 and T3, the coefficients to be identified are unknown *a priori*, as the initial coefficients $\boldsymbol{w}_p^0$, derived in the work of [160], describe a power curve in steady wind conditions. Therefore, the average assimilation loss $\mathcal{L}_p \approx \mathcal{O}(10^{-1})$ highlights the capability of the algorithm to adapt online, possibly taking into consideration changes in the operative conditions of the system itself.

The control performances are summarized in Fig. 13. The controller is able to keep the system close to the desired set-points, as shown in Figs. 13(a) and 13(b). This is especially true for the power tracking, always kept very close to $\tilde{P}$ despite the noticeably varying wind speeds (Fig. 13(b)). On the other hand, the controller consistently keeps the generator speed (ω_g) below-rated conditions by approximately 8 rad/s. This lower-than-rated rotational speed reflects on the actuated torque, in Fig. 13(d), which is always in the middle between the allowed action space. In addition, the very fast actuation rate on the torque ($\dot{\tau} = 15$ kN/s) allows it to rapidly adapt in case of abrupt changes in the wind speed, while fine tuning the actuation with the pitch angle β. The latter follows consistently the wind speed, which is desirable. At higher wind speeds than rated ($u_\infty > 11.4$ m/s) it consistently plays on the blades' angle to reduce the aerodynamic torque to keep the system close to desired conditions. This aerodynamic de-rating of the rotor is visible in Fig. 14 which shows a peak of C_p at rated conditions ($u_\infty \approx 11$ (m/s)) with a $C_p = 0.42$, steadily decreasing for higher winds.

We note that the interplay between the two set-points on power and generator speed could be the modified fine-tuning the normalization coefficients α_1 and α_2, as commonly done in multi-objective optimization problems.

The model-based and model-free policies are compared in Fig. 15. The two optimization loops converge to essentially the same parametrizations, leading to the same policy functions, as also supported by the very similar mean cost functional $\mathcal{L}_a$ listed in Table 2. The only (minor) difference appears in the threshold of torque actuation. In both cases, when the error on the rotational speed goes negative ($e_i = \omega_g - \tilde{\omega}_g < 0$), the controller reduces the pitch angle (Figs. 15(a),15(c)) to increase the aerodynamic efficiency and, consequently, the aerodynamic torque, with the goal of increasing the rotational speed to get closer to the set point. The opposite is true when the generator is rotating too fast: the controller increases the angle of attack of the blades to slow the generator down. The integral of the error over the past revolution, $\int e$, essentially decides on the slopes of the

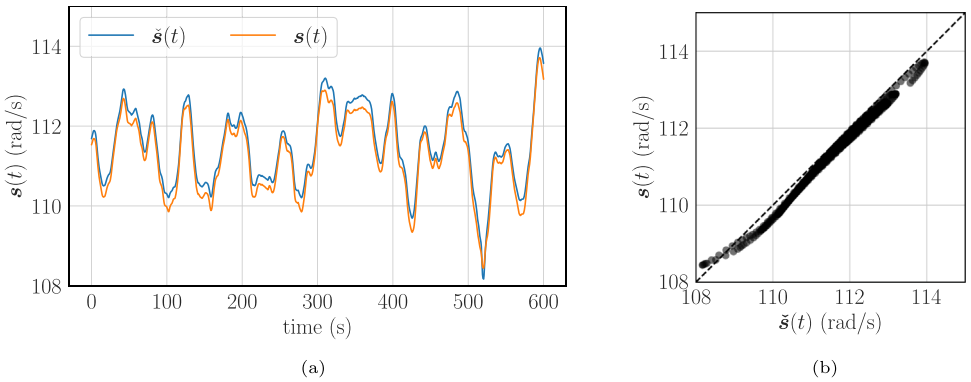


Fig. 12. T1 - Digital Twin agreement with real system. Evaluation of the optimal $\boldsymbol{w}_p = \boldsymbol{w}_p^*$ with randomized initial conditions and disturbances. Time-series (a) and scatter plot (b), plotting real state on the x-axis and predicted one on the y-axis.

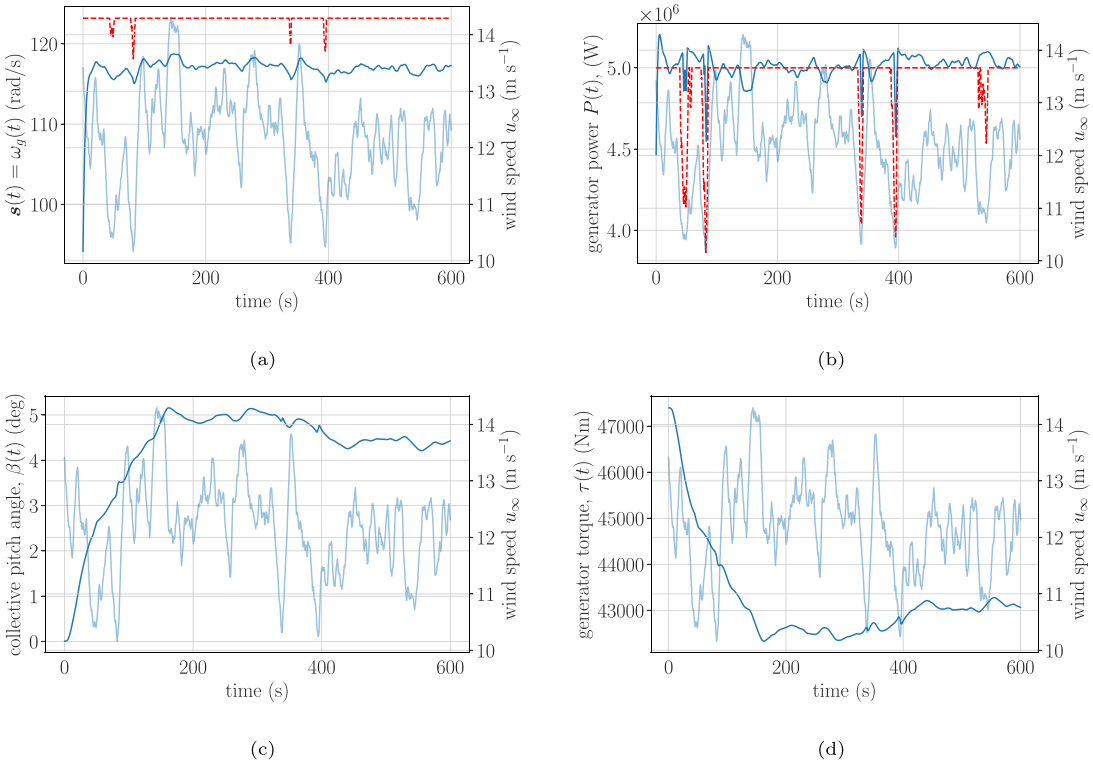


Fig. 13. T1 - Control performances. System evolution against set-points (dashed red line) in (a) and (b), filtered actuation in (c) and (d).

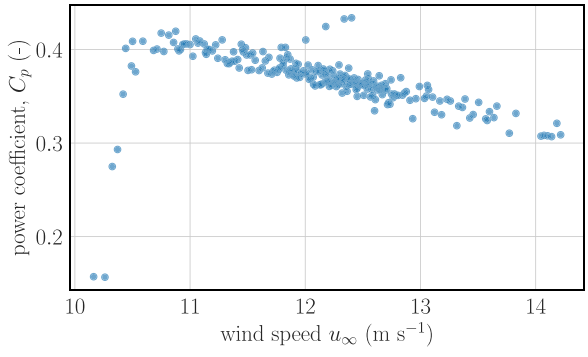
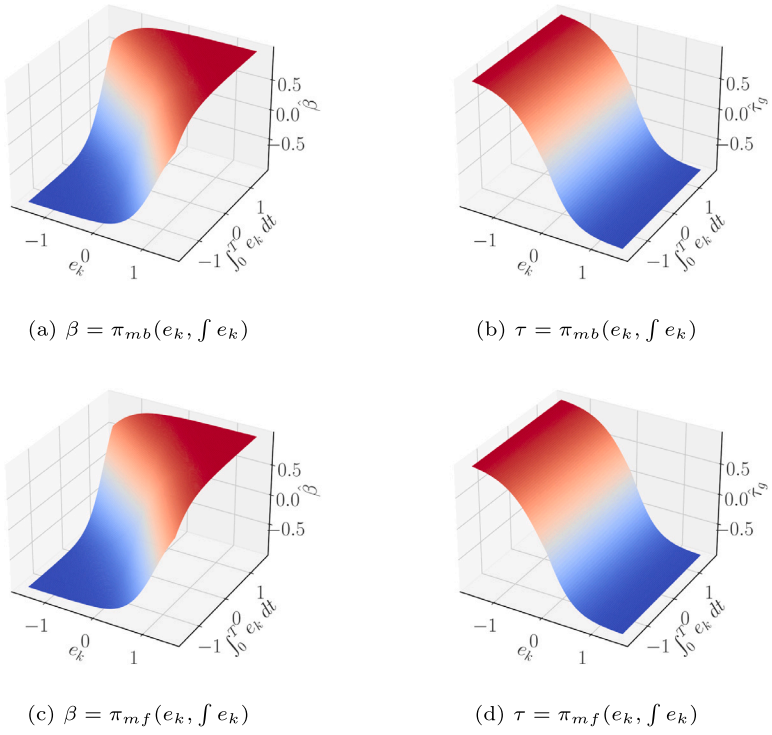


Fig. 14. T1 - Power coefficient (C_p) against operative wind speeds.

control surface. Interestingly, looking at the torque actuation shown in Figs. 15(b) and 15(d), it seems that this control actuation is almost insensitive to this control input, relying on the scalar error solely. This control surface is directly connected with the objective of keeping the power around the rated value of $P = 5$ MW. In fact, as the error becomes negative, the controller increases the torque to keep the power at the end of the generator constant ($P_g = \omega_g \cdot \tau_g$). On the other hand, when the error increases, it reduces the resistive torque, relying on the pitch actuator to reduce the speed while keeping the power constant.

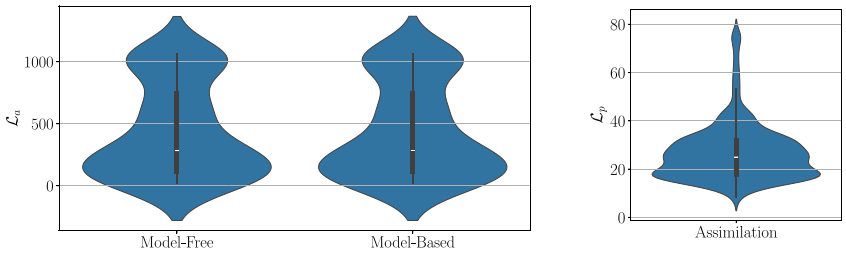
We conclude this analysis by investigating the robustness of the identified models and the control laws. Fig. 16 shows the distribution of the average (a) control cost function for both model based and model free and (b) assimilation, obtained by running one-hundred simulations on randomized initial conditions and wind profiles (unseen during the training phase) and using the identified model parameters $\boldsymbol{w}_p^*$ and policy parameters $\boldsymbol{w}_{a,mb}^*$, $\boldsymbol{w}_{a,mf}^*$. The control performances shows a multi-modal distribution in Fig. 16(b), with the strongest peak in the region of better performances ($\mathcal{L}_a < \mathcal{O}(10^2)$). This shows that at least



(a) $\beta = \pi_{mb}(e_k, \int e_k)$ (b) $\tau = \pi_{mb}(e_k, \int e_k)$

(c) $\beta = \pi_{mf}(e_k, \int e_k)$ (d) $\tau = \pi_{mf}(e_k, \int e_k)$

Fig. 15. T1 - Normalized policy contours. Model-based (a - b) and model-free (c - d) actors, as a function of the error $e_k(t) = \omega_g(t) - \tilde{\omega}_g(t)$ and its integral over the past revolution.



(a) (b)

Fig. 16. T1 - Robustness analysis of the control policies (a) and assimilated closure (b).

one local minima exists within the range of investigate conditions and the training can eventually fall into the sub-optimal one depending on the combination of initial condition and wind profiles. A possible solution to mitigate the sensitivity to the training condition could be to extend the training to a broader range of wind profiles and/or initial conditions.

The assimilations performances, on the other hand, are reliably on-track with respect to what experienced during the learning phase. The digital twin tracks the real system with very small deviation ($\mathcal{L}_p < \mathcal{O}(10^2)$) for most experienced operative conditions.

6.3. Performances on a FWMA trajectory control

In this test case, the initial weights $\boldsymbol{w}_p^0 = [a, b, c]$ ((41) and (42)), are drawn from three random distributions, and a stopping criterion is defined to interrupt the n_G iterations if the cost falls below a given threshold (10^{-2}). Fig. 17(a) shows the assimilation performance of the digital twin in predicting the real-system behaviour. The two show a small deviation. This can be better appreciated in Fig. 17(b), which compares the real and predicted states.

Fig. 17(b) characterizes the identified optimal policy, depicting state variables (18(a) and 18(b)) and control actions 18(c). The figures evidence that the drone managed to closely reach its target ($\boldsymbol{x}_f = 5$ m and $\dot{\boldsymbol{x}}_f = 0$, as fast as possible) thanks to the control of the stroke plane angle β_w and the flapping amplitude A_ϕ. The control of the latter is mainly formed of three phases in a kind of bang–bang fashion. First, the controller sets the drone's wings at their maximum angle of attack ($A_\phi = 88$ deg) to reach the target position as fast as possible. Then, it decelerates and stabilizes the drone picking a flapping angle close to the minimal one (55 deg), with a minimal overshoot. Finally, the wings' angle of attack is increased again, to balance the body weight with the aerodynamic lift. This force is also slightly modulated by the other control variable (β_w) which mainly drives the horizontal motion of the drone. In the same way, the controller pitches the stroke plane up to the maximum angle ($\beta_w = -30$ deg) and then stabilizes it around an angle that allows it to stay close to the desired position, x_f, despite the headwind. The influence of the gust is visible on β_w even though the wind effect is still very limited, due to the selected mass and surface of the body.

Fig. 18(c) analyses the policies found by the model-free (Fig. 19(a)–19(d)) and model-based (Fig. 19(e)–19(h)) loops. To this end, we fix two of the four inputs to zero and span on the other two dimensions, recording the corresponding action. The two policies are remarkably similar, converging to essentially the same parametrization. The main

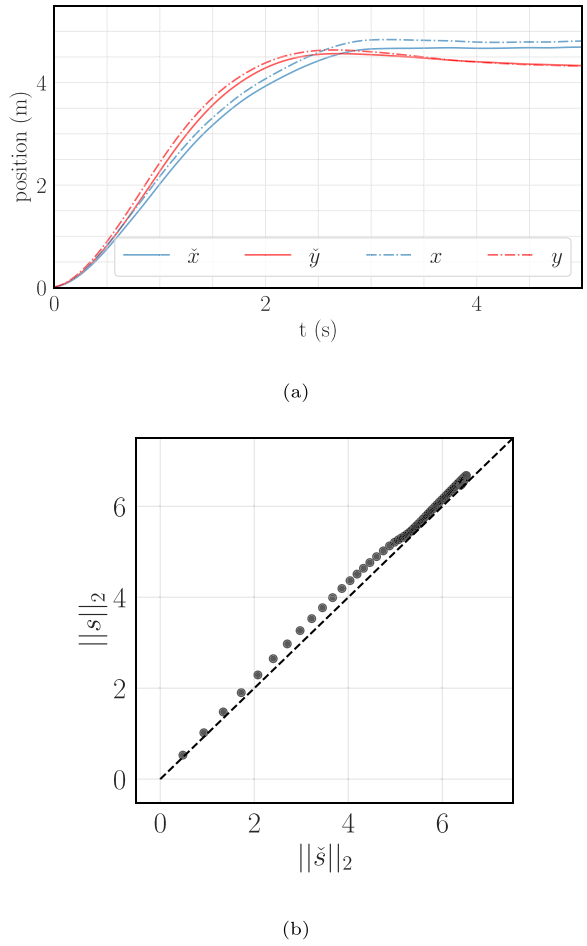


Fig. 17. T2 - Comparison of the time evolution of the drone position for the real environment (solid lines) and the digital twin (dashed line). The right plot shows the high correlation between the norm of the predicted state $\boldsymbol{s} = [\dot{x}, \dot{y}, x, y]$ and the real state $\check{s}$ for an episode length.

difference between the two lies in the concavity of the hyperbolic tangent, as dictated by the biases. This affects the localization of the switching between the two bounds of the actuation space, possibly explaining the differences in their associated costs presented in Table 2. In both cases, for what concerns $\hat{A}_\phi$, the border between the two bounds lies on the diagonal that goes from $(\hat{e}_x, \hat{e}_x) = (-1.0, -1.0)$ to $(\hat{e}_x, \hat{e}_x,) = (0.25, 1.0)$, see Fig. 19(a). The same trend is observed in Fig. 19(b), which analyses the dependencies from e_z and its time derivative. In this case, however, the upper point of the diagonal lies around $(\hat{e}_z, \hat{e}_z) = (0.0, 1.0)$, thus increasing $\hat{A}_\phi$ as the rate of change of e_z goes negative, and decreasing it otherwise. Interestingly, the model-based policy here shows a broader decision zone, as depicted in Fig. 10(f). Now, analysing the actuation on β_w, the threshold appears to be positioned in the diagonals $(\hat{e}_x, \hat{e}_x) = (0.0, -1.0)$ to $(\hat{e}_x, \hat{e}_x) = (0.75, 1.0)$. The sensitivity to the $z-$errors instead show it to lie in $(\hat{e}_z, \hat{e}_z) = (-1.0, 0.0)$ and $(\hat{e}_z, \hat{e}_z) = (1, 0.5)$. Therefore, it seems that the time variation of the rate of change is the most important input on the $x-$dimension, whereas in the $z-$coordinate, it is the scalar $\hat{e}_z$. Arguably this may be caused by the headwind, oriented as opposite of the x-axis. Overall, there appears to be two main mechanisms at play: (1) the errors and their rate of change seems to have the same importance when deciding how to control on $\hat{A}_\phi$ being slightly more sensitive to the scalar instantaneous errors; and (2) when deciding on how to actuate β_w, the controller balances the scalar instantaneous error in one dimension and its rate of change in the other. In fact, when $\dot{e}_x$ increases, it actuates on β_w to get back to nominal conditions, and the same is observed on analysing the reactivity to the instantaneous error e_z and e_x.

We conclude the analysis of the RT algorithm performances on the FWMAV presenting the robustness study of the model-free and model-based policies, shown in Fig. 20. One-hundred episodes were run on the real environment for each policy and their costs are compared with violin plots (Fig. 20(a)). Both sets of weights result in a small cost proving their capability to handle the random gust. The model-free policy manages to reach on average a lower cost and shows a slightly different distribution than the model-based policy, having two peaks at $\mathcal{L}_a = 905$ and $\mathcal{L}_a = 920$, whereas the model-based shows a main peak in the middle, at $\mathcal{L}_a = 910$. However, their differences are in the order $\mathcal{O}(10^1)$, reinforcing the observation that the two have converged to a very similar policy. Lastly, the same procedure is applied on the assimilation side, studying its robustness to varying operative conditions, as shown in Fig. 20(b). It appears that the identified closure function is robust to different operative conditions, registering a mean cost of $\mathcal{L}_p \approx \mathcal{O}(10^{-3})$.

6.4. Performances in cryogenic storage's thermal management

At last, we examine the RT results on the cryogenic thermal management test case.

The digital twin assimilation consists of identifying the weights $\boldsymbol{w_p} = [U_s, U_{HX}, K_{JT}]$ which drive the heat transfer between the cryogenic tank and the environment, as well as the performance of the active TVS. For all simulations, this vector was initialized taking $\boldsymbol{w}_p^0 = [1, 1, 10]$, with the units of the first two components being (W/m^{2}K), and the last being (GPa·s^2/kg^2).

The agreement between the predicted state and the real one is presented in Fig. 21, which shows both the time series of the real and predicted states (Fig. 21(a)–21(c)) and their direct comparison plotting one against the other (Fig. 21(d)–21(f)). All three components are successfully identified, even though slight deviations are found for $\boldsymbol{s}_1 = p$ and $\boldsymbol{s}_2 = T_s$.

The control performances of the optimal parametrization, $\boldsymbol{w}_a^*$, extracted from the learning curve presented in Fig. 10(d) are studied next. The current scenario involves an initial over-pressurized tank operating 0.4 bar above the reference $\tilde{p}$. Fig. 21(f) shows the time evolution of the real pressure $\check{p}$ and propellant mass $\check{m}_l$ over time, while the vented mass flow rate $\check{m}_c$. Figs. 22(b) and 22(c) show the control action over time and the corresponding pressure, respectively. Since the tank is initially in an over-pressurized state sufficiently far from $\tilde{p}$, the valve was regulated to open to its widest ($\dot{m}_c = 0.63$ g/s), allowing the pressure to rapidly reduce until the target 5 bar are reached. This pressure drop is accompanied by a consumption of 87 kg over two roughly 45 h. Once the target is reached, the active TVS regulates the pressure around this value. To do so, $\check{m}_c$ fluctuates between −50% and 25% around its midpoint due to changes in the heat due to the disturbance, although such variations are not seen in the pressure signal.

From these findings, we conclude that the control policy is able to completely control the tank pressure around its desired target value. However, the response time of the TVS is limited by the technical constraints assigned to the circulating pump and Joule-Thomson valve. In addition, depending on the value assigned to the K_1 constant in (47), one could retrieve either faster or more fuel-efficient policies.

We inspect the two found policies in Fig. 23. The two loops, model-based and model-free, converge to essentially the same parametrization. They show almost identical responses to the possible input space, with very minor discrepancies in the inclination of the hyperbolic tangent. Lastly, the robustness of the assimilation and control loops concludes this overview. The procedure consists in evaluating the cost of the optimal weights $\boldsymbol{w}_p$ and $\boldsymbol{w}_a$ over observations unseen during the training phase. To this end, we carried out one hundred simulations

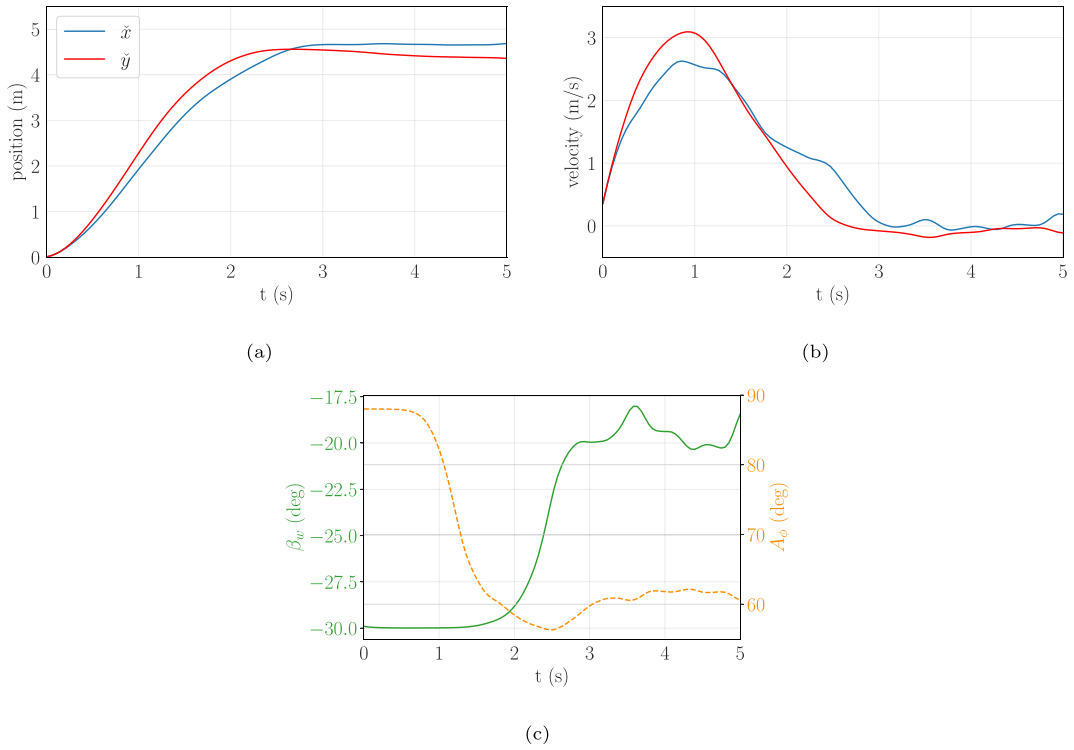


Fig. 18. T2 - Time evolution of the positions x and y (a) and the velocities $\dot{x}$ and $\dot{y}$ (b) of the flapping wing micro air vehicle driven by the control actions $\boldsymbol{a} = [A_\phi, \beta_w]$ (c). The optimal control policy manages to lead the drone to its target $(x, y) = (5, 5)$ and $(\dot{x}, \dot{y}) = (0, 0)$ within one episode time with minor oscillations that results from the gust.

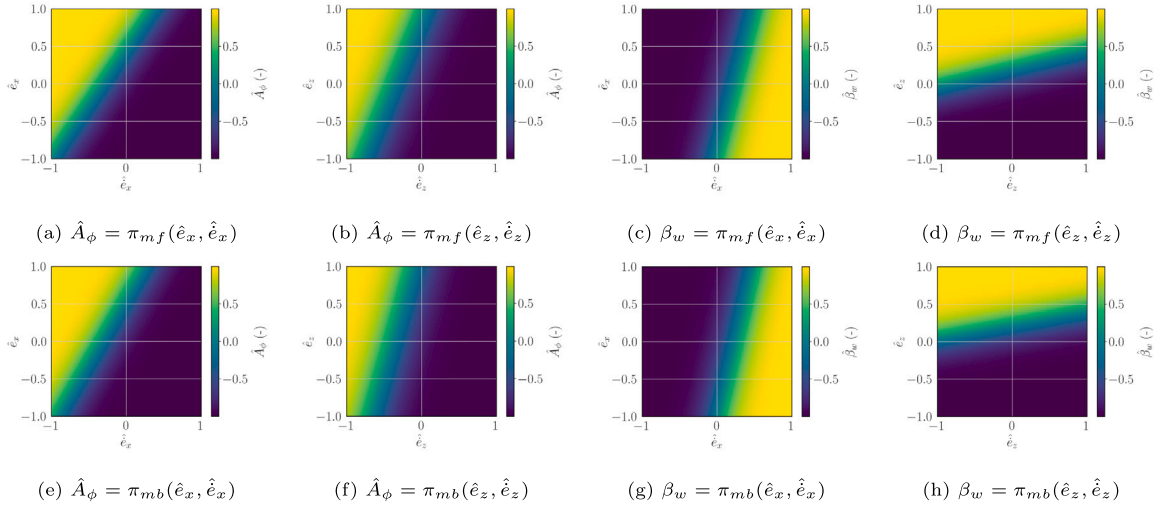


Fig. 19. T2 - Comparison of model-based and model-free policies.

on random initial conditions and disturbances. For each of them, we computed the assimilation cost $\mathcal{L}_p$, as well as the control cost $\mathcal{L}_a$ using the model-based and model-free policies. This investigation, summarized in the violin plots presented in Figs. 24(a) and 24(b), proves that the weights obtained at the end of the learning phase are valid and robust even in unseen scenarios. Moreover, in contrast to what was observed in the wind turbine test case, the model-based and model-free weights are nearly identical, thus their performances are also identical as supported by Table 2, which shows a difference between the two control performances in the order of $\mathcal{O}(10^{-2})$. This is likely a consequence of both approaches converging in the same region of the search space.

7. Conclusions and outlooks

We presented a novel framework to train physics-based digital twins from real-time data while at the same time training a control agent through a combination of model-free and model-based methods. We called this framework Reinforcement Twinning (RT) and positioned it at the intersection between data assimilation, system identification, and

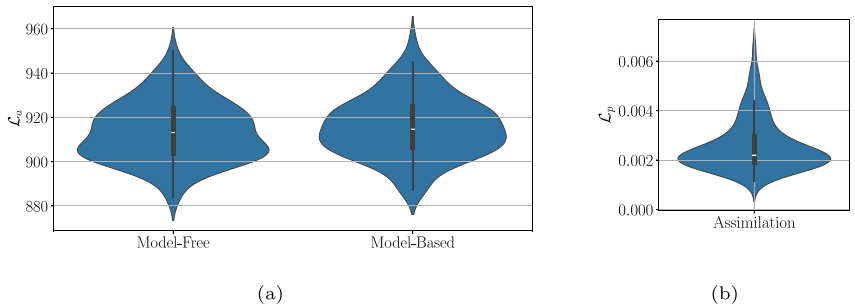


Fig. 20. T2 - Robustness analysis of model-free and model-based policies (a) and assimilated closure (b), compute over 100 episodes.

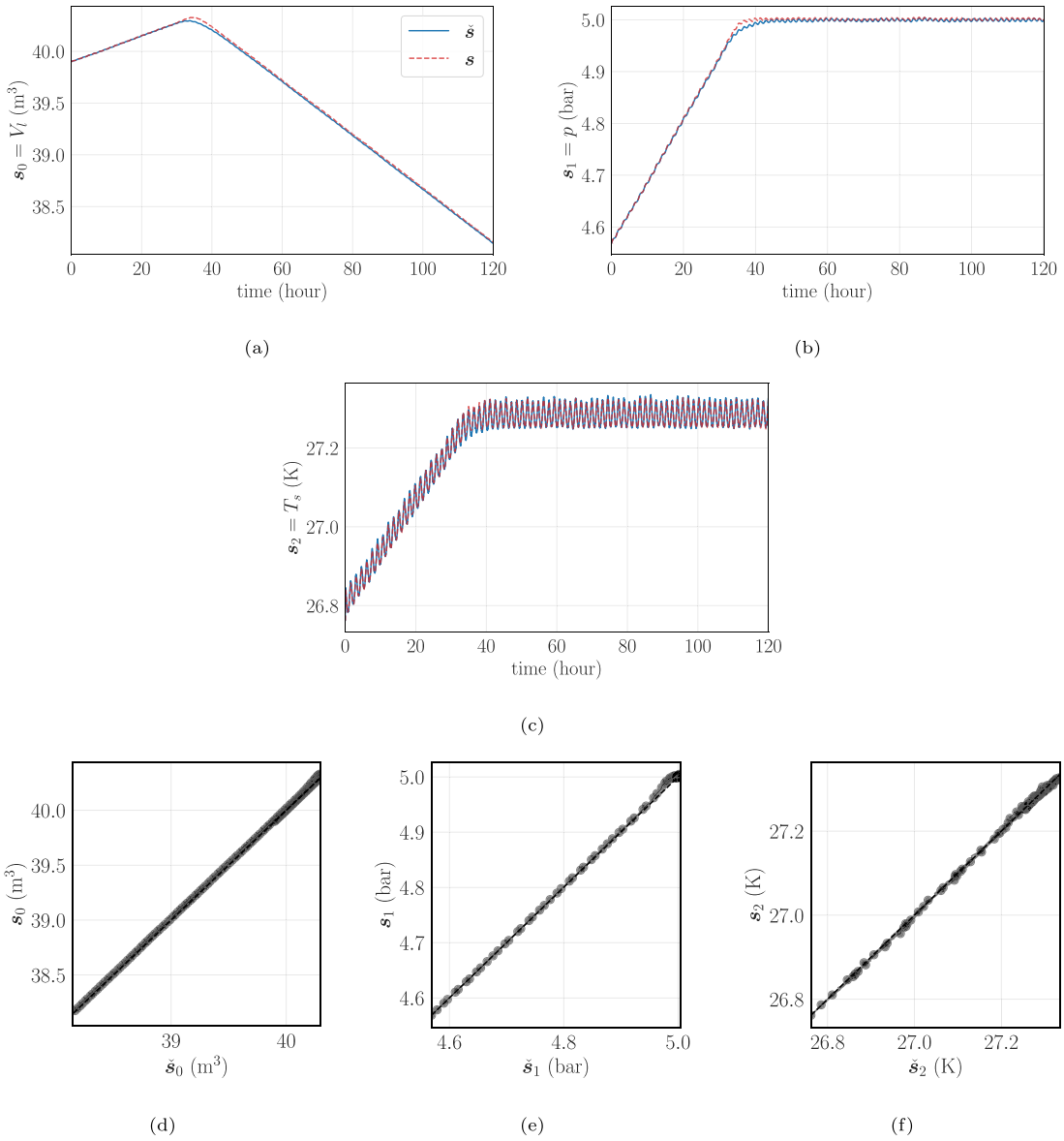


Fig. 21. T3-Performance of the digital twin assimilation. (a) Cost function $\mathcal{L}_p$ normalized with its initial value $\mathcal{L}_p(\boldsymbol{w}_p^0)$ over the number of episodes. Agreement of the predicted state $\boldsymbol{s} = [V_l, p, T_s]$ with the real state $\check{\boldsymbol{s}}$ shown through (b) a scatter plot, and (c) the time series evolution.

reinforcement learning. The digital twins are formulated as physics-based models relying on closure laws expressed as parametric functions. The parameters defining these closures are identified from real-time data using an adjoint-based data assimilation strategy.

Similarly, the control agent acts on the system via a policy, also defined via parametric functions. The same adjoint-based framework is then used on the digital twin to identify the control policy and thus train the agent in the so-called "model-based" training. At the same time, a reinforcement learning approach using the Deep Deterministic Policy Gradient (DDPG) pursues the same task of training the control agent without using explicit knowledge of the system in the so-called

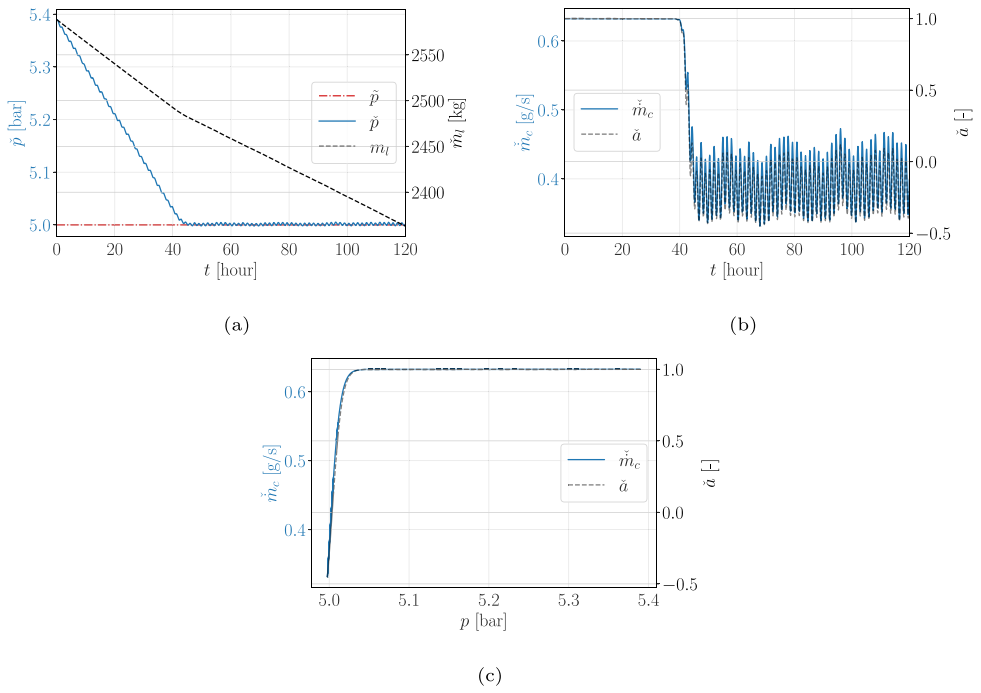


Fig. 22. T3-Tracking performance of the pressure $\check{p}$, and variation of propellant mass $\check{m}_l$ over the real environment, starting from an over-pressure condition. Corresponding control action $\check{a}$ and vented mass flow rate $\check{m}_c$. Thermodynamic actuation $\check{m}_v$ and $\check{a}$ as a function of the pressure p. The acting policy $\pi(\check{\boldsymbol{e}};\boldsymbol{w}_a)$ is retrieved by taking the optimal set of weights $\boldsymbol{w}_a$ from the training.

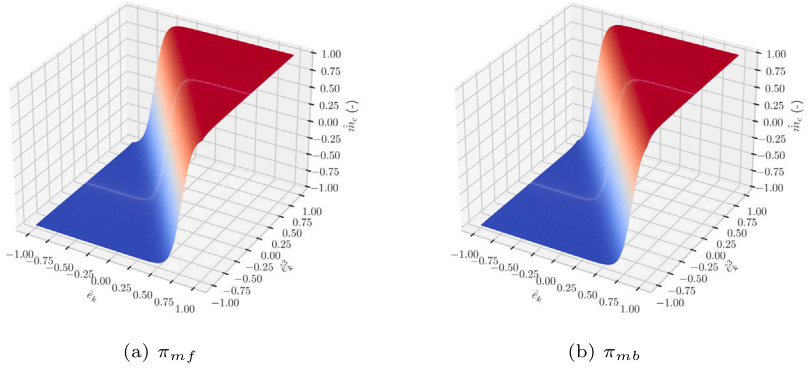


Fig. 23. T3-Surface plot of the found policies, π_{mf} 23(a) and π_{mb} 23(b).

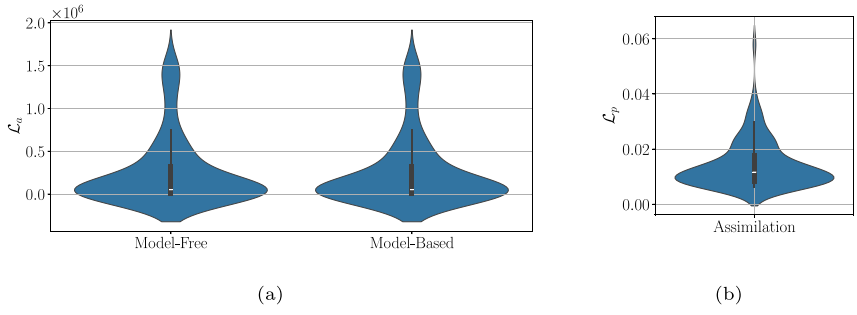


Fig. 24. T3-Robustness analysis of model-free and model-based policies (a) and assimilated closure (b), computed over 100 episodes.

"model-free" training. The "model-based" and the "model-free" policies are then tested on the digital twin once this has reached satisfying predictive performances and the best-performing policy becomes the "live policy" taken on the real system. We explored a first mechanism to let both the "model-based" and the "model-free" learn from the live policy and showed that this can create a virtuous loop in which both approaches lead to a sequence of improvements over the other.

We tested this novel framework on three widely different engineering systems: a wind turbine in a time-varying wind speed, a flapping-wing micro air vehicle travelling against gusts, and the thermal management of a cryogenic tank subject to time-varying heat loads. In these

problems, the physics-based model of the digital twin is formulated as ODEs ensuring the compliance of predictions with fundamental principles (conservation of angular momentum in a wind turbine, Newton's law in the drone's trajectory, and conservation of mass/energy within the cryogenic tank) and the closure law describe unmodelled physics (aerodynamic coefficients and heat transfer correlations).

The performance of the Reinforcement Twinning framework, both in training the digital twin and in the training of the controlling agent, proves the framework's potential beyond any specific application. The merit stems from the complementary characteristics of the two approaches: the model-based is extremely sample-efficient but prone to fall stuck into local minima, while the model-free is sample-inefficient but able to better explore the parameter space and avoid model limitations.

The paths towards future extensions are many. On the one hand, we could introduce a more advanced learning feedback mechanism between the model-free and the model-based methods, which is currently limited to a "cloning" procedure. This could allow, for example, different policy architecture between the two, unleashing the potential of reinforcement learning to handle arbitrarily complex policies in the form of artificial neural networks. On the other hand, we could introduce more sophisticated methods of integrating the digital twin in the training of the model-free loop, for instance, by using the virtual environment to boost the training of the critic network with strategies akin to traditional model-based reinforcement learning.

Nevertheless, these preliminary results open the path towards further hybridization of model-based and model-free methods and, more broadly, between traditional engineering and machine learning. Such a combination is essential to profit from the ever-growing availability of real-time data to achieve excellent predictive and control capabilities without sacrificing interpretability.

CRediT authorship contribution statement

Lorenzo Schena: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Pedro A. Marques:** Writing – review & editing, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Romain Poletti:** Writing – review & editing, Writing – original draft, Software, Investigation, Formal analysis, Conceptualization. **Samuel Ahizi:** Writing – review & editing, Writing – original draft, Investigation, Formal analysis, Conceptualization. **Jan Van den Berghe:** Writing – review & editing, Validation, Software, Investigation. **Miguel A. Mendez:** Writing – review & editing, Writing – original draft, Supervision, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization.

Declaration of competing interest

We confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Data availability

Data will be made available on request.

Acknowledgements

L. Schena and R. Poletti are supported by Fonds Wetenschappelijk Onderzoek (FWO), while P. Marques and J. van Den Berghe are supported by the FRIA grant from the 'Fonds de la Recherche Scientifique (F.R.S. -FNRS)'. S.Ahizi and M.A. Mendez were supported by the Flemish Agentschap Innoveren & Ondernemen in the framework of the CSBO project "Clean Hydrogen Propulsion for Ships (CHyPS)". All authors approved the version of the manuscript to be published.

References

[1] R. Wagner, B. Schleich, B. Haefner, A. Kuhnle, S. Wartzack, G. Lanza, Challenges and potentials of digital twins and Industry 4.0 in product design and production for high performance products, Procedia CIRP 84 (2019) 88–93, http://dx.doi.org/10.1016/j.procir.2019.04.219.

[2] B.R. Barricelli, E. Casiraghi, D. Fogli, A survey on digital twin: Definitions, characteristics, applications, and design implications, IEEE Access 7 (2019) 167653–167671, http://dx.doi.org/10.1109/access.2019.2953499.

[3] F. Chinesta, E. Cueto, E. Abisset-Chavanne, J.L. Duval, F.E. Khaldi, Virtual, digital and hybrid twins: A new paradigm in data-based engineering and engineered data, Arch. Computat. Methods Eng. 27 (1) (2020) 105–134, http://dx.doi.org/10.1007/s11831-018-9301-4, URL: http://link.springer.com/10.1007/s11831-018-9301-4.

[4] A. Rasheed, O. San, T. Kvamsdal, Digital twin: Values, challenges and enablers, 2019, URL: http://arxiv.org/abs/1910.01719, arXiv:1910.01719 [eess].

[5] A. Ammar, H. Nassereddine, N. AbdulBaky, A. AbouKansour, J. Tannoury, H. Urban, C. Schranz, Digital twins in the construction industry: A perspective of practitioners and building authority, Front. Built Environ. 8 (2022) 834671, http://dx.doi.org/10.3389/fbuil.2022.834671, URL: https://www.frontiersin.org/articles/10.3389/fbuil.2022.834671/full.

[6] L. Wright, S. Davidson, How to tell the difference between a model and a digital twin, Adv. Model. Simul. Eng. Sci. 7 (1) (2020) http://dx.doi.org/10.1186/s40323-020-00147-4.

[7] B. Tekinerdogan, On the notion of digital twins: A modeling perspective, Systems 11 (1) (2022) 15, http://dx.doi.org/10.3390/systems11010015, URL: https://www.mdpi.com/2079-8954/11/1/15.

[8] A. van Beek, V.N. Karkaria, W. Chen, Digital twins for the designs of systems: A perspective, Struct. Multidiscip. Optim. 66 (3) (2023) http://dx.doi.org/10.1007/s00158-023-03488-x.

[9] A. Haghshenas, A. Hasan, O. Osen, E.T. Mikalsen, Predictive digital twin for offshore wind farms, Energy Inform. 6 (1) (2023) 1–26.

[10] J. Schoukens, L. Ljung, Nonlinear system identification: A user-oriented road map, IEEE Control Syst. 39 (6) (2019) 28–99, http://dx.doi.org/10.1109/mcs.2019.2938121.

[11] O. Nelles, Nonlinear System Identification, Springer, Berlin Heidelberg, 2001, http://dx.doi.org/10.1007/978-3-662-04323-3.

[12] L. Ljung, Perspectives on system identification, IFAC Proc. Vol. 41 (2) (2008) 7172–7184, http://dx.doi.org/10.3182/20080706-5-kr-1001.01215.

[13] G.D. Nicolao, System identification : Problems and perspectives, in: 12th Workshop on Qualitative Reasoning, 2003, URL: https://api.semanticscholar.org/CorpusID:26963940.

[14] M. Asch, M. Bocquet, M. Nodet, Data Assimilation, Society for Industrial and Applied Mathematics, 2016, http://dx.doi.org/10.1137/1.9781611974546.

[15] M. Bocquet, A. Farchi, Introduction to the Principles and Methods of Data Assimilation in the Geosciences, Technical Report, École des Ponts ParisTech, 2023.

[16] A.C. Lorenc, Analysis methods for numerical weather prediction, Q. J. R. Meteorol. Soc. 112 (474) (1986) 1177–1194, http://dx.doi.org/10.1002/qj.49711247414.

[17] B. Wang, X. Zou, J. Zhu, Data assimilation and its applications, Proc. Natl. Acad. Sci. 97 (21) (2000) 11143–11144, http://dx.doi.org/10.1073/pnas.97.21.11143.

[18] Z. Pu, E. Kalnay, Numerical weather prediction basics: Models, numerical methods, and data assimilation, in: Handbook of Hydrometeorological Ensemble Forecasting, Springer Berlin Heidelberg, 2018, pp. 1–31, http://dx.doi.org/10.1007/978-3-642-40457-3_11-1.

[19] W. Lahoz, B. Khattatov, R. Menard (Eds.), Data Assimilation, Springer, Berlin Heidelberg, 2010, http://dx.doi.org/10.1007/978-3-540-74703-1.

[20] O. Talagrand, P. Courtier, Variational assimilation of meteorological observations with the adjoint vorticity equation. I: Theory, Q. J. R. Meteorol. Soc. 113 (478) (1987) 1311–1328, http://dx.doi.org/10.1002/qj.49711347812.

[21] F.-X.L. Dimet, I.M. Navon, R. Ştefănescu, Variational data assimilation: Optimization and optimal control, in: Data Assimilation for Atmospheric, Oceanic and Hydrologic Applications, vol. III, Springer International Publishing, 2016, pp. 1–53, http://dx.doi.org/10.1007/978-3-319-43415-5_1.

[22] S.E. Ahmed, S. Pawar, O. San, PyDA: A hands-on introduction to dynamical data assimilation with Python, Fluids 5 (4) (2020) 225, http://dx.doi.org/10.3390/fluids5040225.

[23] G. Evensen, Data Assimilation: The Ensemble Kalman Filter, second ed., Spinger, 2009.

[24] M. Bocquet, Ensemble Kalman filtering without the intrinsic need for inflation, Nonlinear Processes Geophys. 18 (5) (2011) 735–750, http://dx.doi.org/10.5194/npg-18-735-2011.

[25] A. Routray, K.K. Osuri, S. Pattanayak, U.C. Mohanty, Introduction to data assimilation techniques and ensemble Kalman filter, in: Advanced Numerical Modeling and Data Assimilation Techniques for Tropical Cyclone Prediction, Springer Netherlands, 2016, pp. 307–330, http://dx.doi.org/10.5822/978-94-024-0896-6_11.

[26] E. Kalnay, H. Li, T. Miyoshi, S.-C. Yang, J. Ballabrera-Poy, 4-D-VAR or ensemble Kalman filter? Tellus A: Dynam. Meteorol. Oceanogr. 59 (5) (2007) 758, http://dx.doi.org/10.1111/j.1600-0870.2007.00261.x.

[27] A.C. Lorenc, N.E. Bowler, A.M. Clayton, S.R. Pring, D. Fairbairn, Comparison of Hybrid-4DEnVar and Hybrid-4DVar data assimilation methods for global NWP, Mon. Weather Rev. 143 (1) (2015) 212–229, http://dx.doi.org/10.1175/mwr-d-14-00195.1.

[28] T. Lin, B. Horne, P. Tino, C. Giles, Learning long-term dependencies in NARX recurrent neural networks, IEEE Trans. Neural Netw. 7 (6) (1996) 1329–1338, http://dx.doi.org/10.1109/72.548162.

[29] J.A.K. Suykens, J.P.L. Vandewalle, B.L.R.D. Moor, Artificial Neural Networks for Modelling and Control of Non-Linear Systems, Springer, US, 1996, http://dx.doi.org/10.1007/978-1-4757-2493-6.

[30] I. Goodfellow, Y. Bengio, A. Courville, Y. Bengio, Deep learning, vol. 1, (no. 2) MIT press Cambridge, 2016.

[31] C.E. Rasmussen, C.K.I. Williams, Gaussian Processes for Machine Learning, MIT Press Ltd, 2005.

[32] M. Norgaard, O. Ravn, N. Poulsen, L. Hansen, Neural networks for modelling and control of dynamic systems, in: Advanced Textbooks in Control and Signal Processing, Springer, London, 2000.

[33] K.J. Astrom, B. Wittenmark, Adaptive Control, second ed., Prentice Hall, 1994.

[34] S.S. Sastry, A. Isidori, Adaptive control of linearizable systems, IEEE Trans. Autom. Control 34 (11) (1989) 1123–1131.

[35] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.

[36] D. Bertsekas, Reinforcement Learning and Optimal Control, Athena Scientific, 2019.

[37] M.L. Puterman, Markov Decision Processes, Wiley, 1994, http://dx.doi.org/10.1002/9780470316887.

[38] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, Model-based reinforcement learning: A survey, 2022, http://dx.doi.org/10.48550/arXiv.2006.16712, arXiv:2006.16712, http://arxiv.org/abs/2006.16712 [cs, stat].

[39] F.-M. Luo, T. Xu, H. Lai, X.-H. Chen, W. Zhang, Y. Yu, A survey on model-based reinforcement learning, 2022-06-19, http://dx.doi.org/10.48550/arXiv.2206.09328, arXiv:2206.09328, http://arxiv.org/abs/2206.09328 [cs].

[40] M. Schwenzer, M. Ay, T. Bergs, D. Abel, Review on model predictive control: An engineering perspective, Int. J. Adv. Manuf. Technol. 117 (5–6) (2021) 1327–1349, http://dx.doi.org/10.1007/s00170-021-07682-3, URL: https://link.springer.com/10.1007/s00170-021-07682-3.

[41] I. Szita, Reinforcement learning in games, in: Adaptation, Learning, and Optimization, Springer Berlin Heidelberg, 2012, pp. 539–577.

[42] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013, arXiv:1312.5602.

[43] V. Uc-Cetina, N. Navarro-Guerrero, A. Martin-Gonzalez, C. Weber, S. Wermter, Survey on reinforcement learning for language processing, Artif. Intell. Rev. 56 (2) (2022) 1543–1575, http://dx.doi.org/10.1007/s10462-022-10205-5.

[44] S. Werner, S. Peitz, Learning a model is paramount for sample efficiency in reinforcement learning control of PDEs, 2023, arXiv:2302.07160.

[45] F. Pino, L. Schena, J. Rabault, M.A. Mendez, Comparative analysis of machine learning methods for active flow control, J. Fluid Mech. 958 (2023) http://dx.doi.org/10.1017/jfm.2023.76.

[46] A. Carrassi, M. Bocquet, L. Bertino, G. Evensen, Data assimilation in the geosciences - An overview on methods, issues and perspectives, 2017, arXiv:1709.02798.

[47] S. Cheng, C. Quilodran-Casas, S. Ouala, A. Farchi, C. Liu, P. Tandeo, R. Fablet, D. Lucor, B. Iooss, J. Brajard, D. Xiao, T. Janjic, W. Ding, Y. Guo, A. Carrassi, M. Bocquet, R. Arcucci, Machine learning with data assimilation and uncertainty quantification for dynamical systems: A review, 2023, URL: http://arxiv.org/abs/2303.10462, arXiv:2303.10462 [cs].

[48] A. Geer, Learning earth system models from observations: Machine learning or data assimilation? Phil. Trans. R. Soc. A 379 (2194) (2021) http://dx.doi.org/10.1098/rsta.2020.0089.

[49] H. Abarbanel, P. Rozdeba, S. Shirman, Machine learning, deepest learning: Statistical data assimilation problems, 2017, arXiv:1707.01415.

[50] Y. Tang, W.W. Hsieh, Coupling neural networks to incomplete dynamical systems via variational data assimilation, Mon. Weather Rev. 129 (4) (2001) 818–834, http://dx.doi.org/10.1175/1520-0493(2001)129<0818:cnntid>2.0.co;2.

[51] R. Arcucci, J. Zhu, S. Hu, Y.-K. Guo, Deep data assimilation: Integrating deep learning with data assimilation, Appl. Sci. 11 (3) (2021) 1114, http://dx.doi.org/10.3390/app11031114.

[52] C. Buizza, C.Q. Casas, P. Nadler, J. Mack, S. Marrone, Z. Titus, C.L. Cornec, E. Heylen, T. Dur, L.B. Ruiz, C. Heaney, J.A.D. Lopez, K.S. Kumar, R. Arcucci, Data learning: Integrating data assimilation and machine learning, J. Comput. Sci. 58 (2022) 101525, http://dx.doi.org/10.1016/j.jocs.2021.101525.

[53] P. Madhavan, Recurrent neural network for time series prediction, in: Proceedings of the 15th Annual International Conference of the IEEE Engineering in Medicine and Biology Society, IEEE, 1993, http://dx.doi.org/10.1109/iembs.1993.978527.

[54] J. Brajard, A. Carassi, M. Bocquet, L. Bertino, Combining data assimilation and machine learning to emulate a dynamical model from sparse and noisy observations: A case study with the Lorenz 96 model, J. Comput. Sci. 44 (2020) 101171, http://dx.doi.org/10.1016/j.jocs.2020.101171, URL: http://arxiv.org/abs/2001.01520, arXiv:2001.01520 [physics, stat].

[55] P. Khandelwal, P. Nadler, R. Arcucci, W. Knottenbelt, Y.-K. Guo, A scalable inference method for large dynamic economic systems, 2021, arXiv:2110.14346.

[56] P. Nadler, R. Arcucci, Y. Guo, A neural SIR model for global forecasting, in: E. Alsentzer, M.B.A. McDermott, F. Falck, S.K. Sarkar, S. Roy, S.L. Hyland (Eds.), Proceedings of the Machine Learning for Health NeurIPS Workshop, in: Proceedings of Machine Learning Research, vol. 136, PMLR, 2020, pp. 254–266, URL: https://proceedings.mlr.press/v136/nadler20a.html.

[57] M. Bocquet, J. Brajard, A. Carrassi, L. Bertino, Data assimilation as a deep learning tool to infer ODE representations of dynamical models, 2019.

[58] I. Ayed, E. de Bézenac, A. Pajot, J. Brajard, P. Gallinari, Learning dynamical systems from partial observations, 2019, URL: http://arxiv.org/abs/1902.11136, arXiv:1902.11136 [physics].

[59] L. Ljung, C.R. Andersson, K. Tiels, T.B. Schön, Deep learning and system identification, IFAC-PapersOnLine (2020) URL: https://api.semanticscholar.org/CorpusID:226118683.

[60] G. Pillonetto, A. Aravkin, D. Gedon, L. Ljung, A.H. Ribeiro, T.B. Schön, Deep networks for system identification: A survey, 2023, arXiv:2301.12832.

[61] S. Chen, S. Billings, P. Grant, Non-linear system identification using neural networks, Internat. J. Control 51 (6) (1990) 1191–1214, http://dx.doi.org/10.1080/00207179008934126.

[62] C. Zhang, K.L. Moore, System identification using neural networks, in: [1991] Proceedings of the 30th IEEE Conference on Decision and Control, vol. 1, 1991, pp. 873–874, URL: https://api.semanticscholar.org/CorpusID:60798054.

[63] J. Sjöberg, H. Hjalmarsson, L. Ljung, Neural networks in system identification, IFAC Proc. Vol. 27 (8) (1994) 359–382, http://dx.doi.org/10.1016/s1474-6670(17)47737-8.

[64] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780, http://dx.doi.org/10.1162/neco.1997.9.8.1735.

[65] H. Jaeger, H. Haas, Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication, Science 304 (5667) (2004) 78–80, http://dx.doi.org/10.1126/science.1091277.

[66] Y. LeCun, B. Boser, J.S. Denker, D. Henderson, R.E. Howard, W. Hubbard, L.D. Jackel, Backpropagation applied to handwritten zip code recognition, Neural Comput. 1 (4) (1989) 541–551, http://dx.doi.org/10.1162/neco.1989.1.4.541.

[67] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, in: Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS '18, Curran Associates Inc., Red Hook, NY, USA, 2018, pp. 6572–6583.

[68] J. Gonzalez, W. Yu, Non-linear system modeling using LSTM neural networks, IFAC-PapersOnLine 51 (13) (2018) 485–489, http://dx.doi.org/10.1016/j.ifacol.2018.07.326.

[69] M.A. Bucci, O. Semeraro, A. Allauzen, L. Cordier, G. Wisniewski, L. Mathelin, Control-oriented model learning with a recurrent neural network, Bull. Am. Phys. Soc. 63 (2018).

[70] D. Canaday, A. Pomerance, D.J. Gauthier, Model-free control of dynamical systems with deep reservoir computing, 2020, arXiv:2010.02285.

[71] C. Andersson, A.H. Ribeiro, K. Tiels, N. Wahlstrom, T.B. Schon, Deep convolutional networks in system identification, in: 2019 IEEE 58th Conference on Decision and Control, CDC, IEEE, 2019, http://dx.doi.org/10.1109/cdc40024.2019.9030219.

[72] A. Rahman, J. Drgoňa, A. Tuor, J. Strube, Neural ordinary differential equations for nonlinear system identification, 2022, arXiv:2203.00120.

[73] M.A. Bucci, O. Semeraro, A. Allauzen, S. Chibbaro, L. Mathelin, Curriculum learning for data-driven modeling of dynamical systems, Eur. Phys. J. E 46 (3) (2023) http://dx.doi.org/10.1140/epje/s10189-023-00269-8.

[74] K. Chatzilygeroudis, V. Vassiliades, F. Stulp, S. Calinon, J.-B. Mouret, A survey on policy search algorithms for learning robot controllers in a handful of trials, 2019, arXiv:1807.02303.

[75] F.-M. Luo, T. Xu, H. Lai, X.-H. Chen, W. Zhang, Y. Yu, A survey on model-based reinforcement learning, 2022, arXiv:2206.09328.

[76] T.M. Moerland, J. Broekens, A. Plaat, C.M. Jonker, Model-based reinforcement learning: A survey, 2022, arXiv:2006.16712.

[77] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018, arXiv:1801.01290.

[78] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2019, arXiv:1509.02971.

[79] K. Hunt, D. Sbarbaro, R. Żbikowski, P. Gawthrop, Neural networks for control systems—A survey, Automatica 28 (6) (1992) 1083–1112, http://dx.doi.org/10.1016/0005-1098(92)90053-I, URL: https://www.sciencedirect.com/science/article/pii/000510989290053I.

[80] T. Kurutach, I. Clavera, Y. Duan, A. Tamar, P. Abbeel, Model-ensemble trust-region policy optimization, 2018, arXiv preprint arXiv:1802.10592.

[81] A. Nagabandi, G. Kahn, R.S. Fearing, S. Levine, Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning, 2017, arXiv:1708.02596.

[82] M. Deisenroth, C. Rasmussen, PILCO: A model-based and data-efficient approach to policy search, 2011, pp. 465–472.

[83] J. Boedecker, J.T. Springenberg, J. Wülfing, M. Riedmiller, Approximate real-time optimal control based on sparse Gaussian process models, in: 2014 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, ADPRL, 2014, pp. 1–8, http://dx.doi.org/10.1109/ADPRL.2014.7010608.

[84] J. Boedecker, J.T. Springenberg, J. Wülfing, M. Riedmiller, Approximate real-time optimal control based on sparse Gaussian process models, in: 2014 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, ADPRL, 2014, pp. 1–8, http://dx.doi.org/10.1109/ADPRL.2014.7010608.

[85] M. Janner, J. Fu, M. Zhang, S. Levine, When to trust your model: Model-based policy optimization, 2021, arXiv:1906.08253.

[86] S. Gu, T. Lillicrap, I. Sutskever, S. Levine, Continuous deep Q-learning with model-based acceleration, 2016, arXiv:1603.00748.

[87] R.S. Sutton, Dyna, an integrated architecture for learning, planning, and reacting, SIGART Bull. 2 (4) (1991) 160–163, http://dx.doi.org/10.1145/122344.122377.

[88] D. Silver, A. Huang, C.J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, D. Hassabis, Mastering the game of Go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.

[89] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, D. Hassabis, A general reinforcement learning algorithm that masters Chess, Shogi, and go through self-play, Science 362 (6419) (2018) 1140–1144.

[90] J.D. Hedengren, R.A. Shishavan, K.M. Powell, T.F. Edgar, Nonlinear modeling, estimation and predictive control in APMonitor, Comput. Chem. Eng. 70 (2014) 133–148, http://dx.doi.org/10.1016/j.compchemeng.2014.04.013, URL: https://linkinghub.elsevier.com/retrieve/pii/S0098135414001306.

[91] Tutorial overview of model predictive control, IEEE Control Syst. 20 (3) (2000) 38–52, http://dx.doi.org/10.1109/37.845037, URL: https://ieeexplore.ieee.org/document/845037/.

[92] T. Weber, S. Racanière, D.P. Reichert, L. Buesing, A. Guez, D.J. Rezende, A.P. Badia, O. Vinyals, N. Heess, Y. Li, R. Pascanu, P. Battaglia, D. Hassabis, D. Silver, D. Wierstra, Imagination-augmented agents for deep reinforcement learning, 2017, arXiv:1707.06203.

[93] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.

[94] X.-Y. Liu, J.-X. Wang, Physics-informed dyna-style model-based deep reinforcement learning for dynamic control, 2021, http://dx.doi.org/10.1098/rspa.2021.0618, arXiv:2108.00128.

[95] M. Lutter, C. Ritter, J. Peters, Deep Lagrangian networks: Using physics as model prior for deep learning, 2019, arXiv:1907.04490.

[96] A. Ramesh, B. Ravindran, Physics-informed model-based reinforcement learning, 2023, arXiv:2212.02179.

[97] M. Lutter, J. Silberbauer, J. Watson, J. Peters, Differentiable physics models for real-world offline model-based reinforcement learning, 2020, arXiv:2011.01734.

[98] X. Liu, J.F. MacArt, Adjoint-based machine learning for active flow control, 2023, arXiv:2307.09980.

[99] N. Baker, F. Alexander, T. Bremer, A. Hagberg, Y. Kevrekidis, H. Najm, M. Parashar, A. Patra, J. Sethian, S. Wild, K. Willcox, S. Lee, Workshop report on basic research needs for scientific machine learning: Core technologies for artificial intelligence, 2019, http://dx.doi.org/10.2172/1478744, URL: https://www.osti.gov/biblio/1478744.

[100] J. Willard, X. Jia, S. Xu, M. Steinbach, V. Kumar, Integrating scientific knowledge with machine learning for engineering and environmental systems, 2020, http://dx.doi.org/10.48550/ARXIV.2003.04919.

[101] R.F. Stengel, Optimal Control and Estimation, Courier Corporation, 1994.

[102] A. Pinosky, I. Abraham, A. Broad, B. Argall, T.D. Murphey, Hybrid control for combining model-based and model-free reinforcement learning, Int. J. Robot. Res. 42 (6) (2023) 337–355, http://dx.doi.org/10.1177/02783649221083331.

[103] G. Dulac-Arnold, D. Mankowitz, T. Hester, Challenges of real-world reinforcement learning, 2019, arXiv:1904.12901.

[104] S. Yamada, A. Watanabe, M. Nakashima, Hybrid reinforcement learning and its application to biped robot control, in: M. Jordan, M. Kearns, S. Solla (Eds.), in: Advances in Neural Information Processing Systems, vol. 10, MIT Press, 1997, URL: https://proceedings.neurips.cc/paper_files/paper/1997/file/7895fc13088ee37f511913bac71fa66f-Paper.pdf.

[105] M. Jordan, R. Jacobs, Hierarchical mixtures of experts and the, Neural Comput. 6 (1994) 181.

[106] C.K. Tham, Reinforcement learning of multiple tasks using a hierarchical CMAC architecture, Robot. Auton. Syst. 15 (4) (1995) 247–274, http://dx.doi.org/10.1016/0921-8890(95)00005-Z, URL: https://www.sciencedirect.com/science/article/pii/092188909500005Z, Reinforcement Learning and Robotics.

[107] B. Freed, T. Wei, R. Calandra, J. Schneider, H. Choset, Unifying model-based and model-free reinforcement learning with equivalent policy sets, 2024, URL: https://openreview.net/forum?id=p5SurcLh24.

[108] Ç. Yıldız, M. Heinonen, H. Lähdesmäki, Continuous-time model-based reinforcement learning, 2021, arXiv:2102.04764.

[109] M. Lutter, B. Belousov, S. Mannor, D. Fox, A. Garg, J. Peters, Continuous-time fitted value iteration for robust policies, 2021, arXiv:2110.01954.

[110] K. Doya, Reinforcement learning in continuous time and space, Neural Comput. 12 (1) (2000) 219–245.

[111] Y. Tassa, T. Erez, E. Todorov, Synthesis and stabilization of complex behaviors through online trajectory optimization, in: 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2012-10, pp. 4906–4913, http://dx.doi.org/10.1109/IROS.2012.6386025, URL: http://ieeexplore.ieee.org/document/6386025/.

[112] J. Van Den Berg, Iterated LQR smoothing for locally-optimal feedback control of systems with non-linear dynamics and non-quadratic cost, in: 2014 American Control Conference, IEEE, 2014-06, pp. 1912–1918, http://dx.doi.org/10.1109/ACC.2014.6859404, URL: http://ieeexplore.ieee.org/document/6859404/.

[113] S. Levine, V. Koltun, Guided policy search, in: International Conference on Machine Learning, PMLR, 2013, pp. 1–9.

[114] S. Levine, P. Abbeel, Learning neural network policies with guided policy search under unknown dynamics, in: Advances in Neural Information Processing Systems, vol. 27, 2014.

[115] Y. Chebotar, K. Hausman, M. Zhang, G. Sukhatme, S. Schaal, S. Levine, Combining model-based and model-free updates for trajectory-centric reinforcement learning, 2017, arXiv:1703.03078.

[116] G. Kahn, T. Zhang, S. Levine, P. Abbeel, Plato: Policy learning using adaptive trajectory optimization, in: 2017 IEEE International Conference on Robotics and Automation, ICRA, IEEE, 2017, pp. 3342–3349.

[117] G. Qu, C. Yu, S. Low, A. Wierman, Combining model-based and model-free methods for nonlinear control: A provably convergent policy gradient approach, 2020, arXiv:2006.07476.

[118] P. Abbeel, M. Quigley, A.Y. Ng, Using inaccurate models in reinforcement learning, in: Proceedings of the 23rd International Conference on Machine Learning, ICML '06, ACM Press, Pittsburgh, Pennsylvania, 2006, pp. 1–8, http://dx.doi.org/10.1145/1143844.1143845, URL: http://portal.acm.org/citation.cfm?doid=1143844.1143845.

[119] V. Pong, S. Gu, M. Dalal, S. Levine, Temporal difference models: Model-free deep RL for model-based control, 2020, arXiv:1802.09081.

[120] N. Heess, G. Wayne, D. Silver, T. Lillicrap, Y. Tassa, T. Erez, Learning continuous control policies by stochastic value gradients, 2015, arXiv:1510.09142.

[121] V. Feinberg, A. Wan, I. Stoica, M.I. Jordan, J.E. Gonzalez, S. Levine, Model-based value estimation for efficient model-free reinforcement learning, 2018, arXiv:1803.00101.

[122] F. Berkenkamp, M. Turchetta, A.P. Schoellig, A. Krause, Safe model-based reinforcement learning with stability guarantees, 2017, arXiv:1705.08551.

[123] S.M. Richards, F. Berkenkamp, A. Krause, The Lyapunov neural network: Adaptive stability certification for safe learning of dynamical systems, in: Conference on Robot Learning, PMLR, 2018, pp. 466–476.

[124] M. Han, Y. Tian, L. Zhang, J. Wang, W. Pan, $ H_\infty $ Model-free Reinforcement Learning with Robust Stability Guarantee, 2020-07-25, arXiv:1911.02875, URL: http://arxiv.org/abs/1911.02875.

[125] S. Bhatnagar, R.S. Sutton, M. Ghavamzadeh, M. Lee, Natural actor–critic algorithms, Automatica 45 (11) (2009) 2471–2482, http://dx.doi.org/10.1016/j.automatica.2009.07.008, URL: https://linkinghub.elsevier.com/retrieve/pii/S0005109809003549.

[126] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, 2015, http://dx.doi.org/10.48550/ARXIV.1511.05952.

[127] M.A. Bucci, O. Semeraro, A. Allauzen, G. Wisniewski, L. Cordier, L. Mathelin, Control of chaotic systems by deep reinforcement learning, Proc. R. Soc. A 475 (2231) (2019) 20190351.

[128] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, M. Riedmiller, Deterministic policy gradient algorithms, 2014.

[129] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013, arXiv:http://arxiv.org/abs/1312.5602v1.

[130] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533, http://dx.doi.org/10.1038/nature14236.

[131] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, 2016, arXiv:1511.05952.

[132] R.M. Errico, What is an adjoint model? Bull. Am. Meteorol. Soc. 78 (11) (1997).

[133] A.M. Bradley, PDE-constrained optimization and the adjoint method, 2019.

[134] Y. Cao, S. Li, L. Petzold, R. Serban, Adjoint sensitivity analysis for differential-algebraic equations: The adjoint DAE system and its numerical solution, SIAM J. Sci. Comput. 24 (3) (2003) 1076–1089, http://dx.doi.org/10.1137/s1064827501380630.

[135] A.J. Smola, B. Schölkopf, A tutorial on support vector regression, Stat. Comput. 14 (3) (2004) 199–222, http://dx.doi.org/10.1023/b:stco.0000035301.49549.88.

[136] C.-C. Chang, C.-J. Lin, LIBSVM, ACM Trans. Intell. Syst. Technol. 2 (3) (2011) 1–27, http://dx.doi.org/10.1145/1961189.1961199.

[137] H. Zhang, E.M. Constantinescu, Optimal checkpointing for adjoint multistage time-stepping schemes, J. Comput. Sci. 66 (2023) 101913, http://dx.doi.org/10.1016/j.jocs.2022.101913.

[138] J.H. Laks, L.Y. Pao, A.D. Wright, Control of wind turbines: Past, present, and future, in: 2009 American Control Conference, IEEE, 2009, http://dx.doi.org/10.1109/acc.2009.5160590.

[139] L.Y. Pao, K.E. Johnson, A tutorial on the dynamics and control of wind turbines and wind farms, in: 2009 American Control Conference, IEEE, 2009, http://dx.doi.org/10.1109/acc.2009.5160195.

[140] F.D. Bianchi, H. De Battista, R.J. Mantz, Wind Turbine Control Systems: Principles, Modelling and Gain Scheduling Design, vol. 19, Springer, 2007.

[141] N.J. Abbas, A. Wright, L. Pao, An update to the national renewable energy laboratory baseline wind turbine controller, J. Phys.: Conf. Ser. 1452 (1) (2020) 012002.

[142] S. Bhowmik, R. Spee, Performance optimization for doubly fed wind power generation systems, IEEE Trans. Ind. Appl. 35 (4) (1999).

[143] A.M. Howlader, N. Urasaki, K. Uchida, A. Yona, T. Senjyu, C.-H. Kim, A.Y. Saber, Parameter identification of wind turbine for maximum power-point tracking control, Electr. Power Compon. Syst. 38 (5) (2010) 603–614, http://dx.doi.org/10.1080/15325000903376974, URL: http://www.tandfonline.com/doi/abs/10.1080/15325000903376974.

[144] J. Jonkman, S. Butterfield, W. Musial, G. Scott, Definition of a 5-MW Reference Wind Turbine for Offshore System Development, Technical Report, National Renewable Energy Lab.(NREL), Golden, CO (United States), 2009.

[145] X. Zheng, T. Jin, A reliable method of wind power fluctuation smoothing strategy based on multidimensional non-linear exponential smoothing short-term forecasting, IET Renew. Power Gener. 16 (16) (2022) 3573–3586, http://dx.doi.org/10.1049/rpg2.12395.

[146] E.A. Bossanyi, The Design of closed loop controllers for wind turbines, Wind Energ. 3 (3) (2000) 149–163, http://dx.doi.org/10.1002/we.34, URL: https://onlinelibrary.wiley.com/doi/10.1002/we.34.

[147] P.J. Moriarty, A.C. Hansen, AeroDyn Theory Manual, Technical Report, National Renewable Energy Lab., Golden, CO (US), 2005.

[148] G. De Cillis, S. Cherubini, O. Semeraro, S. Leonardi, P. De Palma, The influence of incoming turbulence on the dynamic modes of an NREL-5MW wind turbine wake, Renew. Energy 183 (2022) 601–616.

[149] G. De Cillis, O. Semeraro, S. Leonardi, P. De Palma, S. Cherubini, Dynamic-mode-decomposition of the wake of the NREL-5MW wind turbine impinged by a laminar inflow, Renew. Energy 199 (2022) 1–10.

[150] M. Coquelet, L. Bricteux, M. Moens, P. Chatelain, A reinforcement-learning approach for individual pitch control, Wind Energy 25 (8) (2022) 1343–1362.

[151] B.J. Jonkman, TurbSim User's Guide, Technical Report, National Renewable Energy Lab.(NREL), Golden, CO (United States), 2006.

[152] F. Pimenta, J. Pacheco, C.M. Branco, C.M. Teixeira, F. Magalhães, Development of a digital twin of an onshore wind turbine using monitoring data, J. Phys. Conf. Ser. 1618 (2) (2020) 022065, http://dx.doi.org/10.1088/1742-6596/1618/2/022065.

[153] E. Branlard, J. Jonkman, C. Brown, J. Zang, A digital-twin solution for floating offshore wind turbines validated using a full-scale prototype, Wind Energy Sci. Discuss. 2023 (2023) 1–34, http://dx.doi.org/10.5194/wes-2023-50, URL: https://wes.copernicus.org/preprints/wes-2023-50/.

[154] O.O. Olatunji, P.A. Adedeji, N. Madushele, T.C. Jen, Overview of digital twin technology in wind turbine fault diagnosis and condition monitoring, in: 2021 IEEE 12th International Conference on Mechanical and Intelligent Manufacturing Technologies, ICMIMT, 2021, pp. 201–207, URL: https://api.semanticscholar.org/CorpusID:236190314.

[155] M. Fahim, V. Sharma, T.-V. Cao, B. Canberk, T.Q. Duong, Machine learning-based digital twin for predictive modeling in wind turbines, IEEE Access 10 (2022) 14184–14194, URL: https://api.semanticscholar.org/CorpusID:246420062.

[156] J.G. Leishman, Challenges in modelling the unsteady aerodynamics of wind turbines, Wind Energy: Int. J. Prog. Appl. Wind Power Convers. Technol. 5 (2–3) (2002) 85–132.

[157] I. Staffell, R. Green, How does wind farm performance decline with age? Renew. Energy 66 (2014) 775–786, http://dx.doi.org/10.1016/j.renene.2013.10.041, URL: https://www.sciencedirect.com/science/article/pii/S0960148113005727.

[158] K.E. Johnson, Adaptive Torque Control of Variable Speed Wind Turbines, University of Colorado at Boulder, 2004.

[159] K.E. Johnson, Control of variable-speed wind turbines: Standard and adaptive techniques for maximizing energy capture, IEEE Control Syst. Mag. 26 (3) (2006) 70–81.

[160] Y.-M. Saint-Drenan, R. Besseau, M. Jansen, I. Staffell, A. Troccoli, L. Dubus, J. Schmidt, K. Gruber, S.G. Simões, S. Heier, A parametric model for wind turbine power curves incorporating environmental conditions, Renew. Energy 157 (2020) 754–768, http://dx.doi.org/10.1016/j.renene.2020.04.123, URL: https://www.sciencedirect.com/science/article/pii/S0960148120306613.

[161] N. Haider, A. Shahzad, M.N.M. Qadri, S.I.A. Shah, Recent progress in flapping wings for micro aerial vehicle applications, Proc. Inst. Mech. Eng. C 235 (2) (2020) 245–264, http://dx.doi.org/10.1177/0954406220917426.

[162] B. Cheng, B.W. Tobalske, D.R. Powers, T.L. Hedrick, S.M. Wethington, G.T. Chiu, X. Deng, Flight mechanics and control of escape manoeuvres in hummingbirds. I. Flight kinematics, J. Exp. Biol. 219 (22) (2016) 3518–3531.

[163] V.M. Ortega-Jiménez, R. Dudley, Ascending flight and decelerating vertical glides in Anna's hummingbirds, J. Exp. Biol. 221 (24) (2018) jeb191171.

[164] F. Fei, Z. Tu, Y. Yang, J. Zhang, X. Deng, Flappy hummingbird: An open source dynamic simulation of flapping wing robots and animals, in: 2019 International Conference on Robotics and Automation, ICRA, IEEE, 2019, pp. 9223–9229.

[165] Y. Xue, X. Cai, R. Xu, H. Liu, Wing kinematics-based flight control strategy in insect-inspired flight systems: Deep reinforcement learning gives solutions and inspires controller design in flapping MAVs, Biomimetics 8 (3) (2023) 295.

[166] J.P. Whitney, R.J. Wood, Aeromechanics of passive rotation in flapping flight, J. Fluid Mech. 660 (2010) 197–220.

[167] X. Cai, D. Kolomenskiy, T. Nakata, H. Liu, A CFD data-driven aerodynamic model for fast and precise prediction of flapping aerodynamics in various flight velocities, J. Fluid Mech. 915 (2021) A114.

[168] J.W. Kruyt, E.M. Quicazán-Rubio, G.F. Van Heijst, D.L. Altshuler, D. Lentink, Hummingbird wing efficacy depends on aspect ratio and compares with helicopter rotors, J. R. Soc. Interface 11 (99) (2014) 20140585.

[169] H.E. Taha, M.R. Hajj, A.H. Nayfeh, Flight dynamics and control of flapping-wing MAVs: A review, Nonlinear Dynam. 70 (2012) 907–939.

[170] Y. Lee, K.-B. Lua, T. Lim, K. Yeo, A quasi-steady aerodynamic model for flapping flight with improved adaptability, Bioinspiration Biomimetics 11 (3) (2016) 036005.

[171] M.H. Dickinson, F.-O. Lehmann, S.P. Sane, Wing rotation and the aerodynamic basis of insect flight, Science 284 (5422) (1999) 1954–1960.

[172] S.P. Sane, M.H. Dickinson, The control of flight force by a flapping wing: Lift and drag production, J. Exp. Biol. 204 (15) (2001) 2607–2626.

[173] S.P. Sane, The aerodynamics of insect flight, J. Exp. Biol. 206 (23) (2003) 4191–4208.

[174] J.A. Salzman, Fluid management in space-based systems, in: Engineering, Construction, and Operations in Space V, American Society of Civil Engineers, 1996, http://dx.doi.org/10.1061/40177(207)71.

[175] S. Motil, M. Meyer, S. Tucker, Cryogenic fluid management technologies for advanced green propulsion systems, in: 45th AIAA Aerospace Sciences Meeting and Exhibit, American Institute of Aeronautics and Astronautics, 2007, http://dx.doi.org/10.2514/6.2007-343.

[176] P.R. Chai, A.W. Wilhite, Cryogenic thermal system analysis for orbital propellant depot, Acta Astronaut. 102 (2014) 35–46, http://dx.doi.org/10.1016/j.actaastro.2014.05.013, URL: https://linkinghub.elsevier.com/retrieve/pii/S0094576514001738.

[177] S. Mer, J.-P. Thibault, C. Corre, Active insulation technique applied to the experimental analysis of a thermodynamic control system for cryogenic propellant storage, J. Therm. Sci. Eng. Appl. 8 (2) (2016) http://dx.doi.org/10.1115/1.4032761.

[178] W. Jiang, P. Sun, P. Li, Z. Zuo, Y. Huang, Transient thermal behavior of multi-layer insulation coupled with vapor cooled shield used for liquid hydrogen storage tank, Energy 231 (2021) 120859, http://dx.doi.org/10.1016/j.energy.2021.120859.

[179] C. Lin, N. Dresar, M. Hasan, Pressure control analysis of cryogenic storage systems, J. Propuls. Power 20 (1991) http://dx.doi.org/10.2514/1.10387.

[180] R. Imai, K. Nishida, O. Kawanami, Y. Umemura, T. Himeno, Ground based experiment and numerical calculation on thermodynamic vent system in propellant tank for future cryogenic propulsion system, Cryogenics 109 (2020) 103095, http://dx.doi.org/10.1016/j.cryogenics.2020.103095.

[181] X. Qin, P. Li, P. Sun, Y. Huang, Testing and comparison of a thermodynamic vent system operating in different modes in a liquid nitrogen tank, Appl. Therm. Eng. 197 (2021) 117393, http://dx.doi.org/10.1016/j.applthermaleng.2021.117393.

[182] L. Hastings, S. Tucker, R. Flachbart, A. Hedayat, S. Nelson, Marshall space flight center in-space cryogenic fluid management program overview, in: 41st AIAA/ASME/SAE/ASEE Joint Propulsion Conference and Exhibit, American Institute of Aeronautics and Astronautics, 2005, http://dx.doi.org/10.2514/6.2005-3561.

[183] B. Wang, Y. Huang, J. Wu, T. Wang, G. Lei, Experimental study on pressure control of liquid nitrogen tank by thermodynamic vent system, Appl. Therm. Eng. 125 (2017) 1037–1046, http://dx.doi.org/10.1016/j.applthermaleng.2017.07.067, URL: https://www.sciencedirect.com/science/article/pii/S1359431117315491.

[184] L.J. Hastings, R. Flachbart, J. Martin, A. Hedayat, M. Fazah, T. Lak, H. Nguyen, J. Bailey, Spray bar zero-gravity vent system for on-orbit liquid hydrogen storage, Technical Memorandum (TM) NASA/TM-2003-212926, National Aeronautics and Space Administration, Marshall Space Flight Center, Alabama 35812, 2003, URL: https://ntrs.nasa.gov/api/citations/20040000092/downloads/20040000092.pdf.

[185] S. Barsi, Ventless Pressure Control of Cryogenic Storage Tanks (Ph.D. thesis), CASE Western Reserve University, 2011.

[186] S. Barsi, M. Kassemi, Investigation of tank pressurization and pressure control—Part I: Experimental study, J. Therm. Sci. Eng. Appl. 5 (4) (2013) http://dx.doi.org/10.1115/1.4023891.
[187] S. Barsi, M. Kassemi, Investigation of tank pressurization and pressure control—Part II: Numerical modeling, J. Therm. Sci. Eng. Appl. 5 (4) (2013) http://dx.doi.org/10.1115/1.4023892.
[188] S. Mer, D. Fernandez, J.-P. Thibault, C. Corre, Optimal design of a thermodynamic vent system for cryogenic propellant storage, Cryogenics 80 (2016) 127–137, http://dx.doi.org/10.1016/j.cryogenics.2016.09.012.
[189] P. Marques, S. Ahizi, M.A. Mendez, Real time data assimilation for the thermodynamic modeling of a cryogenic fuel tank, in: 36th International Conference on Efficiency, Cost, Optimization, Simulation and Environmental Impact of Energy Systems, ECOS 2023, ECOS 2023, Las Palmas De Gran Canaria, Spain, 2023, pp. 1041–1052, http://dx.doi.org/10.52202/069564-0095, URL: http://www.proceedings.com/069564-0095.html.
[190] C.H. Panzarella, M. Kassemi, On the validity of purely thermodynamic descriptions of two-phase cryogenic fluid storage, J. Fluid Mech. 484 (2003) 41–68, http://dx.doi.org/10.1017/S0022112003004002, URL: http://www.journals.cambridge.org/abstract_S0022112003004002.
[191] C. Panzarella, D. Plachta, M. Kassemi, Pressure control of large cryogenic tanks in microgravity, Cryogenics 44 (6) (2004) 475–483, http://dx.doi.org/10.1016/j.cryogenics.2004.03.009, URL: https://linkinghub.elsevier.com/retrieve/pii/S0011227504000633.
[192] E.W. Lemmon, I.H. Bell, M.L. Huber, M.O. McLinden, NIST Standard Reference Database 23: Reference Fluid Thermodynamic and Transport Properties-REFPROP, Version 10.0, National Institute of Standards and Technology, 2018, http://dx.doi.org/10.18434/T4/1502528, URL: https://www.nist.gov/srd/refprop.
[193] Y. Cengel, A. Ghajar, Heat and Mass Transfer: Fundamentals and Applications, sixth ed., McGraw Hill, 2019.



L. Schena is a joint Ph.D. candidate at the von Karman Institute for Fluid Dynamics and the Vrije Universiteit Brussel (VUB). His research focuses on combining model-based and model-free control techniques for wind turbine control, using both numerical and experimental methods. He is supported by the Fonds Wetenschappelijk Onderzoek (FWO) Strategic Basic n. 1S67923N.



P.A. Marquesis a joint Ph.D. candidate at the von Karman Institute for Fluid Dynamics and the Université Libre de Bruxelles (ULB). His research focuses on developing thermodynamic models of cryogenic liquid storage in gravity-dominated and microgravity conditions by combining experimental measurements with data assimilation techniques. He is supported by 'Fonds de la Recherche Scientifique (F.R.S. - FNRS)' with the FRIA grant FC47297.



R. Poletti is a joint Ph.D. candidate at the von Karman Institute for Fluid Dynamics and Ghent University. His research focuses on high-fidelity CFD simulations and developing data-driven reduced-order models to optimize the wing kinematics of flapping-wing drones using reinforcement learning. He is supported by the Fonds Wetenschappelijk Onderzoek (FWO) Strategic Basic n. 1SD7823N.



S. Ahizi is a project engineer at the von Karman Institute for Fluid Dynamics working on the development of digital twins. His research focuses on the development of reduced-order thermodynamic models for the management of cryogenic propellant using data assimilation methods.



J. van den Berghe is a joint Ph.D. candidate at the von Karman Institute for Fluid Dynamics and the Université Catholique de Louvain. His research focuses on numerical simulations and data driven reduced order modelling of supersonic ejectors. He is supported by 'Fonds de la Recherche Scientifique (F.R.S. - FNRS)' with the FRIA grant FC47455.



M.A. Mendez is an Associate Professor at the von Karman Institute for Fluid Dynamics, where he teaches courses on modelling and control of fluid flows, measurement techniques, signal processing and machine learning. His main research activities include experimental fluid mechanics (particularly image-based velocimetry and image processing), reduced-order modelling, flow control, and machine learning, with application ranging from coating flows to aerodynamics, microgravity and cryogenic storage.